

PROIECTAREA CU MICROPROCESOARE

C3 2025-2026 Sem. 2

CURS 9 Sapt 9 22 aprilie 2026 16:00-19:00

serban@upit.ro

Regulament disciplina

Nota finala este formata din activitatile:

- Laborator 15%
- Lucrare control 15% **13 mai 2026**
- Presentare si sustinere proiect 20%
- Examen 50% **16 iunie 2026**
- Bonus prezenta – 10%

Conditii pentru promovare:

- Nota de la laborator trebuie sa fie minim 5 (prezenta obligatorie la toate sedintele de laborator);
- Nota proiect minim 5 (conditie obligatorie pentru intrare in examen);
- Nota de la examen trebuie sa fie minim 5;
- Note de minim 5 la Lucrarea de control.

In cazul reluarii disciplinei intr-un alt an universitar, activitatile nepromovate trebuie parcurse din nou.

Continut disciplina

Familia de MCU Intel 8051 (hw & sw)

Circuite programabile I/O de tip port paralel

Circuite programabile I/O de tip timer

Circuite programabile I/O de tip USART

Circuite programabile I/O pentru controlul intreruperilor

Alte circuite specializate I/O programabile

Alte familii de MCU

Realizarea microsystemelor cu MCU (hw & sw)

Wearable

Harvesting

Bibliografie

Kenneth AYALA, *The 8051 Microcontroller: Architecture, Programming and Applications*, West Publishing Company, 1991

Applied Logic Engineering, *8051 Interfacing and Applications*, 1991

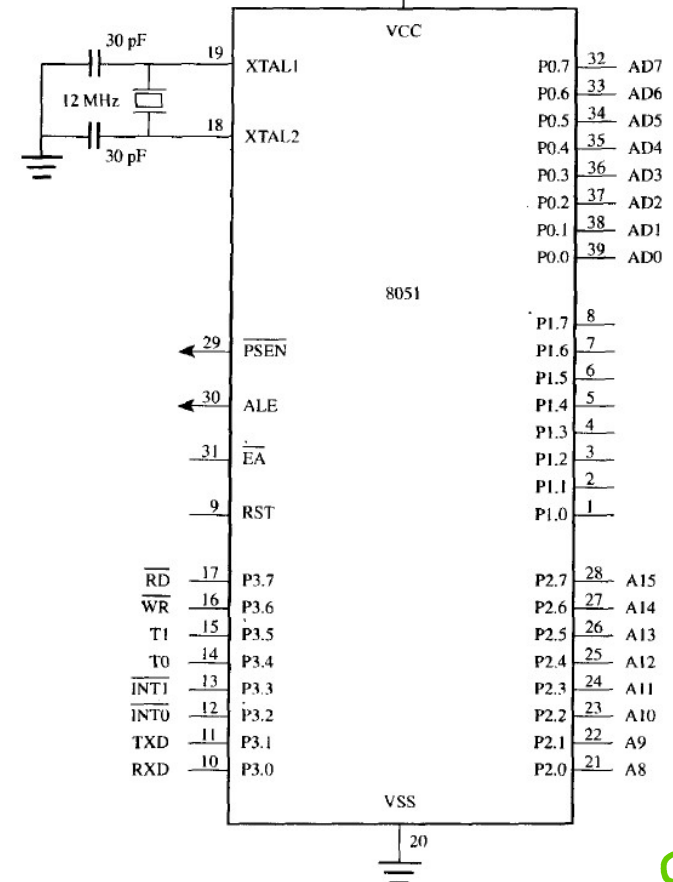
Jack GANSSLE, *The Art of designing Embedded Systems*, 2nd ed., Newnes, Elsevier, 2008

Ken ARNOLD, *Embedded Controller Hardware Design*, LLH Technology Publishing, 2001

MOSTEK, *Z80 Processor - Technical Manual*

Ramesh Gaonkar, *Z-80 Microprocessor Architecture, Interfacing, Programming and Design*, Prentice Hall, 3 ed., 2000

Documentatii de firme (Intel, Zilog)



CMOS

Marja de Zwart

$$|V_{OH} - V_{IH}| = 0,5V$$
$$|V_{IL} - V_{OL}| = 0,45V$$

Pinii XTAL₁ și XTAL₂ –intrari pe care se conectează în exterior un cristal de cuarț în ritmul căruia microcontroller-ul operează.

Pinul RESET – intrarea pentru inițializarea hardware a controller-ului; dupa RESET registrul PC este initializat cu 0000h.

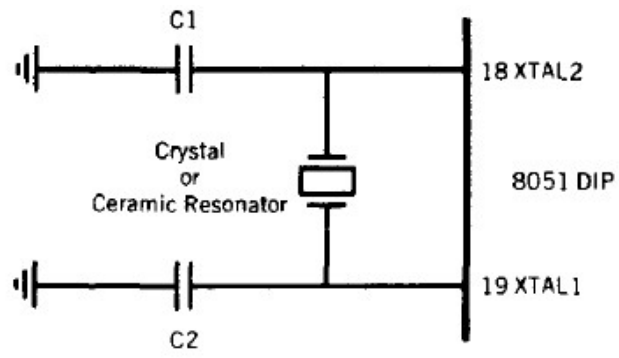
Pinul $\overline{EA}$ (external address) – este o intrare care permite configurarea controller-ului. Astfel, dacă $\overline{EA}=0$ controller-ul formează magistrale externe de adrese și date la care se pot conecta în exterior alte resurse suplimentare (circuite de memorie, circuite de I/O), iar dacă $\overline{EA}=1$ nu se formează magistrale externe și controller-ul operează doar cu resursele interne.

Pinul $\overline{PSEN}$ (PSEL) (Program Select Enable) – este o ieșire activată de controller în momentul când operează cu spațiul adreselor memoriei de programe (internă sau externă).

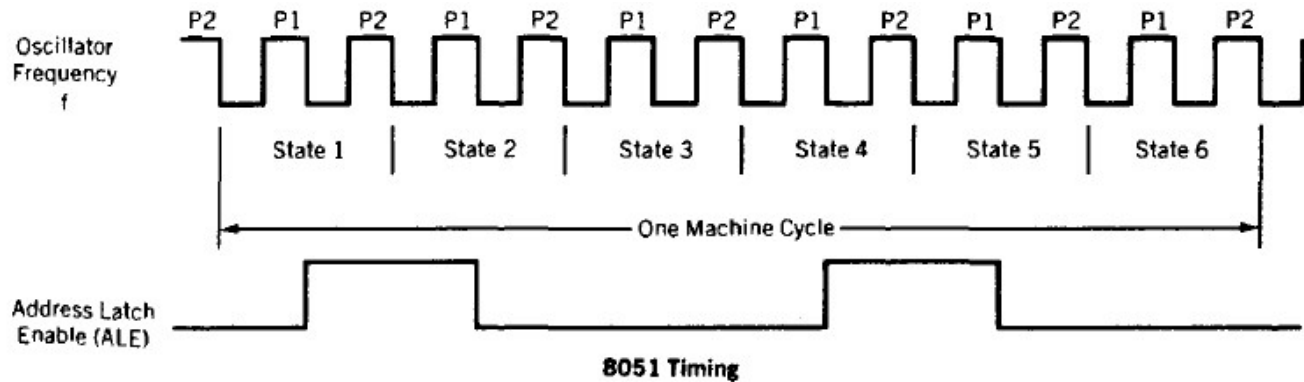
Pinul ALE (Address Latch Enable) – semnalul permite demultiplexarea magistralei comune de adrese și date generată de către procesor atunci când $\overline{EA}=0$. În această situație porturile paralele P₀ și P₂ își pierd funcționalitatea de port paralel și P₀ formează o magistrală multiplexată de adrese și date cu liniile denumite AD₀ – AD₇, iar P₂ generează octetul cel mai semnificativ (ocms) al magistralei de adrese, A₈ – A₁₅.

Pinii $\overline{RD}$ și $\overline{WR}$ – sunt semnale de ieșire care se activează la lucrul controller-ului cu spațiul adreselor memoriei externe de date, fapt care presupune că $\overline{EA}=0$; $\overline{RD}=0$ la citirea memoriei externe de date, iar $\overline{WR}=0$ la scrierea memoriei externe de date; în mod normal pinii pe care apar cele două semnale fac parte din portul paralel P3, dar își schimbă funcționalitatea când $\overline{EA}=0$. Cele două semnale nu se activează la lucrul cu memoria de programe, dar permit decodificarea externă a spațiului memoriei de date.

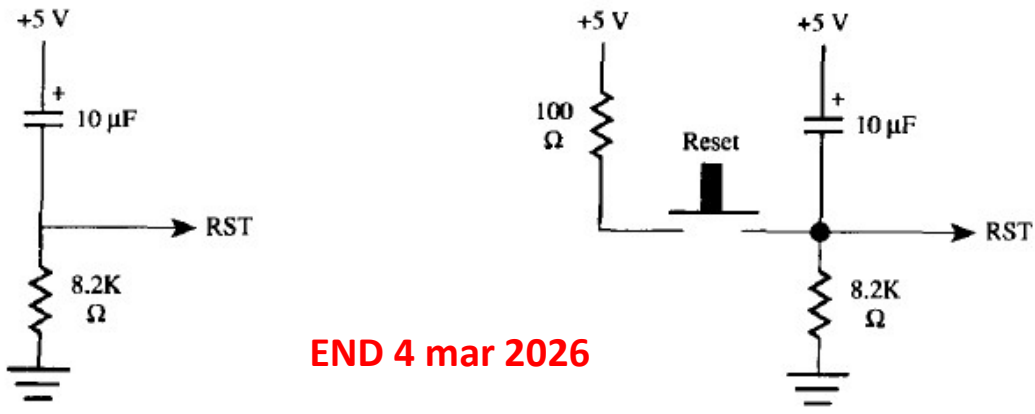
CIRCUIT EXTERN OSCILATOR LA MCU 8051



CICLU MASINA 8051



CIRCUITE EXTERNE DE RESET LA MCU 8051

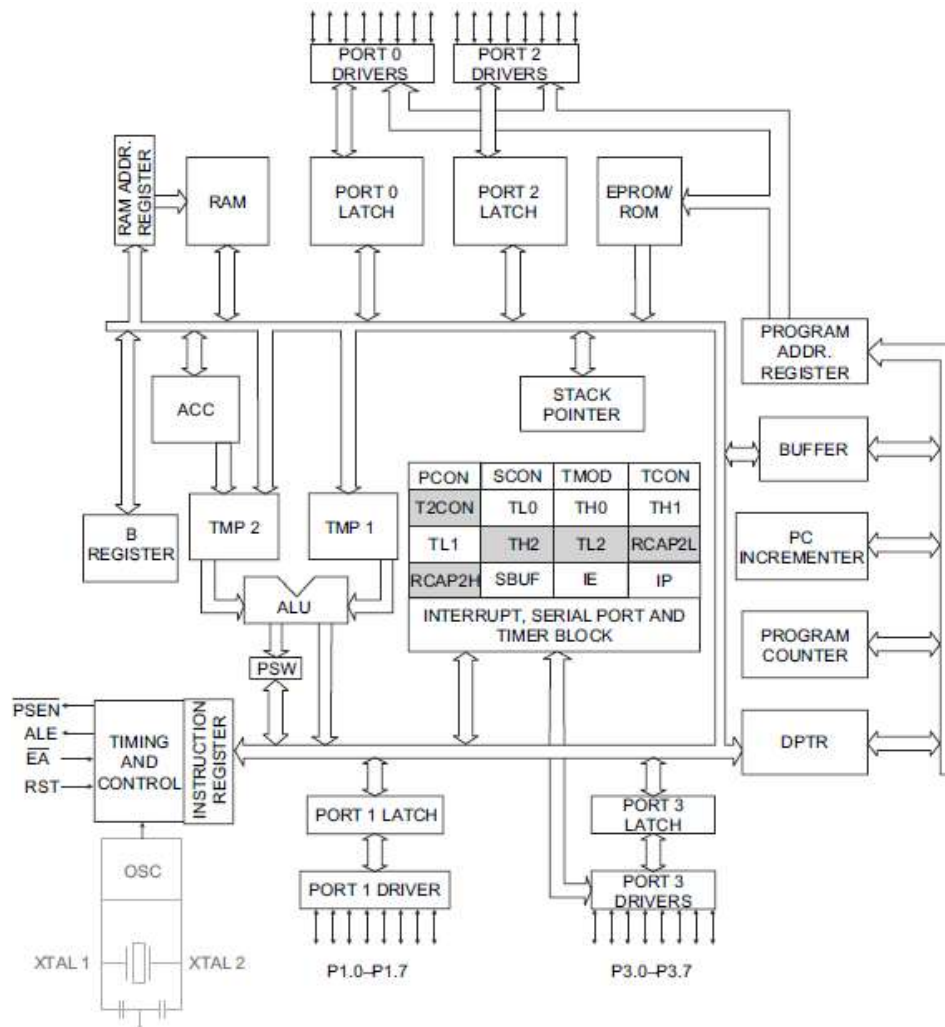


END 4 mar 2026

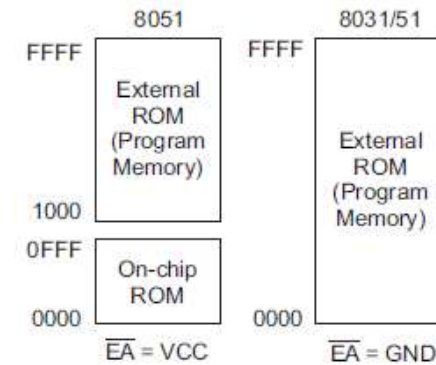
EFECTE RESET LA MCU 8051

REGISTER(S)	CONTENTS
Program counter	0000H
Accumulator	00H
B register	00H
PSW	00H
SP	07H
DPTR	0000H
Ports 0-3	FFH
IP (8031/8051)	XX000000B
IP (8032/8052)	XX000000B
IE (8031/8051)	0XX00000B
IE (8032/8052)	0X000000B
Timer registers	00H
SCON	00H
SBUF	00H
PCON (HMOS)	0XXXXXXXB
PCON (CMOS)	0XXX0000B

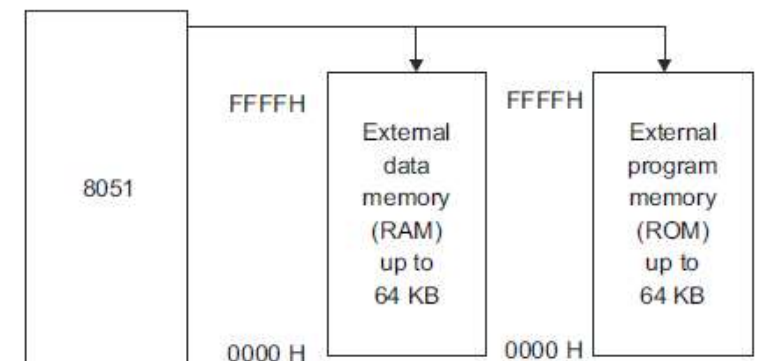
STRUCTURA INTERNA MCU 8051



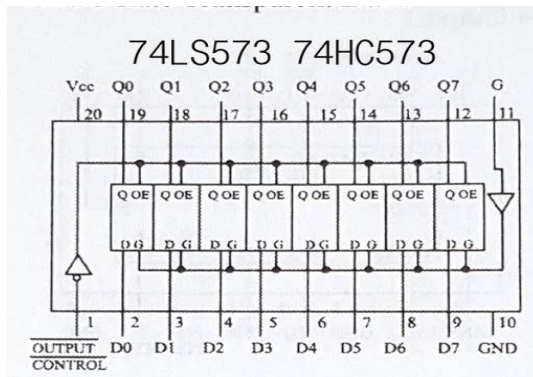
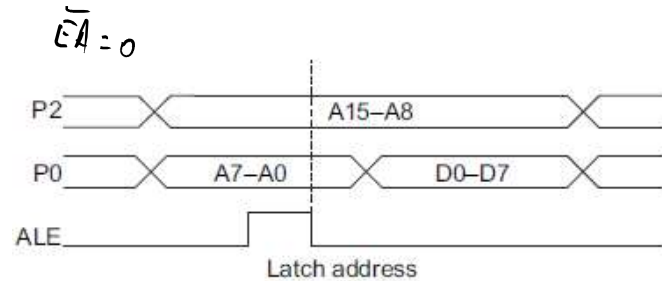
MEMORIA DE PROGRAME LA MCU 8051



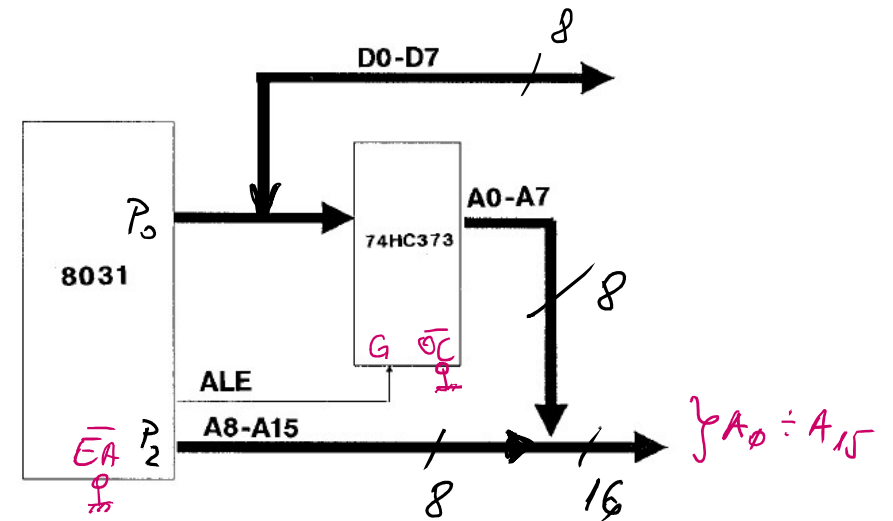
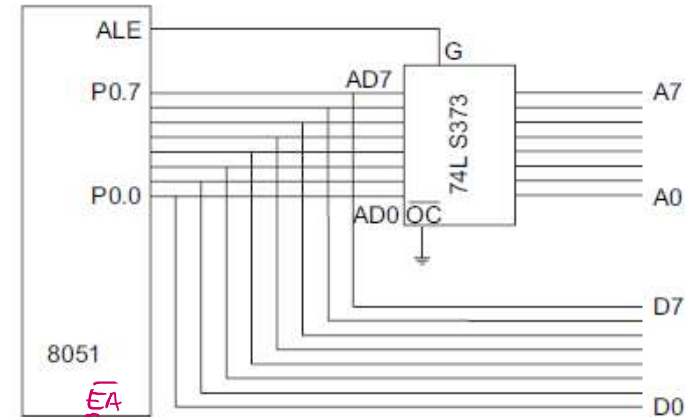
MEMORIA EXTERNA LA MCU 8051 (EA = 0)



FORMAREA MAGISTRALELOR EXTERNE MCU 8051 (EA = 0)



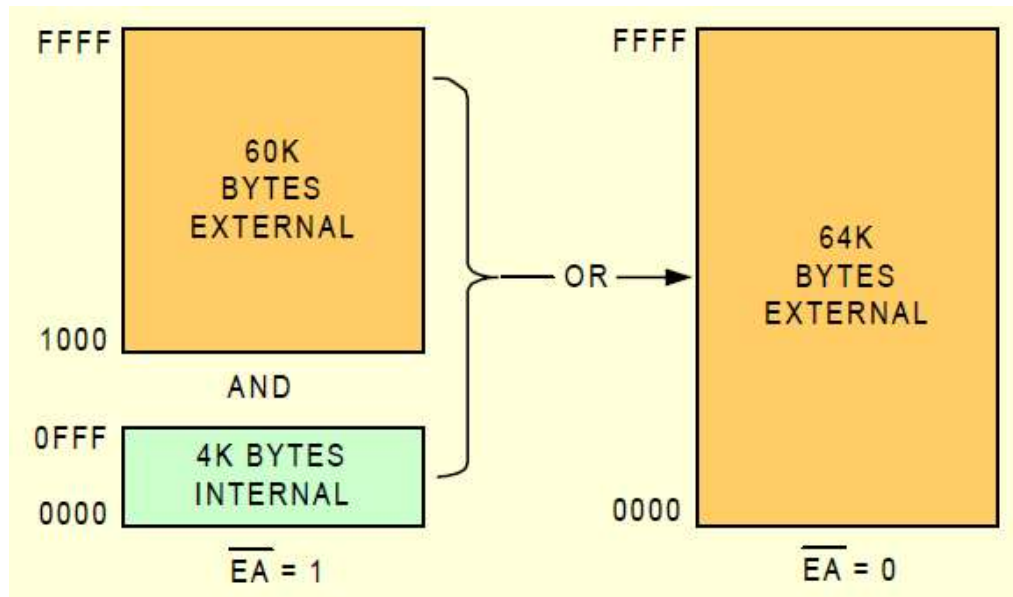
latch transparent
latch-ul
înzădireste
(înghetă)



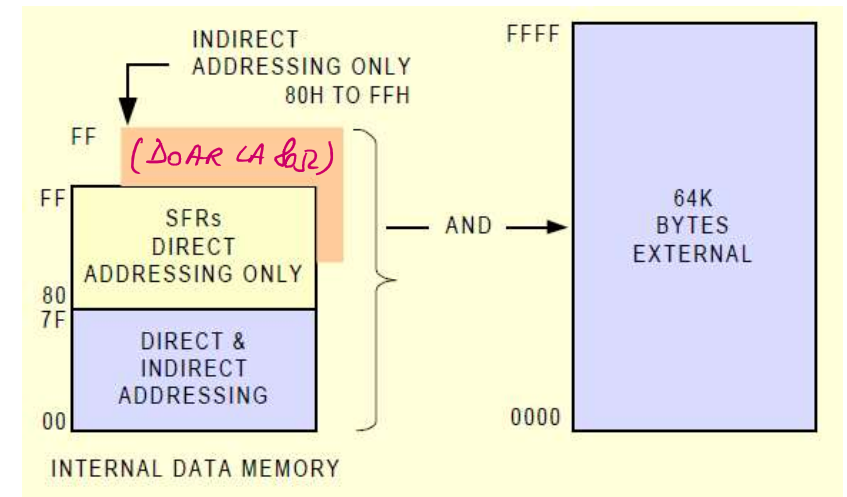
FUNCTIONALITATI PINI CONTROL LA MCU 8051

	*PSEN	*RD	*WR
Program Instruction Fetch	0	1	1
External Data Read	1	0	1
External Data Write	1	1	0

MEMORIA DE PROGRAME LA MCU 8051

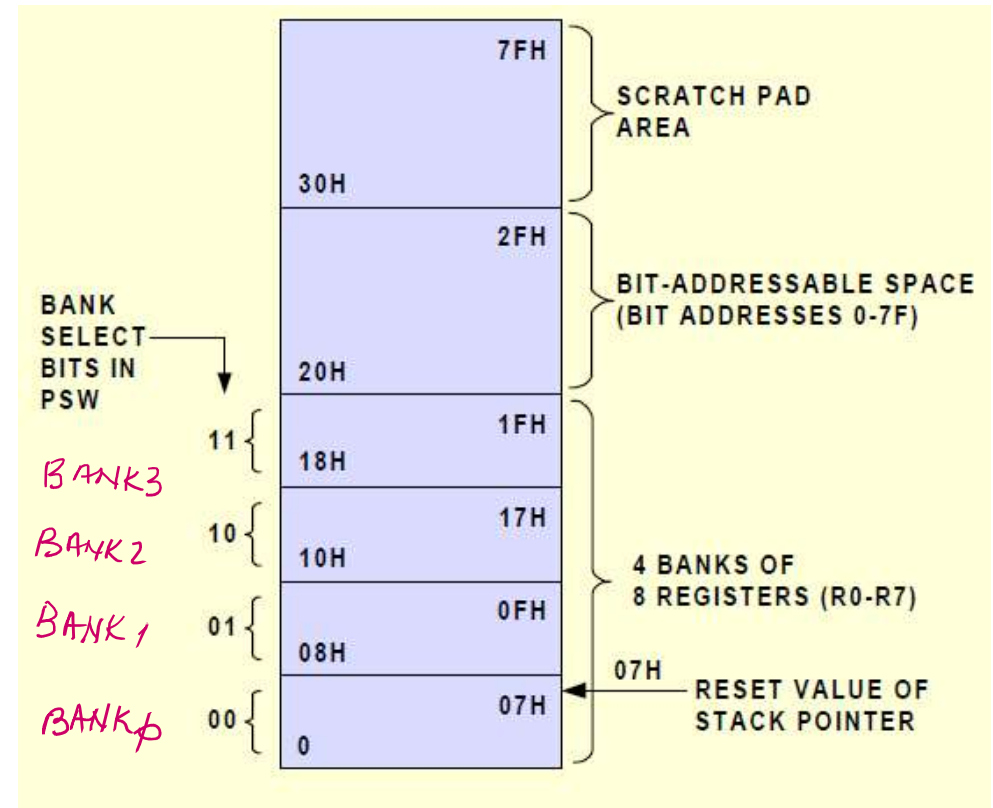
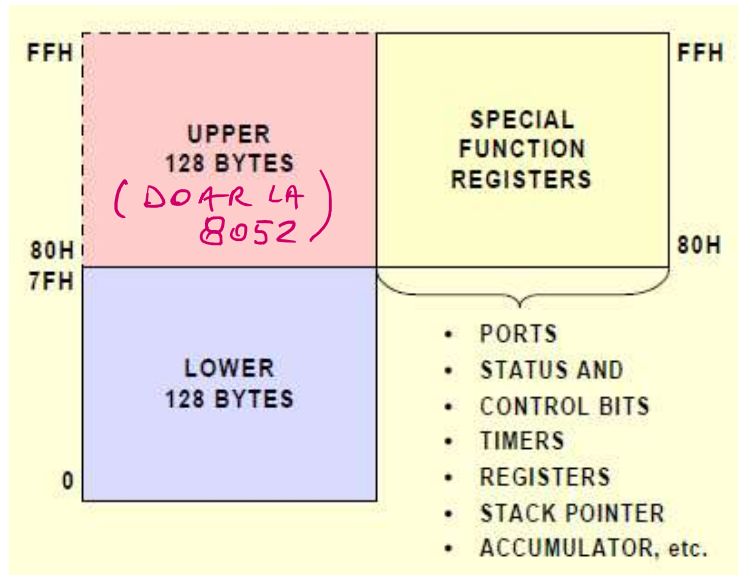


MEMORIA DE DATE LA MCU 8051



SFR - Special Function Registers

MEMORIA RAM INTERNA (DATE) LA MCU 8051



$R_0, R_1, R_2, \dots, R_7$

MEMORIA RAM INTERNA (DATE) LA MCU 8051

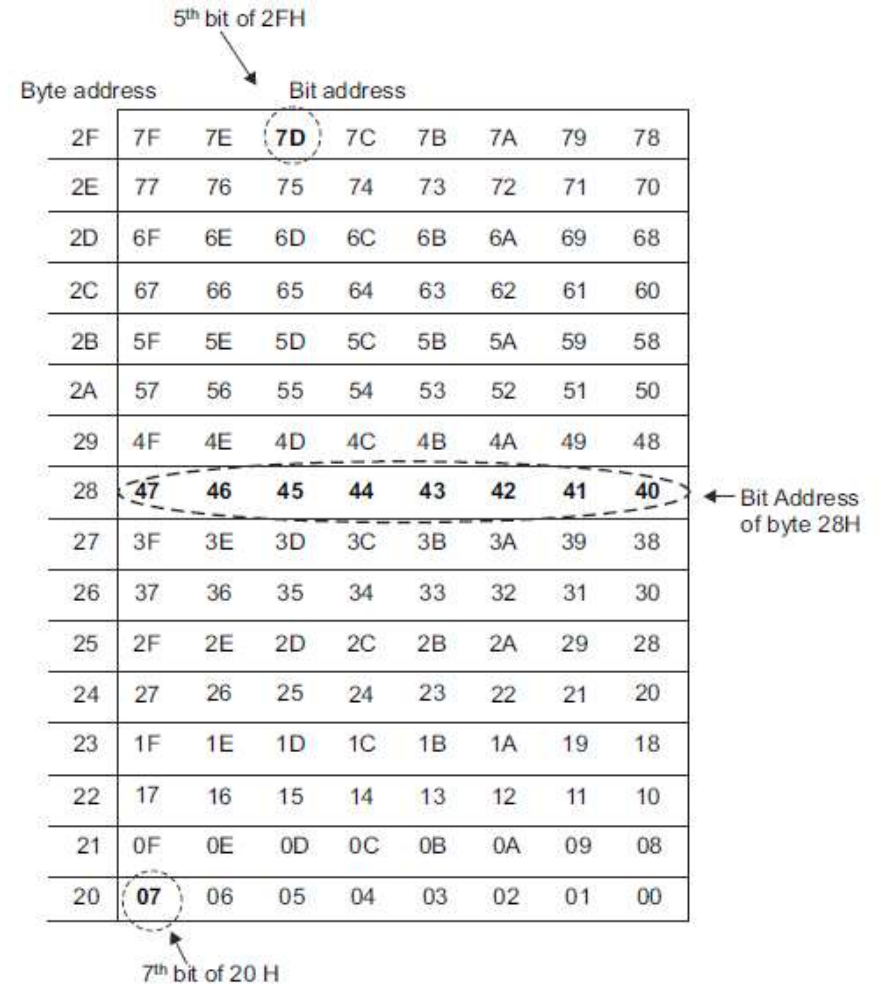
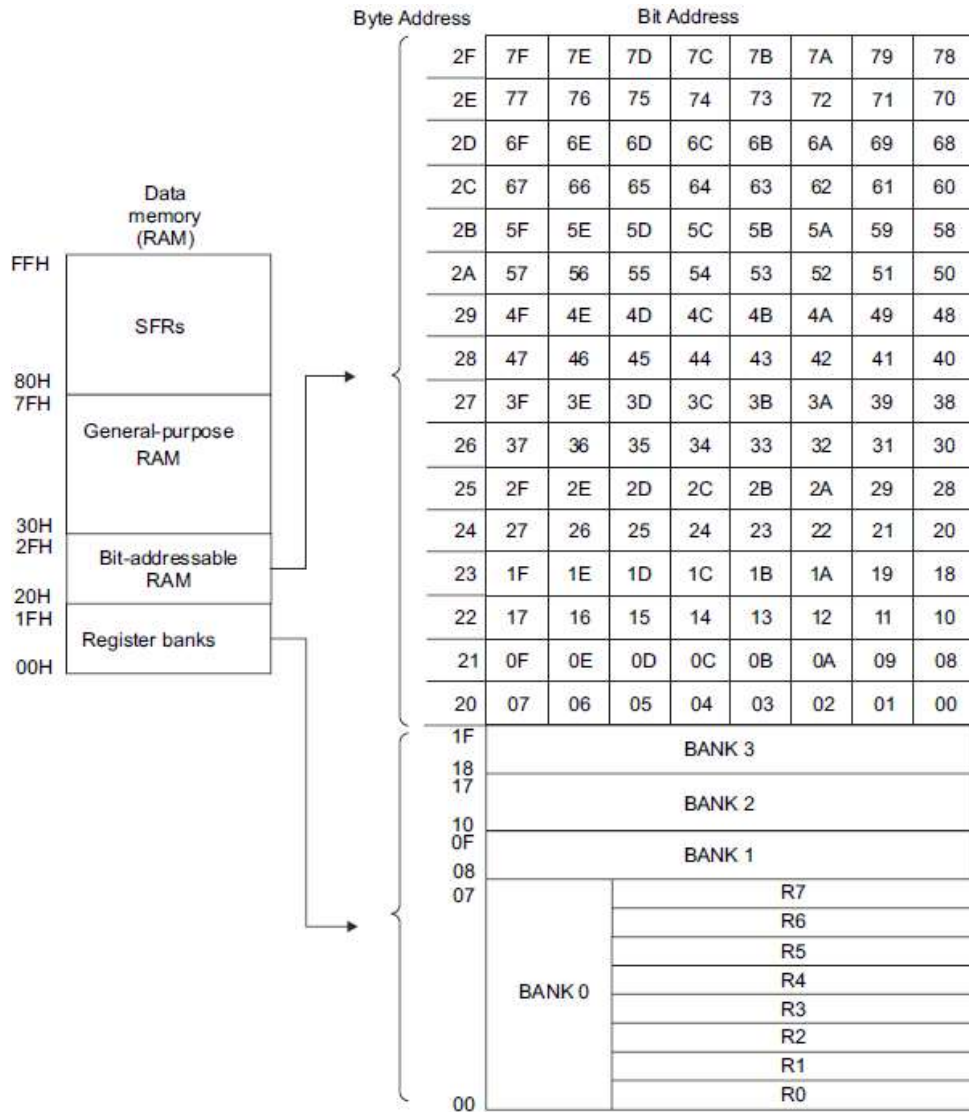
Byte address	Bit address
7F	General purpose RAM
30	
2F	7F 7E 7D 7C 7B 7A 79 78
2E	77 76 75 74 73 72 71 70
2D	6F 6E 6D 6C 6B 6A 69 68
2C	67 66 65 64 63 62 61 60
2B	5F 5E 5D 5C 5B 5A 59 58
2A	57 56 55 54 53 52 51 50
29	4F 4E 4D 4C 4B 4A 49 48
28	47 46 45 44 43 42 41 40
27	3F 3E 3D 3C 3B 3A 39 38
26	37 36 35 34 33 32 31 30
25	2F 2E 2D 2C 2B 2A 29 28
24	27 26 25 24 23 22 21 20
23	1F 1E 1D 1C 1B 1A 19 18
22	17 16 15 14 13 12 11 10
21	0F 0E 0D 0C 0B 0A 09 08
20	07 06 05 04 03 02 01 00
1F	Bank 3
18	
17	Bank 2
10	
0F	Bank 1
08	
07	Default register bank for R0–R7
00	

RAM

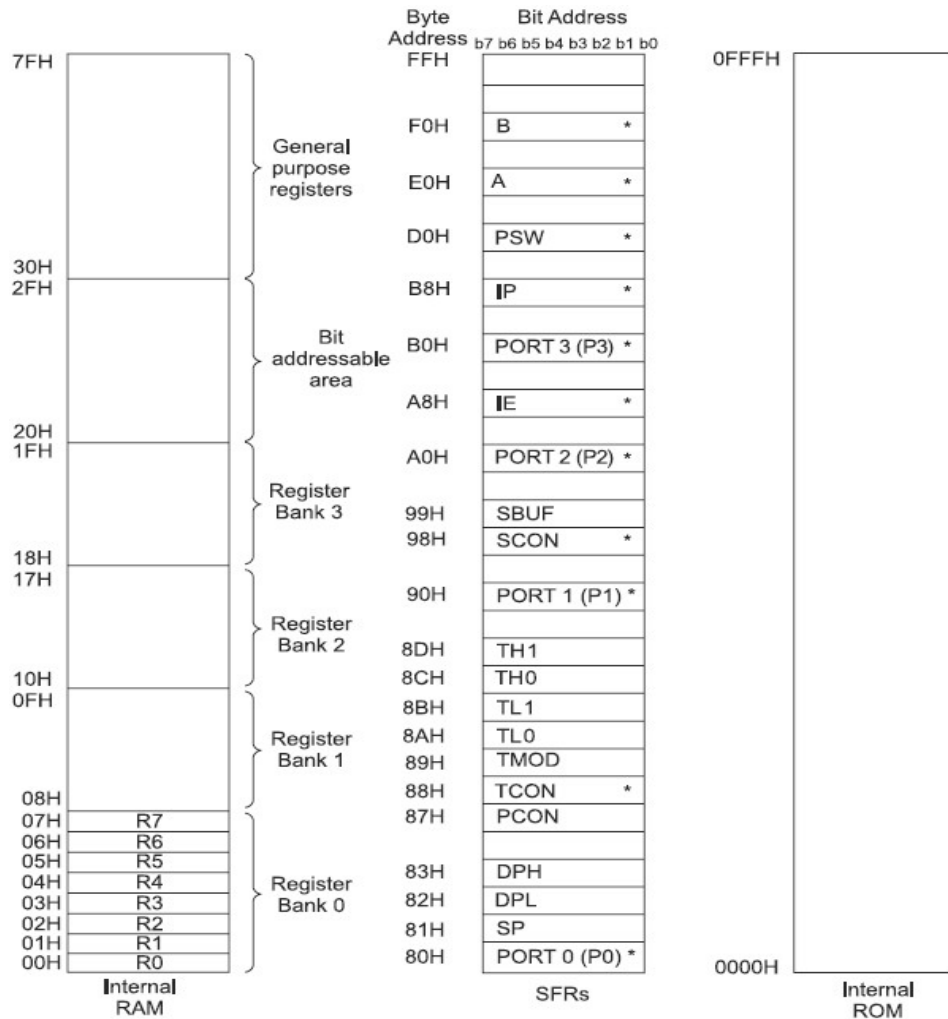
Byte address	Bit address	
FF		
F0	F7 F6 F5 F4 F3 F2 F1 F0	B
E0	E7 E6 E5 E4 E3 E2 E1 E0	ACC
D0	D7 D6 D5 D4 D3 D2 – D0	PSW
B8	– – – BC BB BA B9 B8	IP
B0	B7 B6 B5 B4 B3 B2 B1 B0	P3
A8	AF – – AC AB AA A9 A8	IE
A0	A7 A6 A5 A4 A3 A2 A1 A0	P2
99	not bit addressable	
98	9F 9E 9D 9C 9B 9A 99 98	SCON
90	97 96 95 94 93 92 91 90	P1
8D	not bit addressable	
8C	not bit addressable	
8B	not bit addressable	
8A	not bit addressable	
89	not bit addressable	
88	8F 8E 8D 8C 8B 8A 89 88	TCON
87	not bit addressable	
83	not bit addressable	
82	not bit addressable	
81	not bit addressable	
80	87 86 85 84 83 82 81 80	P0

SPECIAL FUNCTION REGISTERS

ZONA ADRESABILA PE BIT RAM INTERN MCU 8051



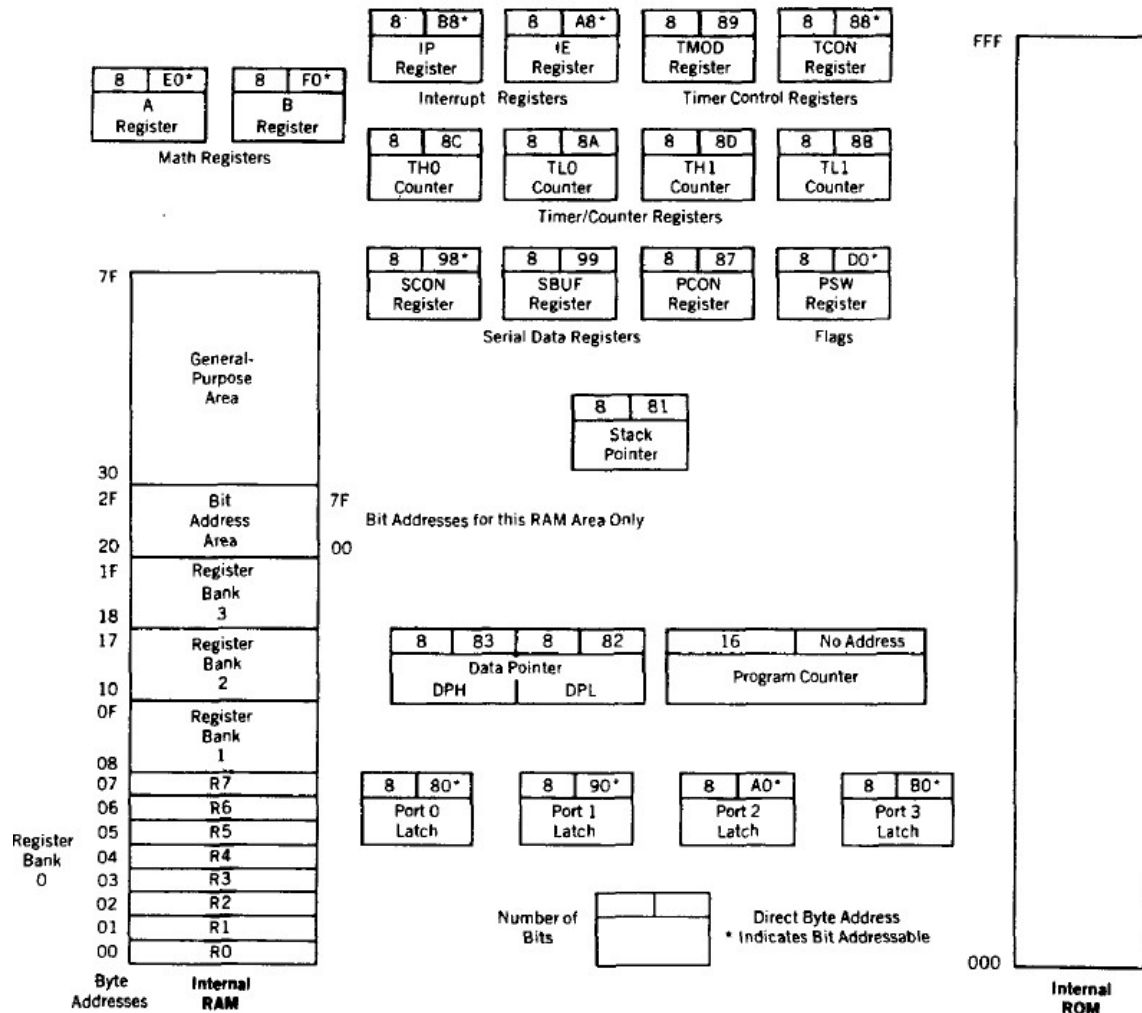
MCU INTEL 8051 – Programming Model



*Indicates the SFRs which are also bit addressable

NAME	FUNCTION	INTERNAL RAM ADDRESS (HEX)
A	Accumulator	0E0
B	Arithmetic	0F0
DPH	Addressing external memory	83
DPL	Addressing external memory	82
IE	Interrupt enable control	0A8
IP	Interrupt priority	0B8
P0	Input/output port latch	80
P1	Input/output port latch	90
P2	Input/output port latch	A0
P3	Input/output port latch	0B0
PCON	Power control	87
PSW	Program status word	0D0
SCON	Serial port control	98
SBUF	Serial port data buffer	99
SP	Stack pointer	81
TMOD	Timer/counter mode control	89
TCON	Timer/counter control	88
TL0	Timer 0 low byte	8A
TH0	Timer 0 high byte	8C
TL1	Timer 1 low byte	8B
TH1	Timer 1 high byte	8D

MCU INTEL 8051 – Programming Model



MCU INTEL 8051 – Programming Model

MCU 8051 – Special Function Registers - SFR

+	0	1	2	3	4	5	6	7
F8								
F0	B							
E8								
E0	ACC							
D8								
D0	PSW							
C8	T2CON		RCAP2L	RCAP2H	TL2	TH2		
C0								
B8	IP							
B0	P3							
A8	IE							
A0	P2							
98	SCON	SBUF						
90	P1							
88	TCON	TMOD	TL0	TL1	TH0	TH1		
80	P0	SP	DPL	DPH				PCON

Registri CPU:

ACC (A), B, DPH, DPL (DPTR),
SP, PSW, PCON

Registri Porturi Paralele:

P0, P1, P2, P3

Registri Timere:

TCON, TMOD, TH0, TL0,
TH1, TL1

Registri UART:

SCON, SBUF

Registri logica intreruperi:

IE, IP

 - Denotes bit addressable Special Function Registers in this table and all following diagrams

REGISTRI CPU DIN SFR

ACC (A)

B (MUL AB ; DIV AB)
multiply divide

DPH, DPL (DPTR) – Data Pointer

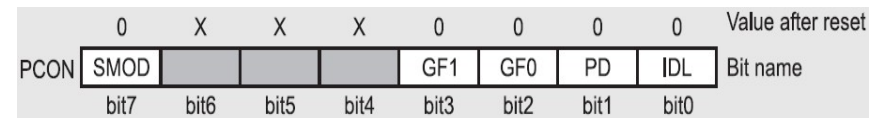
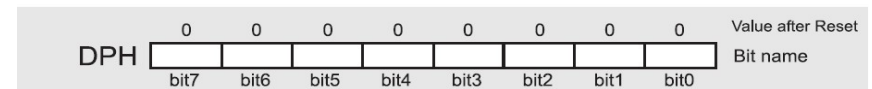
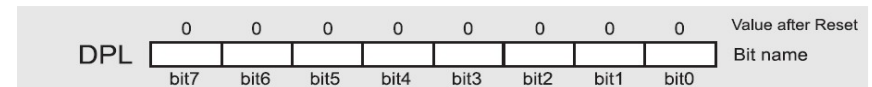
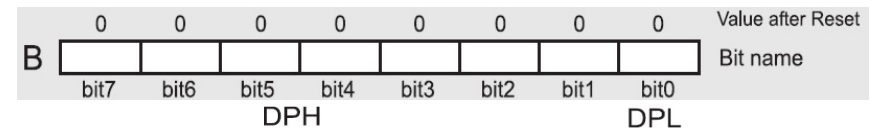
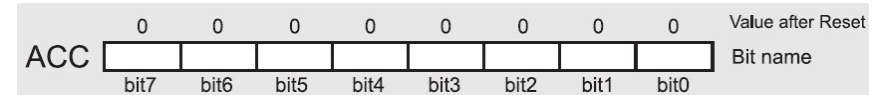
SP - Stack Pointer (initializat la Reset cu 07h)

PSW – (Program Status Word)

PCON – (Power Control)

(IDLE, POWER DOWN)

MUL AB
A, B ← A × B
ocupa ocupa
A B ← A / B
cat rest
2 3
11/4



IDL - idle - 1mA - 5μW
PD - power down - 10μA - 50μW
GF_{1,0} - general flag.

DPH, DPL (DPTR) – Data Pointer

DPH și DPL – prin reuniunea lor formează singurul registru pe 16 biți accesibil utilizatorului, denumit DPTR (data pointer). Prin intermediul acestui registru se poate adresa memoria externă de programe și date, dar și memoria internă de programe. Nu poate folosit pentru adresarea RAM-ului intern sub forma DPTR.

move

Astfel, instrucțiunile de transfer de date cu memoria RAM externă sunt:

- MOVX A,@DPTR – realizează o citire din memoria RAM externă; $A \leftarrow @DPTR$ (X reprezentând eXternal). $A \leftarrow (DPTR)$
- MOVX @DPTR, A – realizează o scriere în memoria RAM externă; $@DPTR \leftarrow A$ sau $(DPTR) \leftarrow A$.

Transferul din memoria de programe se face cu MOVC A, @A + DPTR (C reprezintă Code). În acumulator este adus conținutul locației memoriei de programe a cărei adresă se obține prin însumarea conținutului DPTR cu cel al acumulatorului anterior:

$A \leftarrow (A+DPTR)$.

MOVC permite citirea memoriei de programe atât internă ($\overline{EA} = 1$) cât și externă ($\overline{EA} = 0$). Această instrucțiune trebuie pregătită prin încărcarea DPTR și a acumulatorului.

MOV DPTR,#adresă memorie

MOV A, #deplasament

MOVC A,@A+DPTR

END 11 mar 2026

MOV A, #31h
A ← #31h
MOV A, 31h
A ← (31h)

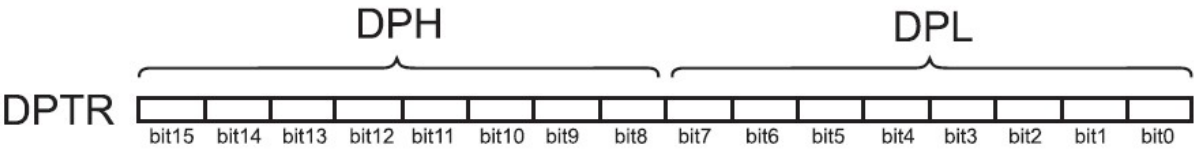
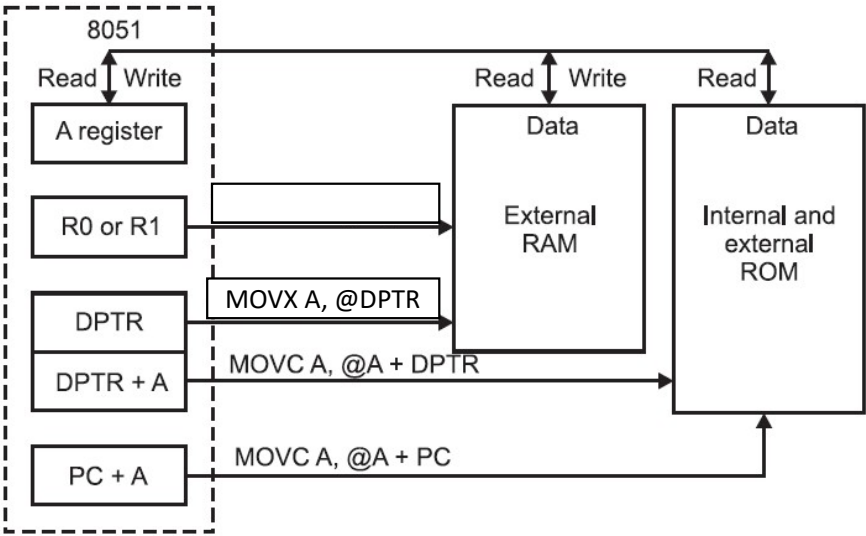
Notatii folosite in instructiuni:

- reprezinta operand imediat

@ - continut al unei locatii de memorie a carei adresa urmeaza dupa simbol

Salvarea în stiva a DPTR se poate face doar individual pe jumătățile din care este format.

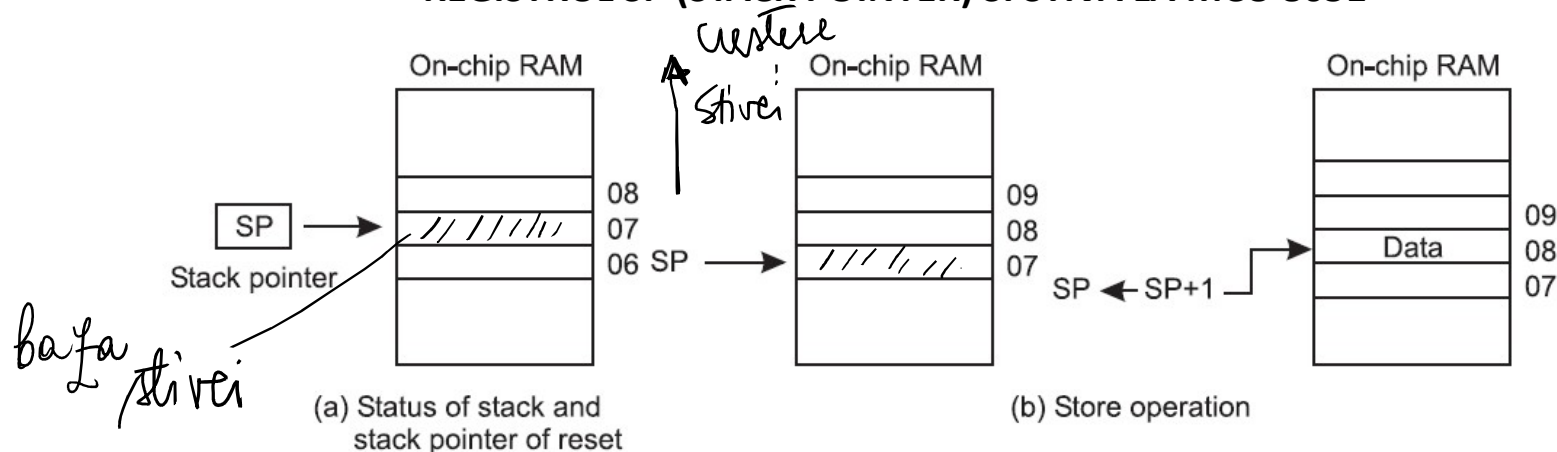
DPH, DPL (DPTR) – Data Pointer



DPL								Value after Reset
0	0	0	0	0	0	0	0	
bit7	bit6	bit5	bit4	bit3	bit2	bit1	bit0	Bit name

DPH								Value after Reset
0	0	0	0	0	0	0	0	
bit7	bit6	bit5	bit4	bit3	bit2	bit1	bit0	Bit name

REGISTRUL SP (STACK POINTER) SI STIVA LA MCU 8051



Stiva se formează în RAM-ul intern și este adresată în tehnică LIFO de un registru specializat, denumit SP, pe 8 biți. La inițializarea controller-ului, SP se încarcă cu adresa 07_H (baza stivei), prima locație în care se poate depune în stivă fiind 08_H. Stiva crește către sfârșitul memoriei (la depuneri în stivă) și descrește la extrageri, adică SP se incrementează la depuneri și se decrementează la extrageri. Registrul SP este localizat în zona de memorie RAM internă, denumită SFR.

Din cele de mai sus rezultă faptul că proiectanții propun formarea stivei în zona în care se formează bank-urile de regiștri 1,2,3. Dacă aplicația în care este folosit controller-ul permite acest lucru opțiunea este posibilă. Dacă nu (se folosesc bank-urile de regiștri), atunci este posibilă schimbarea zonei de memorie RAM în care se formează stiva. În acest sens registrul SP se încarcă cu o nouă adresă de bază a stivei printr-o instrucțiune de tipul *MOV SP, #adresă_bază_stivă*, unde noua adresă de bază a stivei poate fi mai mare de 30_H (sau într-o altă zonă liberă de sarcini).

REGISTRUL PSW MCU 8051

Processor Status Word (PSW) - Bit Addressable

CY	AC	F0	RS1	RS0	OV	USR	P
CY	Carry Flag						
AC	Auxiliary Carry Flag						
F0	General Purpose Flag						
RS1	Register Bank Selector 1. MSB of selector.						
RS0	Register Bank Selector 0. LSB of selector.						
OV	Overflow Flag						
USR	User Definable Flag						
P	Accumulator Parity Flag						

$$35 + 36 = 71$$

$$00110101_2 +$$

$$00110110_2$$

$$01101011_2 = 6Bh + 06h \xrightarrow{DAA} 71$$

$$35_{(10)} \xrightarrow{BCD} 00110101_{(2)}$$

$$23_{16} \xrightarrow{16^1 16^0}$$

$$= 00100011_2$$

BCD? ?!

DAA

Bitul CY – (bitul de transport/împrumut) care este setat în mod automat de mecanismele MCU în urma unor operații aritmetice de adunare respectiv scădere. Bitul poate fi acționat și manual de către utilizator prin instrucțiuni specifice.

Bitul AC – (auxiliar carry) este bitul de transport între cele două jumătăți ale acumulatorului în urma operațiilor aritmetice de adunare și scădere.

Bitul F0 – (flag zero) este bit accesibil utilizatorului și la dispoziția acestuia care poate fi setat – resetat în funcție de necesitățile programului în care este folosit.

Bitii RS₁ și RS₀ – (Registers select) selectează bank-ul de regiștri de uz general R₀ până la R₇ activ la un moment dat. La inițializarea MCU RS₁=RS₀=0.

Bitul OV – (overflow) este un bit acționat de mecanismele MCU prin care se indică depășirea gamei de reprezentare a rezultatelor obținute ca urmare a efectuării a operației aritmetice (adunare sau scădere) cu numere reprezentate în CC2 pe 8 biti.

Bitul P – (parity) este bit prin care se indică paritatea rezultatului obținut în urma operațiilor aritmetico-logice. Bitul se calculează cu metoda parității pare. Spre deosebire de procesorul Z-80, bitul P este acționat atât în urma operațiilor logice cât și a celor aritmetice.

REGISTRUL PCON MCU 8051

Power Control Register (PCON) - Not Bit Addressable

SMOD	-	-	-	GF1	GF0	PDWN	IDLE
------	---	---	---	-----	-----	------	------

SMOD Serial baud rate generator mode. If SMOD=1 the baud rate of the UART is doubled.

- Reserved.

- Reserved.

- Reserved.

GF1 General Purpose flag.

GF0 General Purpose flag.

PDWN Power Down flag. Setting this bit causes activation of power down mode.

IDLE Idle flag. Setting this bit causes activation of idle mode.

Bitul IDLE=1 conduce controller-ul la o stare de funcționare în care procesorul este oprit, dar toate circuitele I/O sunt funcționale iar memoria RAM internă este alimentată. Consumul controller-ului se reduce la sub 1mA. Revenirea în starea de funcționare se poate face fie printr-o întrerupere generată de circuitele I/O, fie printr-un reset aplicat procesorului.

Bitul PDWN=1 conduce la intrarea procesorului într-o stare în care doar memoria RAM internă rămâne alimentată și sunt oprite procesorul, respectiv toate circuitele de I/O. Consumul în această stare se reduce la până 10 uA.

Biții GF₀ și GF₁ – sunt biți ce pot fi scriși de utilizator, prin care se poate realiza o codare a 4 situații posibile în urma cărora ar fi rezultat stările PD, respectiv IDL. Codarea va putea fi folosită de către procesor la revenirea în starea de funcționare pentru a detecta situația care a generat intrarea în stările PD, respectiv IDL.

Bitul SMOD – când este 1 logic dublează viteza de comunicație serială proiectată uzual pentru UART.

Porturile paralele din MCU 8051

Sunt notate P_0, P_1, P_2, P_3 , iar pinii gestionați au denumirile: $P_{00}-P_{07}, P_{10}-P_{17}, P_{20}-P_{27}, P_{30}-P_{37}$.

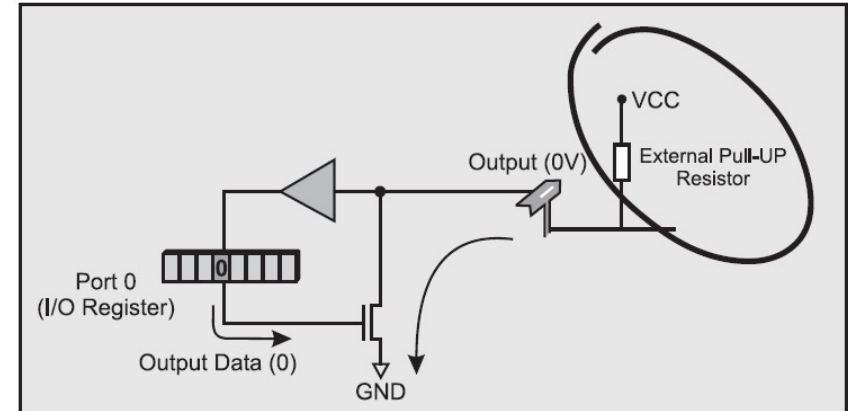
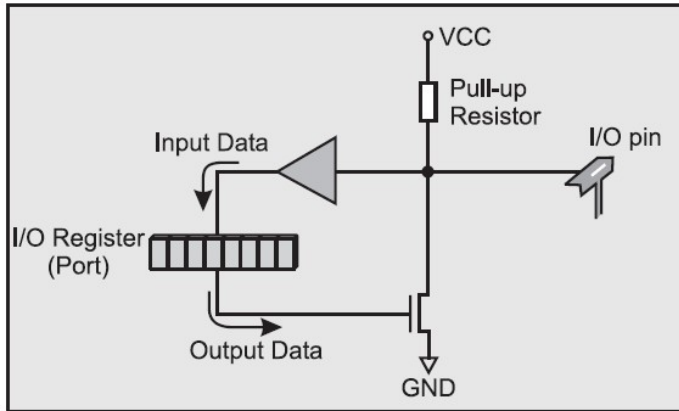
Dintre cele 4 porturi doar P_1 nu are ale funcționalități suplimentare. Cele 4 porturi sunt gestionate prin intermediul a 4 regiștri aflați în zona SFR care au aceleași denumiri: P_0, P_1, P_2, P_3 . Cei 4 regiștri menționați sunt regiștri de date pentru porturile asociate, neexistând nici un registru pentru controlul software al acestuia. Utilizarea regiștrilor se face în modul următor:

- o scriere efectuată în unul din regiștri descriși conduce la generarea semnalelor digitale corespunzătoare pe liniile portului corespondent;
- o citire efectuată din unul din regiștri menționați provoacă preluarea informației corespunzătoare semnalelor digitale existente pe liniile portului asociat;

Obs. Nu există mecanism software de stabilire a direcționalității liniilor unui port. Liniile unui port pot avea direcționalități diverse. Reamintim că regiștrii P_0, P_1, P_2, P_3 sunt în prima coloană (stânga) din tabelul SFR și au proprietatea de a fi acționați la nivel de bit prin instrucțiuni specifice.

Modificări pe liniile portului (intrări) nu pot fi semnalizate prin întreruperi procesorului din controller, tehnica de identificare a unei modificări fiind de tip polling (interogare software permanentă).

Porturile paralele din MCU 8051



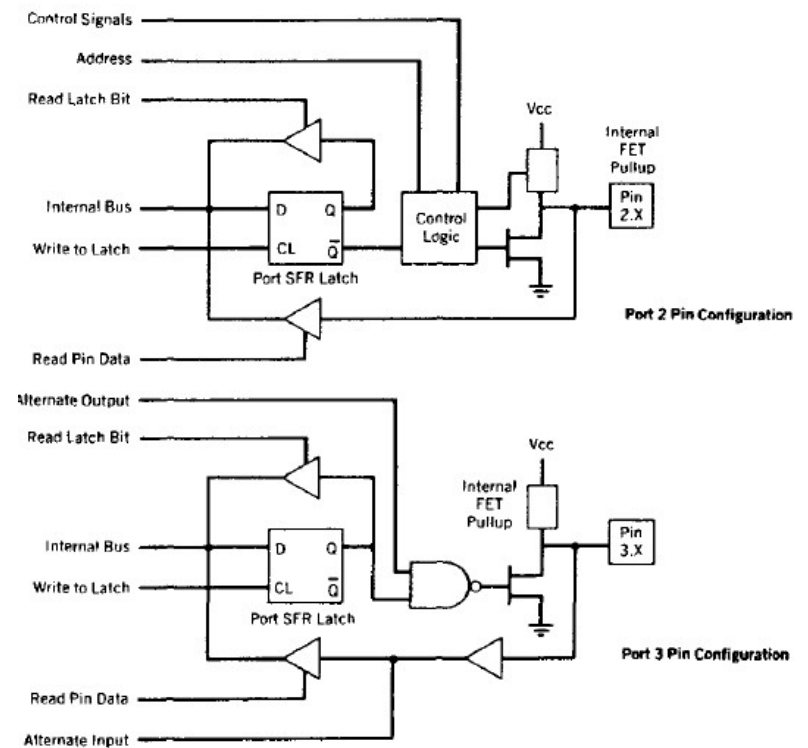
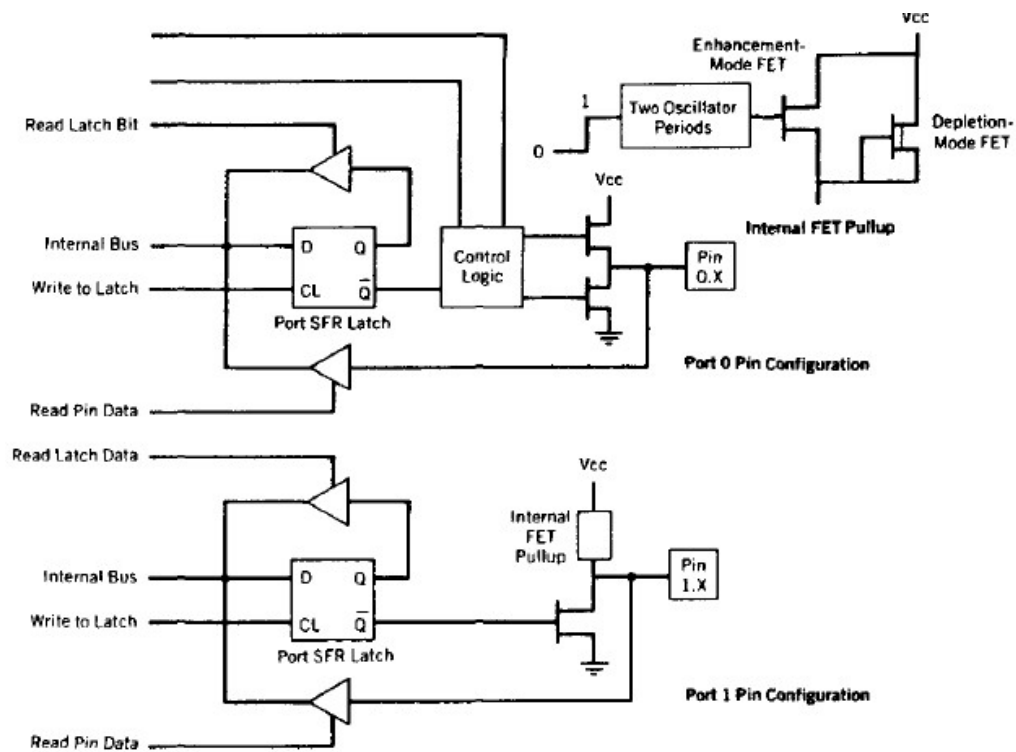
Când sunt configurați ca ieșiri (logic 0), pinii dintr-un singur port pot „primi” curent de până la 10mA. Dacă toți cei 8 biți ai unui port sunt activi, curentul total trebuie să fie limitat la 15mA (la portul P0: 26mA). Dacă toate porturile (32 biți) sunt active, curentul maxim total trebuie limitat la 71mA.

Când sunt configurați ca intrări (logic 1), rezistența de pull-up încorporată asigură un curent foarte slab, dar suficient de puternic pentru a conecta până la 4 intrări TTL din seria LS.

Chiar dacă toți pinii porturilor au mai mult sau mai puțin structură internă similară este necesar să se acorde atenție la cum și ce vor fi folosiți.

De exemplu: Dacă sunt utilizate ca ieșiri cu nivel de tensiune ridicat (5V), atunci portul P0 ar trebui să fie evitat deoarece pinii săi nu au rezistor adăugat pentru conectarea la + 5V; prin urmare, numai un nivel logic scăzut poate fi obținut; dacă un alt port este utilizat în același scop, ar trebui să avem în vedere că rezistențele de pull-up au valori relativ mari. În consecință, se poate conta numai pe câteva sute de microamperi de curent care ies dintr-un pin.

Porturile paralele din MCU 8051



Canalele Timer din MCU 8051

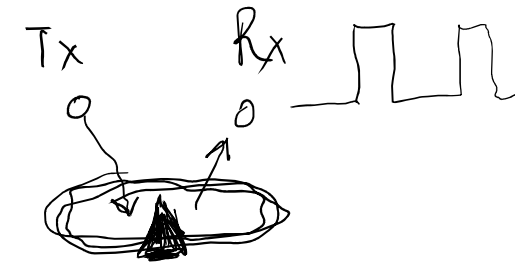
La nivelul circuitului 8051 există două canale timer care pot opera independent capacitatea de numărare este de 16 biți. Sunt posibile 4 moduri de operare și două regimuri de funcționare programabile software.

Regimul de funcționare permite alegerea sursei semnalului de ceas în ritmul căruia canalul timer operează. Se poate opta pentru ceas extern controller-ului aplicat pe una din intrările T_0 sau T_1 , în funcție de canal sau ceas intern provenit de la oscilatorul local (divizat prin 12).

Cele două canale timer sunt gestionate de regiștri TMOD și TCON, respectiv regiștrii de date, TH_0 și TL_0 la canalul timer 0, respectiv TH_1 și TL_1 la canalul timer 1.

Indiferent de sursa semnalului de ceas și indiferent de modul de operare, canalele timer din 8051 numără înainte până la atingerea stării finale.

Starea finală poate fi $2^8=256$ când numără pe 8 biți sau $2^{16}=65536$ când se numără pe 16 biți. Numărul de strări prin care se trece în procesul de numărare la aplicarea unui impuls de ceas este dat de diferența dintre starea finală, amintită mai sus, și starea inițială care se înscrie la începutul procesului de numărare în perechea de regiștri TH_i și TL_i .



Registrul TMOD - Timer Mode

	7	6	5	4	3	2	1	0
	GATE	C/ $\bar{T}$	M ₁	M ₀	GATE	C/ $\bar{T}$	M ₁	M ₀
	Timer 1				Timer 0			

Bitii M0 si M1 – aleg modul de operare la canal

Bitii C/ $\bar{T}$ – denumirea provine din counter/timer; permite alegerea regimului de funcționare (sursa semnalului de ceas) la respectivul canal timer. Astfel, C/ $\bar{T}$ =1 înseamnă regim counter (numerator), sursa semnalului de ceas este externă aplicată pe intrarea T_i a canalului, iar C/ $\bar{T}$ =0, înseamnă regim timer (temporizator), sursa semnalului de ceas este internă.

Bitii GATE – dacă GATE=1 declanșarea numărării la respectivul canal timer este validată hardware de un semnal extern printr-un semnal de tip GATE=1 aplicat pe un pin din P₃ notat $\overline{INT}_i$. Acești pini apar că au triplă funcționalitate: pin de tip port paralel, pin pentru primirea întreruperilor externe și pin pentru declanșarea hardware a procesului de numărare.

Registrul TCON – Timer Control

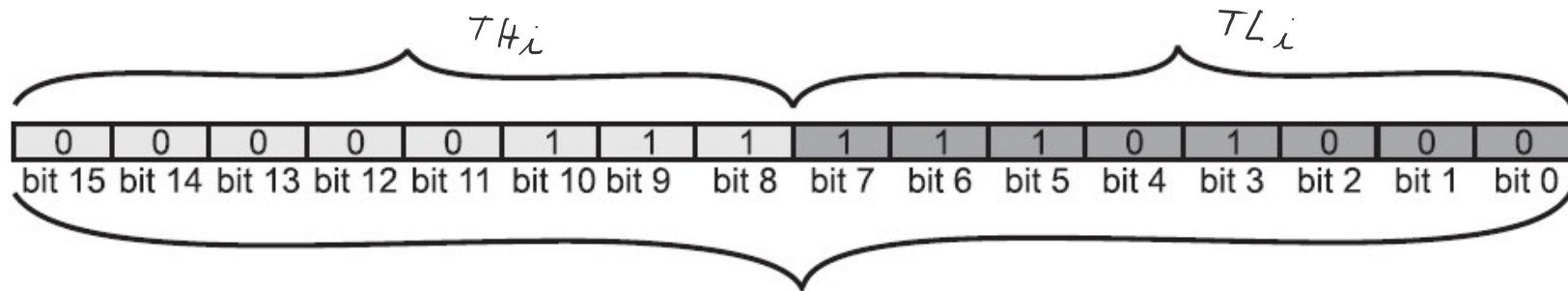
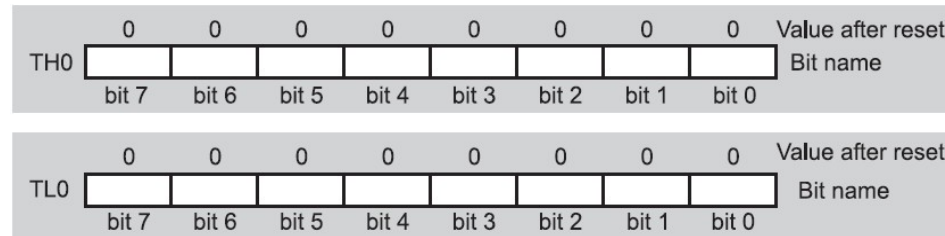
	7	6	5	4	3	2	1	0
	TF ₁	TR ₁	TF ₀	TR ₀	Folosiți la logica de întreruperi			

Bitii TR_i (timer run) – bit ce poate fi acționat de utilizator. TR_i=1 validează din punct de vedere software numărarea la canal. Dacă TR_i=0 numărarea este blocată la canalul respectiv.

Bitii TF_i (timer flag) – este de tip read only. Mecanismele canalului timer setează acest bit (1 logic) la finalul numărării, odată cu atingerea stării finale. Acești biți reprezintă sursă de întreruperi pentru procesor. De obicei operarea cu canale timer la 8051 se face cu întreruperi. Începutul tratării unei întreruperi provenite de la canalul timer resetează automat bitul TF_i care a generat întreruperea.

Obs. Numararea pe canalele timer din 8051 se face catre inainte (prin incrementare).

Registrii pentru constanta de divizare (presetare) THi si TLi



Pasi programare canal timer 8051

- Programare TMOD;
- Programare TH, TL la canal;
- Programare TCON (bitul TR <- 1).

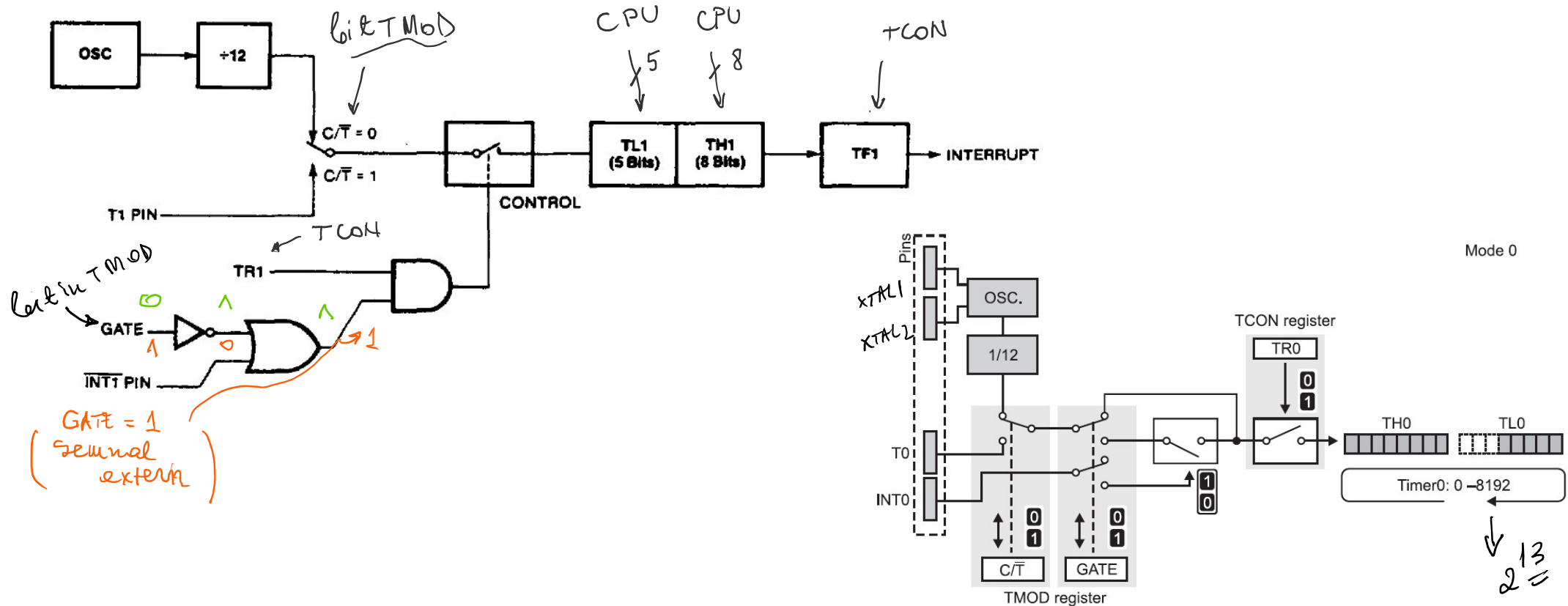
END 18 mar 2026

Obs. Numararea pe canalele timer din 8051 se face catre inainte (prin incrementare).

Modurile de operare la canalele Timer din MCU 8051

Modul 0 – cu numărare pe 13 biti și blocare la sfârșitul numărării

Modul 0 a fost proiectat pentru a asigura compatibilitatea cu un circuit anterior (MCU 8048). Pentru a relua procesul de numărare este necesară reîncărcarea lui TH_i și TL_i . Este selectat prin bitii $M1 = 0$ și $M0 = 0$ în TMOD.

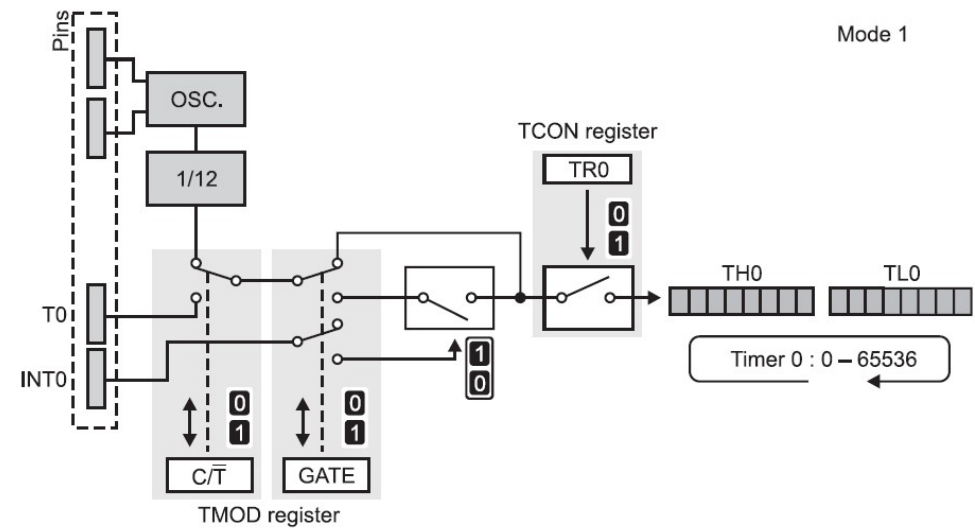
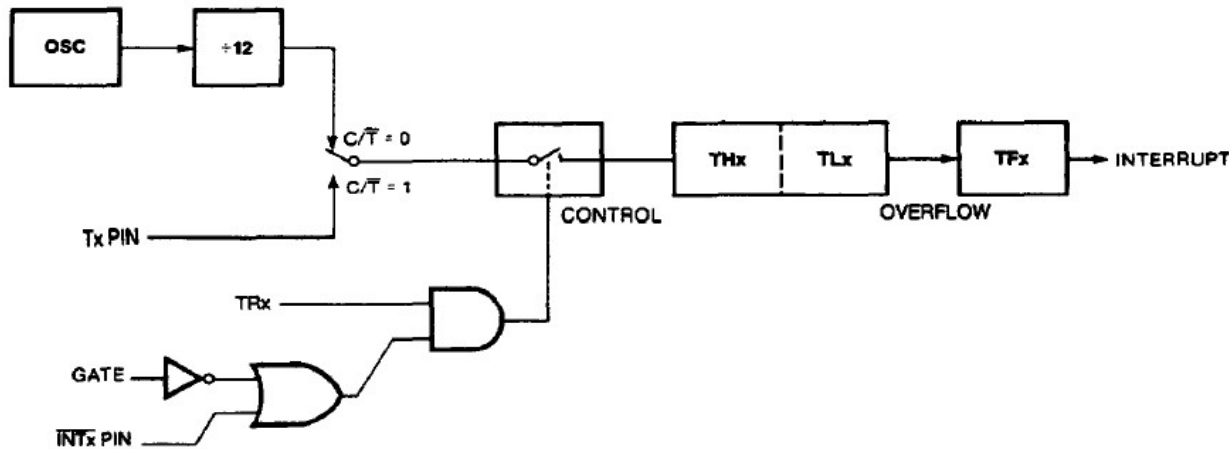


Modurile de operare la canalele Timer din MCU 8051

Modul 1 – cu numarare pe 16 biti si blocare la sfarsitul numararii; modul este selectat prin $M1 = 0$ si $M0 = 1$ in TMOD.

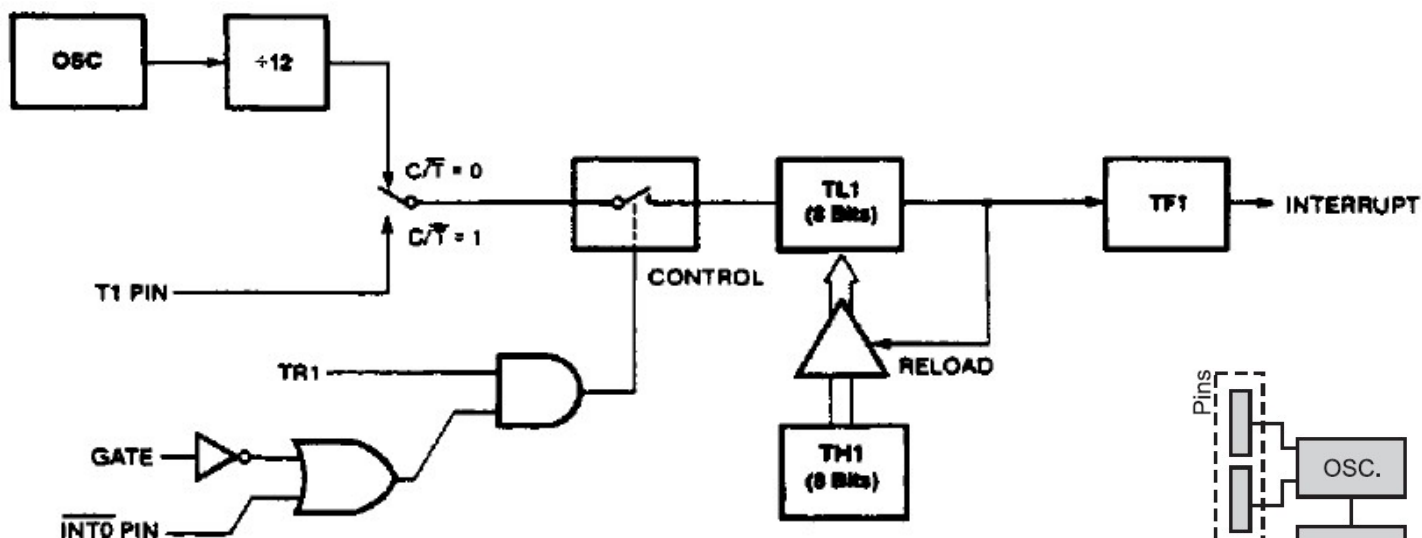
Reluarea procesului de numarare se face prin:

- Programare TH, TL la canal;

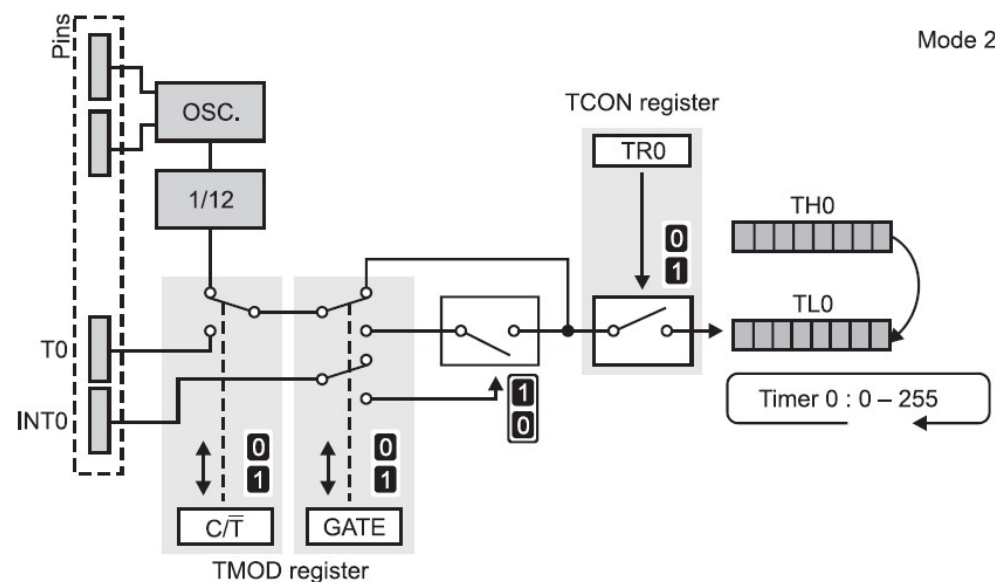


Modurile de operare la canalele Timer din MCU 8051

Modul 2 – cu numărare pe 8 biți și reluare automată la sfârșitul numărării; modul este selectat prin $M1 = 1$ și $M0 = 0$ în TMOD.

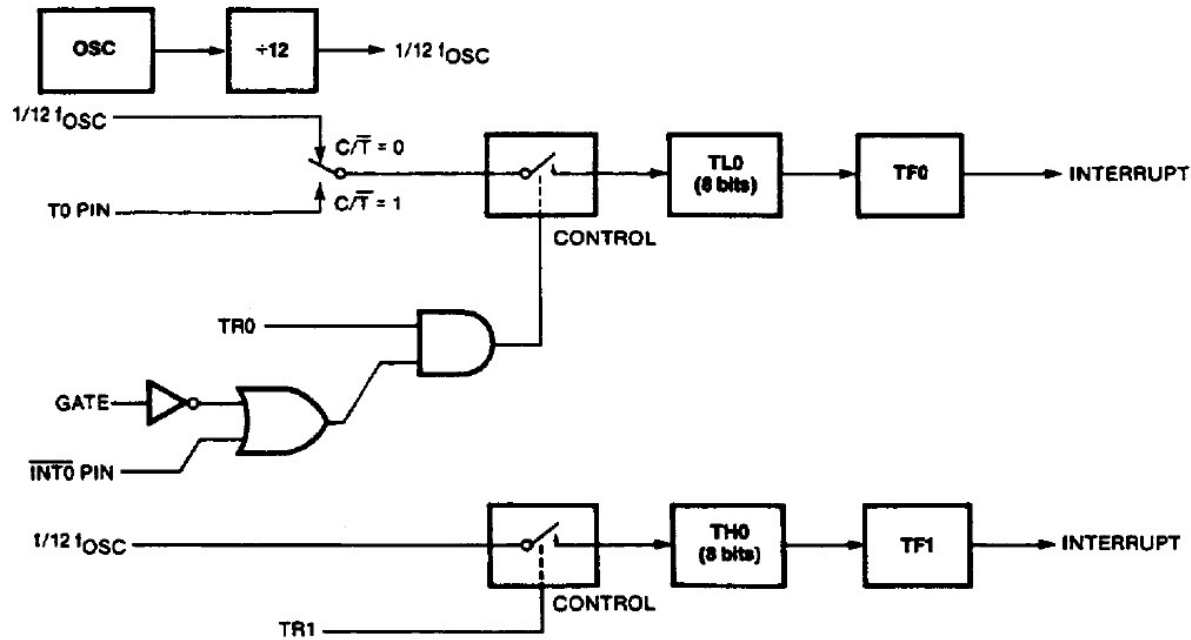


Obs. Timer-ul 1 operând în modul 2 se constituie ca sursă a semnalului de ceas pentru canalul de comunicație serială UART stabilind viteza de comunicație serială (baud-rate).

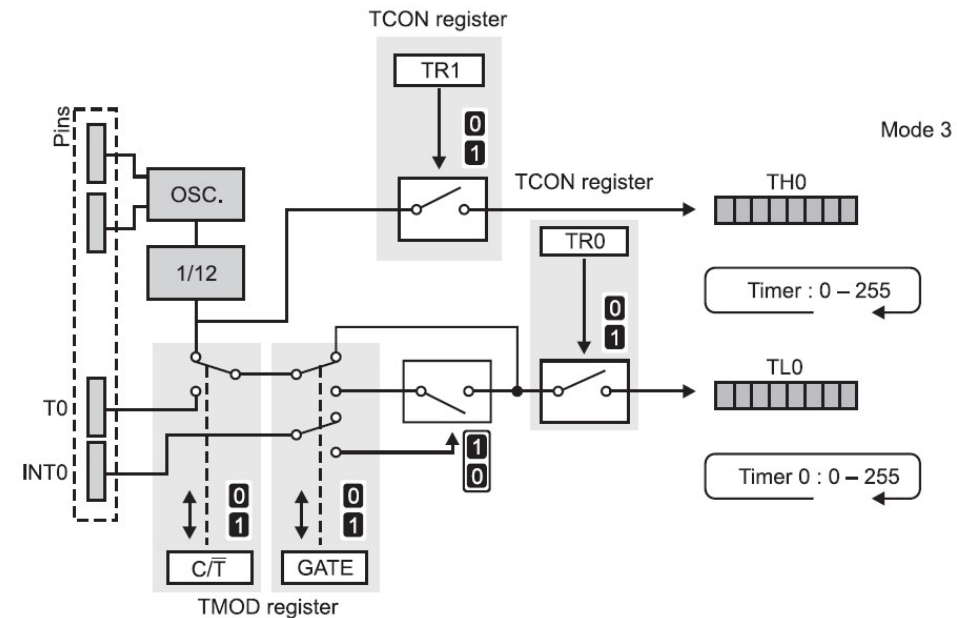


Modurile de operare la canalele Timer din MCU 8051

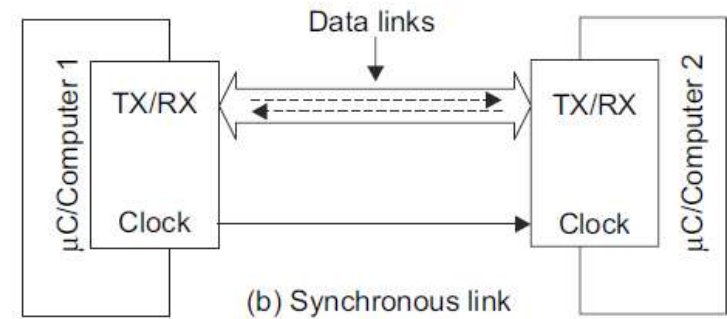
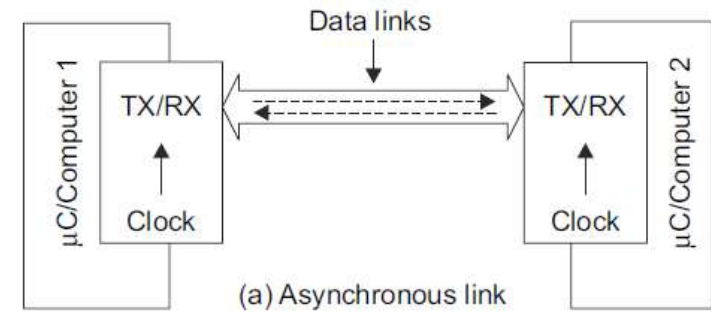
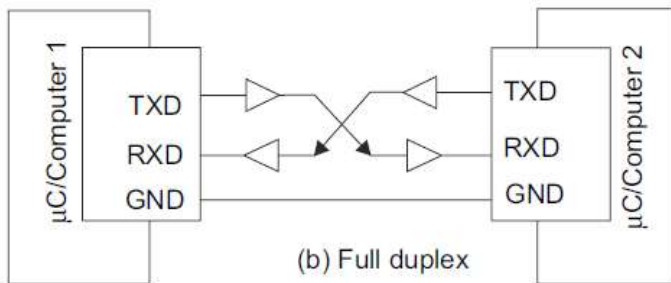
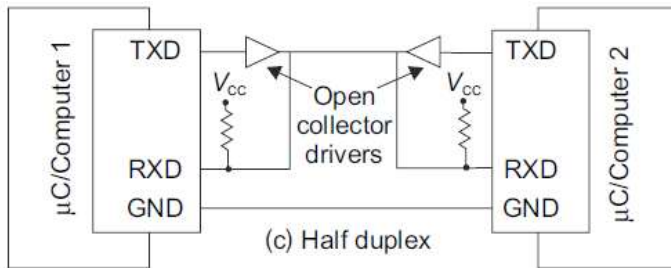
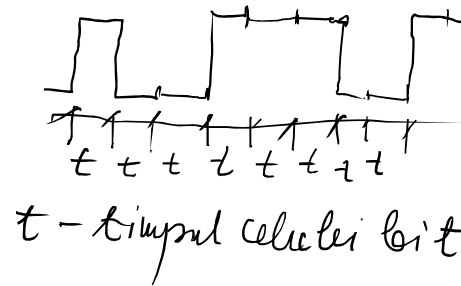
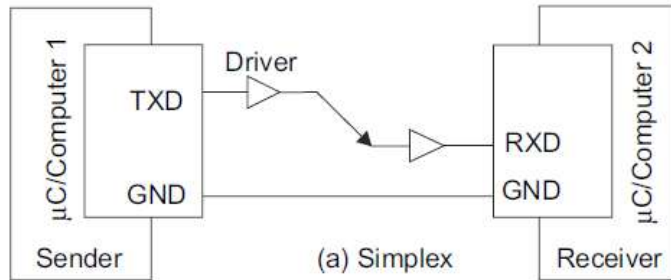
Modul 3 (M1=M0=1 in TMOD; doar la Timer 0)



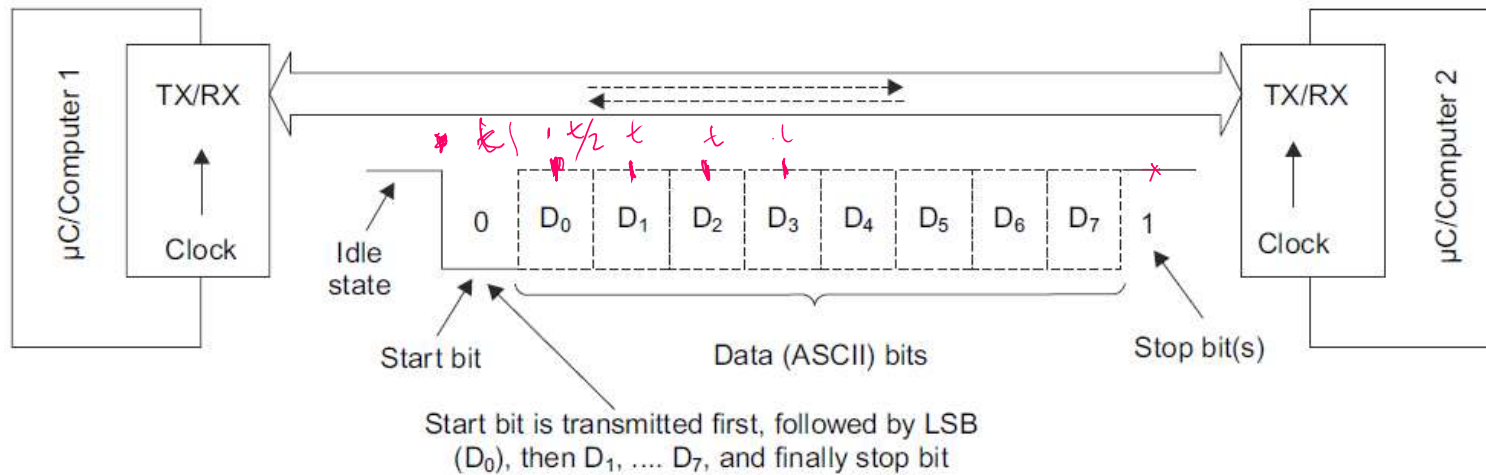
Obs. Modul de operare 3 este posibil doar pentru canalul timer 0 și în condițiile în care timerul 1 este folosit în modul 2 ca sursă a semnalului de ceas pentru UART. În acest mod mai multe resurse ale timerului 1 devin libere de sarcini și pot fi folosite în modul 3 de către timerul 0. În modul 3 apar două canale timer suplimentare la nivelul timerului 0 împreună cu resursele devenite libere din timerul 1.



Comunicatii seriale in microsisteme de calcul

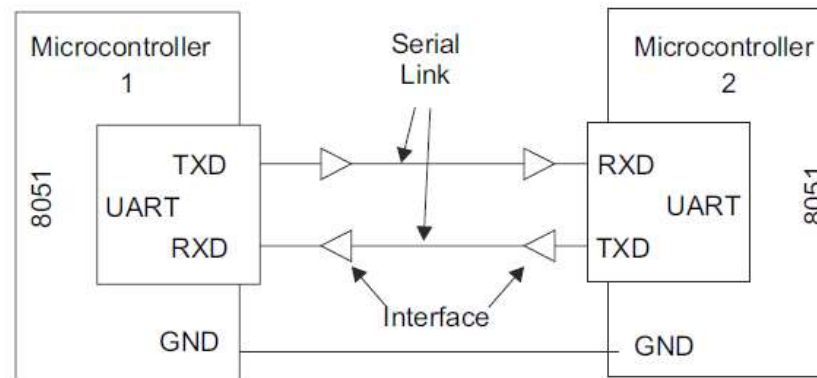


Pachet (frame) de date in format serial asincron



baud vs b/s

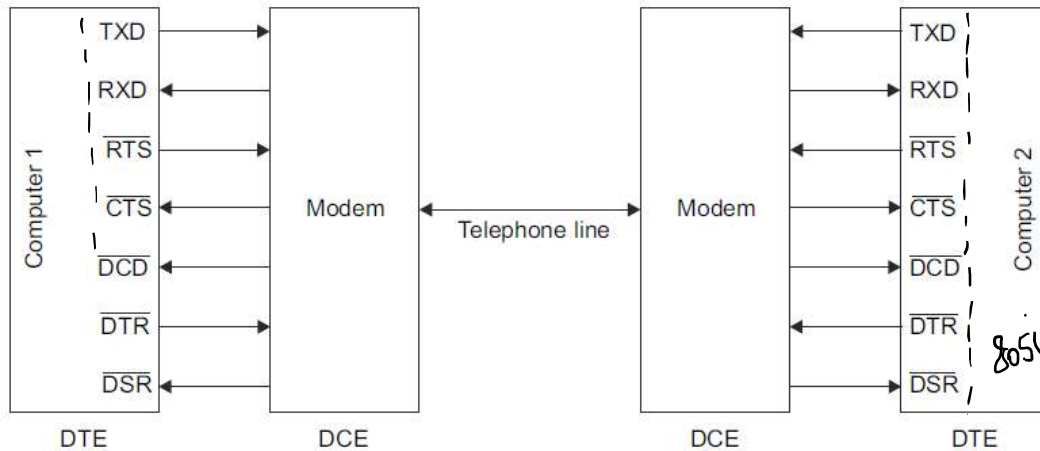
UART - UNIVERSAL ASYNCHRONOUS RECEIVER AND TRANSMITTER



Viteza de comunicatie seriala se masoara in b/s sau bps.
Multipli folositi: kbps, Mbps, Gbps.

Folosind tehnici de modulatie informatia se mai poate transmite prin simboluri: schimbare de faza, frecvente diferite sau amplitudini diferite (folosind semnale sinusoidale) – baud(s)

RS 232 - SERIAL DATA TRANSMISSION STANDARD



Protocolul Hand-shaking intre DTE si DCE

După aplicarea tensiunii de alimentare, DTE activează semnalul Data Terminal Ready (DTR) pentru a informa DCE (modemul) că este pregătit pentru comunicație. Similar, când modemul este gata, acesta activează semnalul Data Send Ready (DSR) către DTE. Odată ce semnalul DSR a fost primit, DTE face o cerere de utilizare a canalului de date prin activarea semnalului RTS pentru a începe transmisia. Apoi, se apelează modemul de la celălalt capăt (capătul receptorului). Acest modem (de obicei în modul answer - răspuns) răspunde prin trimiterea unui semnal cu o frecvență purtătoare de 2255 Hz. Când modemul de la terminalul expeditor primește acest semnal, acesta trimite Data Carrier Detect (DCD) către DTE, ulterior modemul trimite Clear To Send (CTS) care arată că linia este gata pentru transmisie.

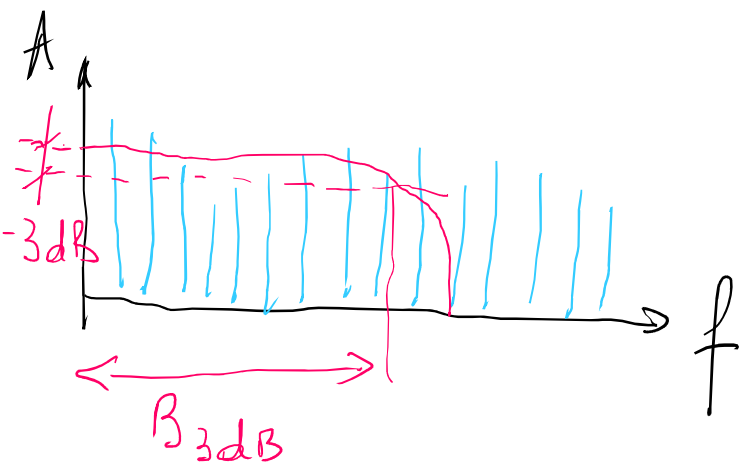
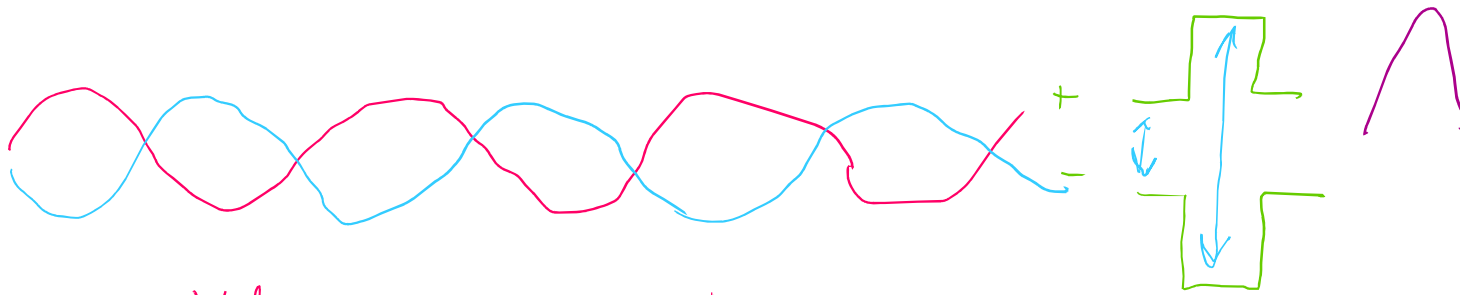
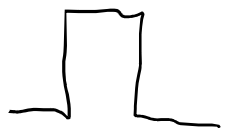
Imediat după primirea semnalului CTS, DTE de pe partea transmițătorului trimite date seriale pe ieșirea sa TXD. Similar, are loc un proces similar de tip hand-shaking și pentru sensul invers de comunicație.

DCE – Data communication equipment

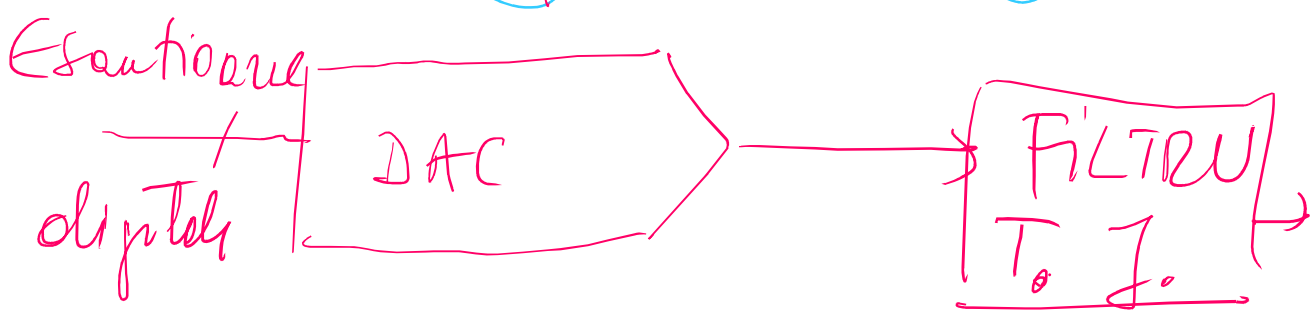
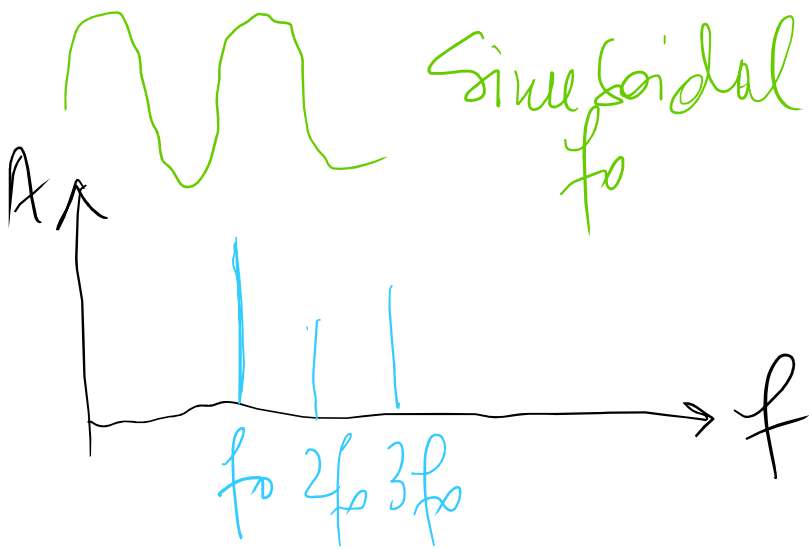
DTE – Data terminal equipment

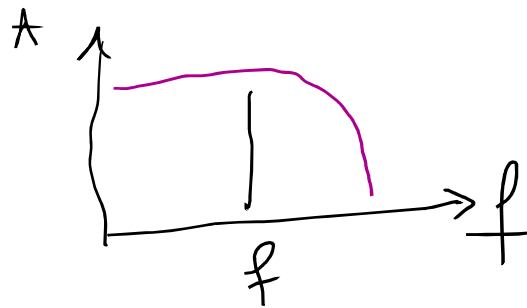
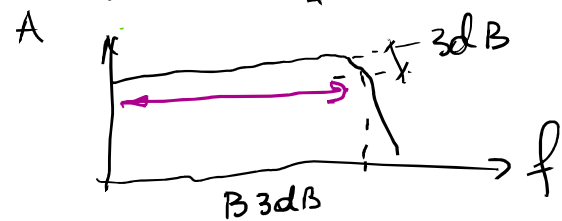
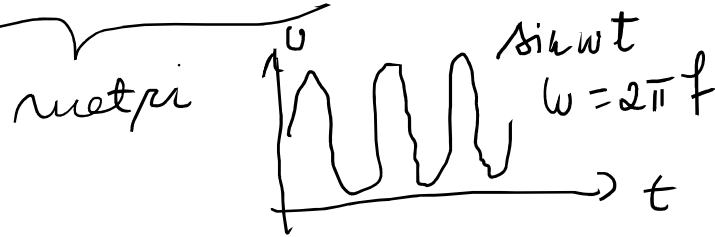
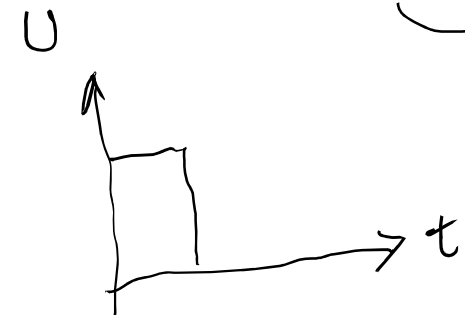
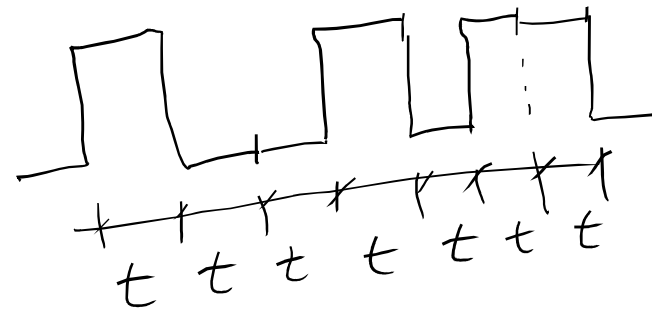
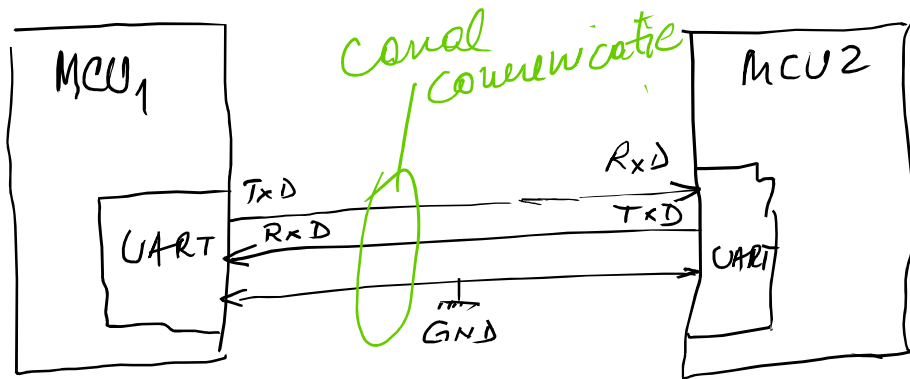
LL –Leased Line

Data Carrier Detect ($\overline{\text{DCD}}$)
Received Data (RXD)
Transmitted Data (TXD)
Data Terminal Ready ($\overline{\text{DTR}}$)
Signal Ground (GND)
Data Set Ready ($\overline{\text{DSR}}$)
Request To Send ($\overline{\text{RTS}}$)
Clear To Send ($\overline{\text{CTS}}$)
Ring Indicator (RI)

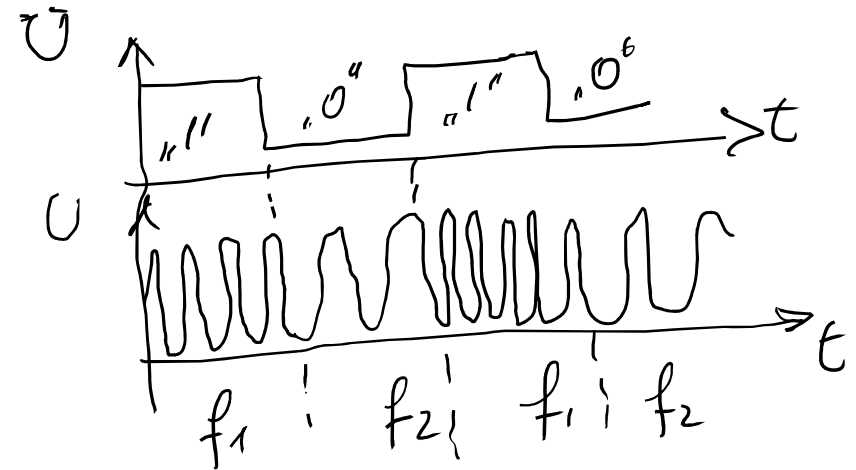


Vol
major: 10

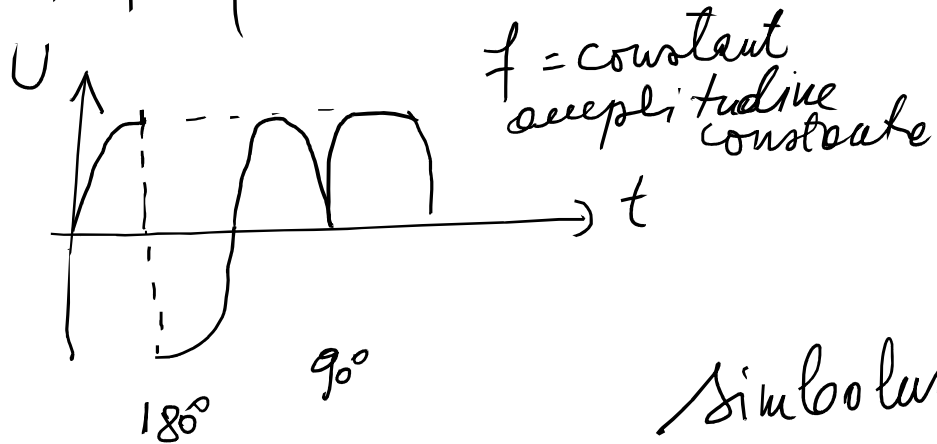




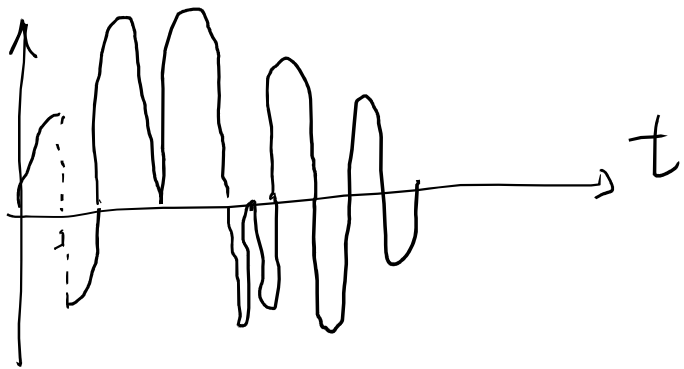
MODEM - modulator
demodulator
- amplitude i fazo konst



PM - phase modulation



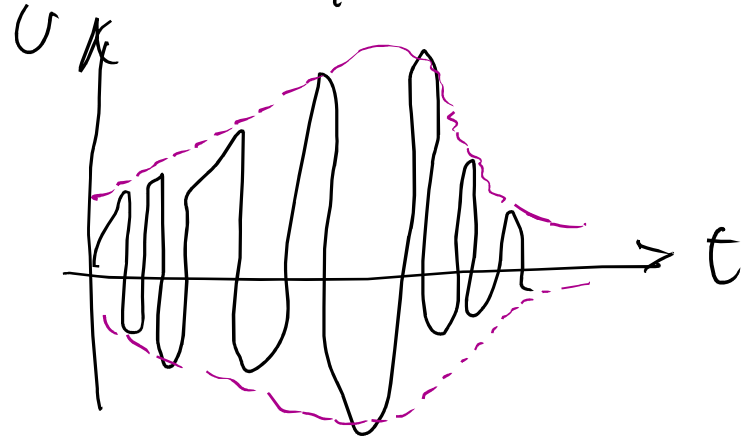
Simboluri



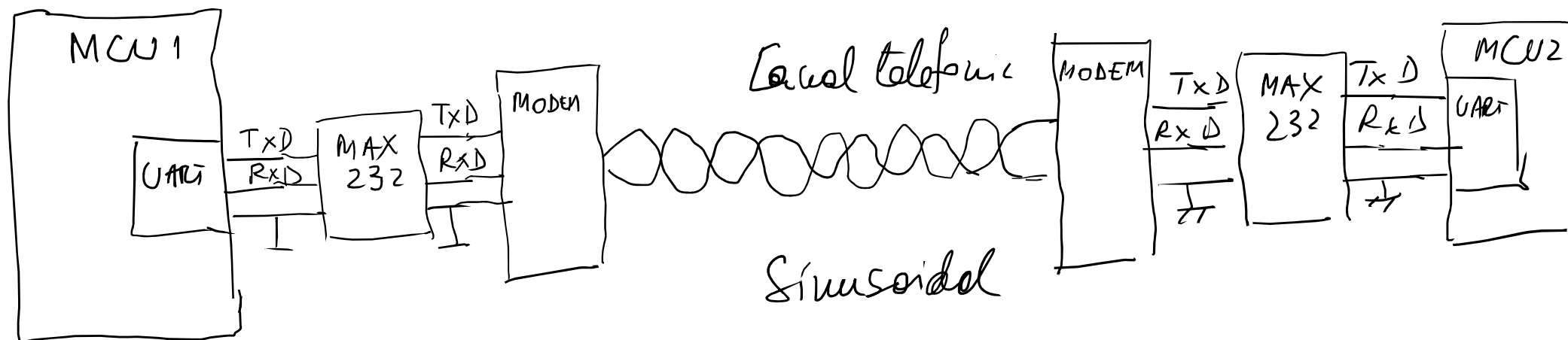
bands

0°	- 000
45°	- 001
90°	- 010
135°	- 011
180°	- 100
225°	- 101
270°	- 110
315°	- 111
360°	

AM - amplitude modulation



0°	- 00
90°	- 01
180°	- 10
270°	- 11



END 25 03 2026

Canalul UART din MCU 8051

SBUF – Serial Buffer

TI – Transmission Interrupt

TB8 - Transmission Bit 8

RB8 – Receive Bit 8

RI – Receive Interrupt

SCON – Serial Control

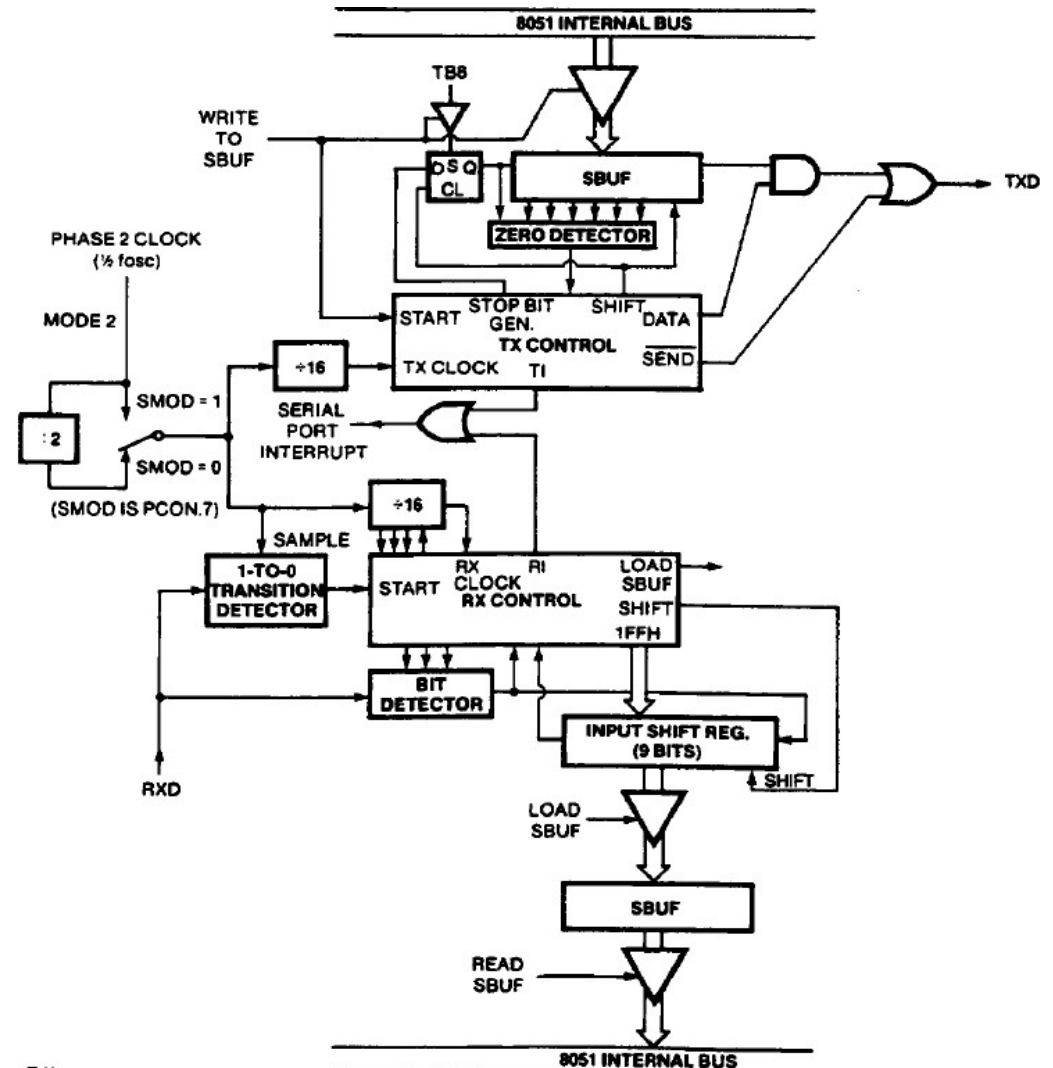
Viteze de comunicatie seriala RS232:

150bps, 300bps, 600bps, 1200bps, 2400bps,
4800bps, 9600bps, 19200bps, 38,4kbps, ...
14,4kbps, 28,8kbps, 57,6kbps

Care este timpul de transmisie al unui pachet serial cu 8 biti date, 1bit start, 1bit stop (10 biti) daca viteza de comunicatie este 9,6kbps?

$$t = 1/9600 \text{ [s]}$$

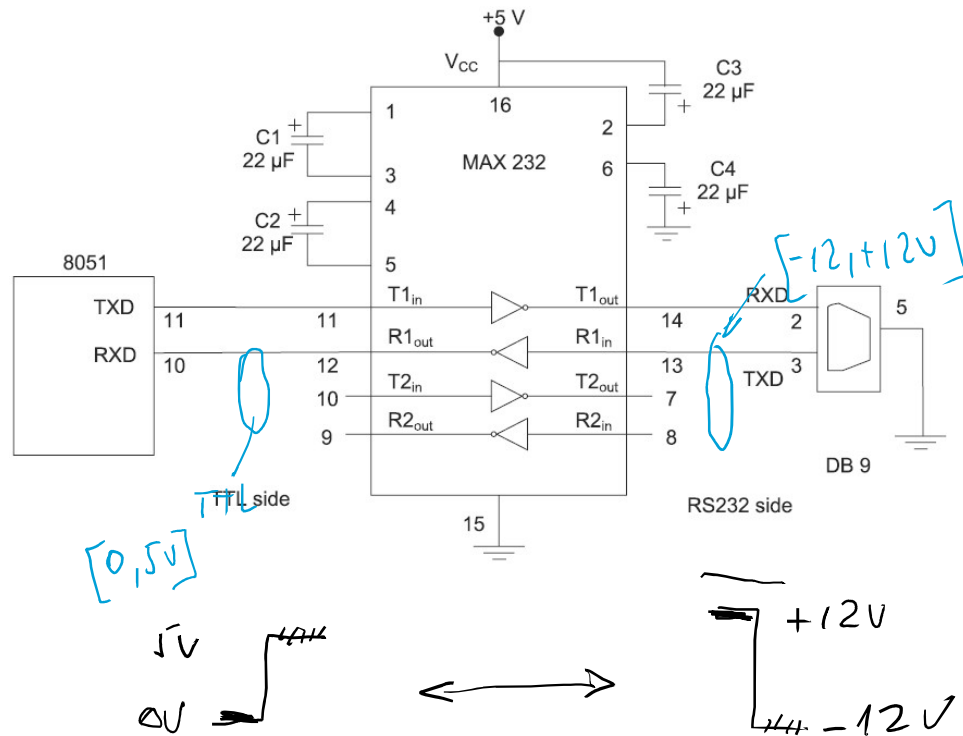
$$T_{\text{transm}} = t * n = 1/9600 * 10 = 1/960 \text{ [s]} = 0,00104 \text{ [s]} = 1,04 \text{ ms}$$



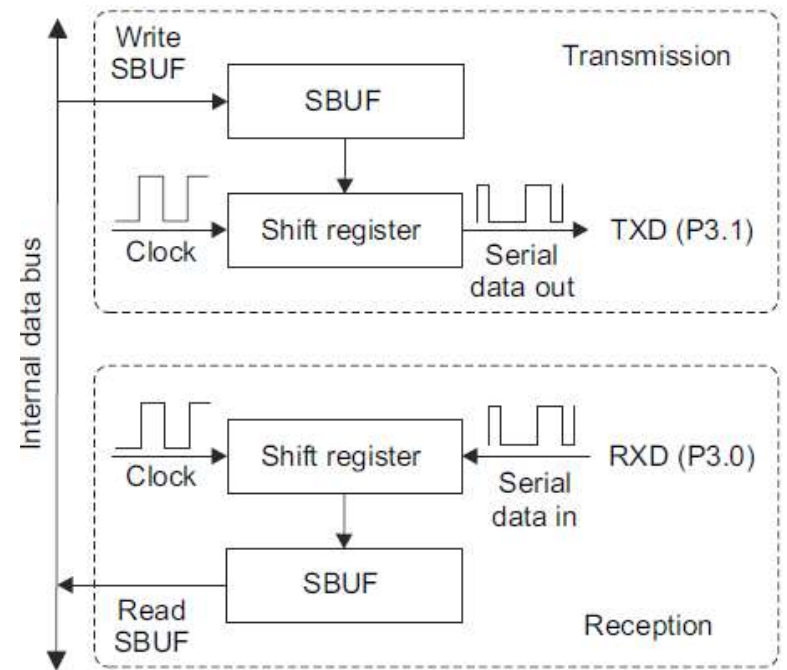
Canalul UART din MCU 8051

UART - UNIVERSAL ASYNCHRONOUS RECEIVER AND TRANSMITTER

Interfatare RS232 pentru MCU 8051



Registrul SBUF din MCU 8051



Canalul UART din MCU 8051

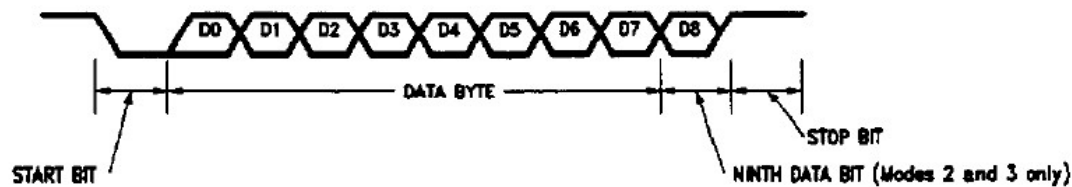
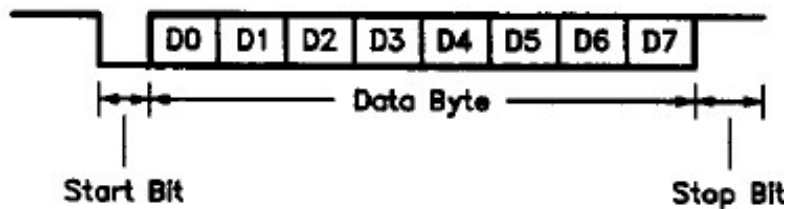
UART - UNIVERSAL ASYNCHRONOUS RECEIVER AND TRANSMITTER

Registrul SCON din MCU 8051

SCON	Register	(Serial				Port	Control		Register)
	0	0	0	0	0	0	0	0	Value after reset
SCON	SM0	SM1	SM2	REN	TB8	RB8	TI	RI	Bit name
	bit7	bit6	bit5	bit4	bit3	bit2	bit1	bit0	

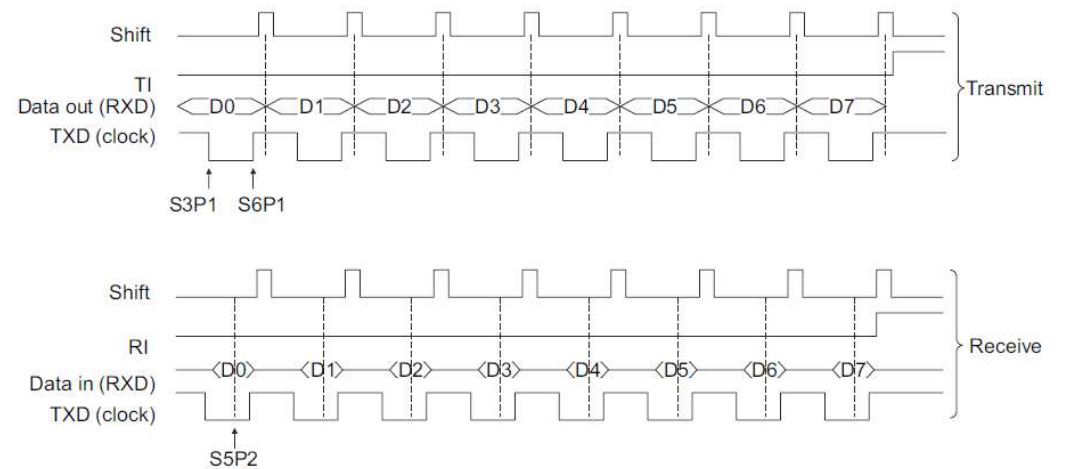
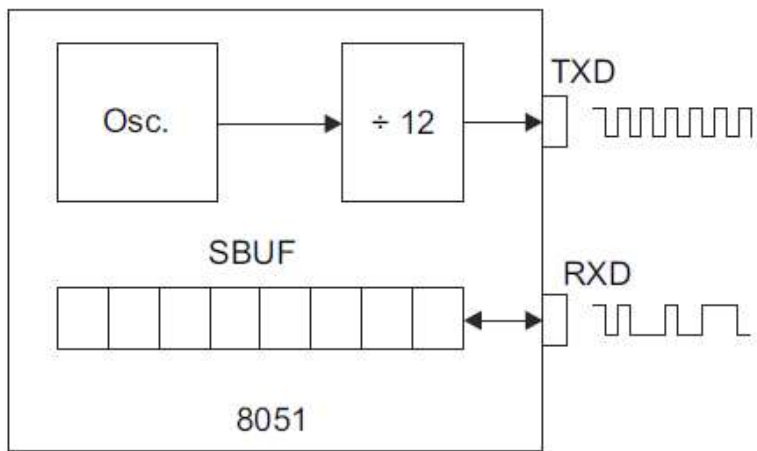
SM0	SM1	Mode	Description	Baud Rate
0	0	0	8-bit Shift Register	1/12 the quartz frequency
0	1	1	8-bit UART	Determined by the timer 1
1	0	2	9-bit UART	1/32 the quartz frequency (1/64 the quartz frequency)
1	1	3	9-bit UART	Determined by the timer 1

SM2 – activeaza modul de lucru multiprocesor in modurile de operare 2 si 3;
REN – receive enable (validare receptie);
TB8 – bitul 8 la transmisie in modurile de operare 2 si 3;
RB8 – bitul 8 la receptie in modurile de operare 2 si 3;
TI – transmit interrupt flag;
RI – receive interrupt flag



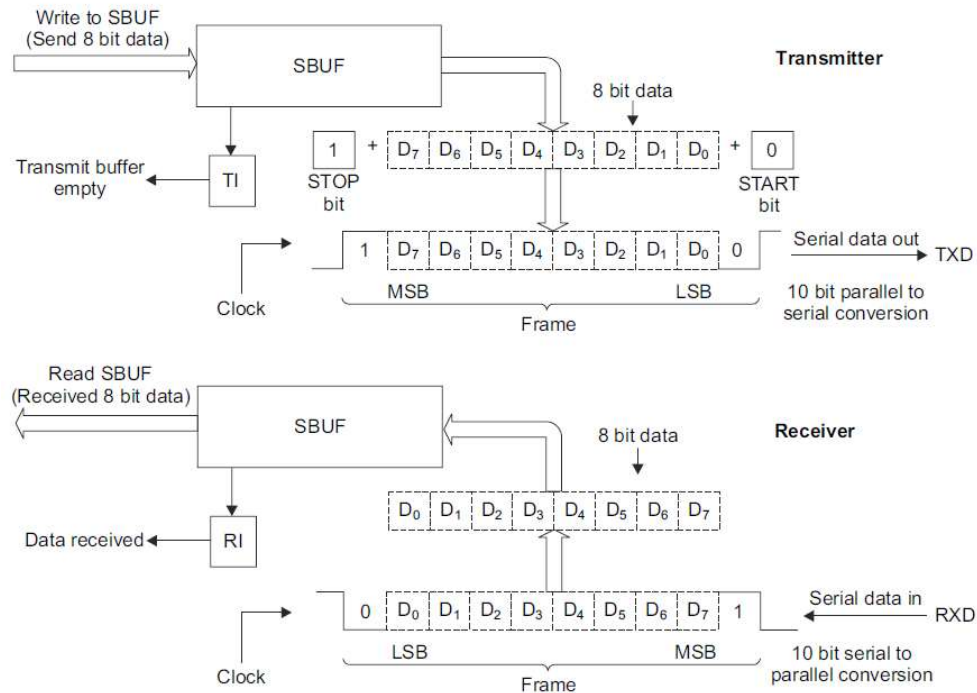
Canalul UART din MCU 8051

Modul 0 de operare registru serial la UART din MCU 8051

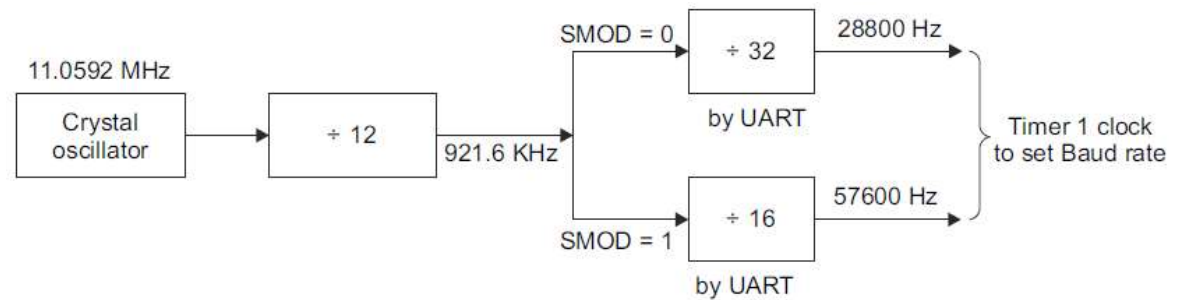


Canalul UART din MCU 8051

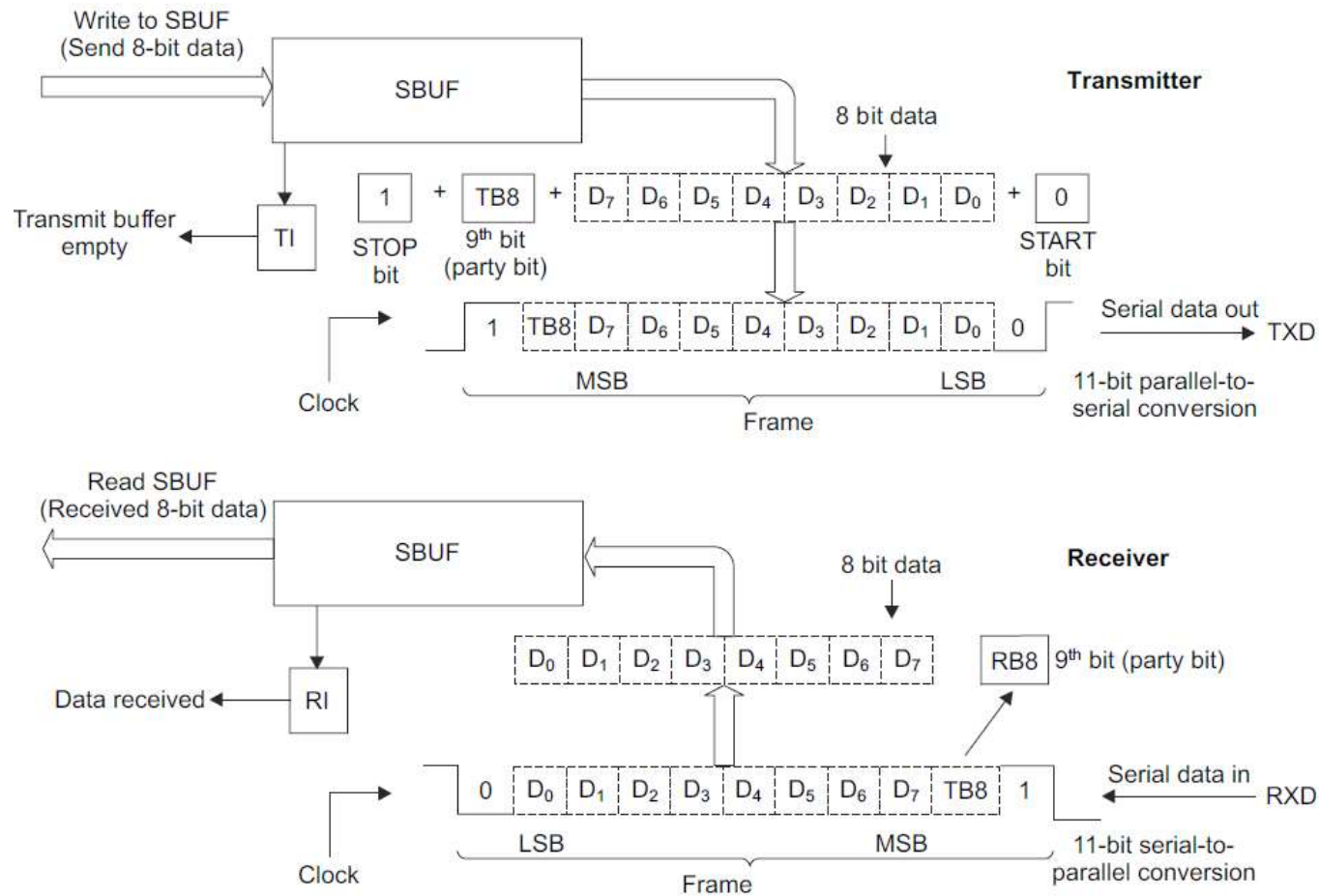
Modul 1 de operare UART din MCU 8051



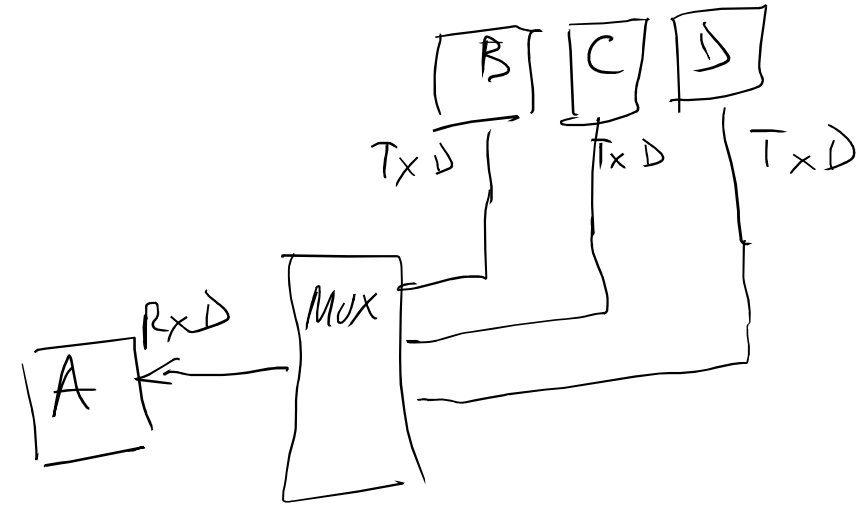
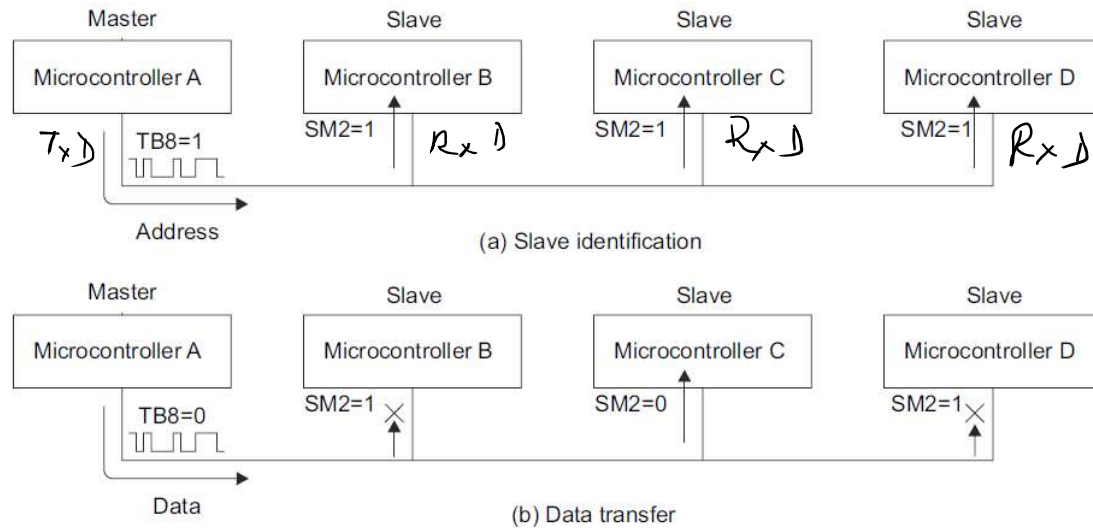
Timerul 1 ca sursa de ceas pentru UART din MCU 8051



Modul 2 de operare UART din MCU 8051; Modul 3 de operare UART din MCU 8051



Modul Comunicatie multiprocesor



Fiecarui MCU i se atribuie o adresă unică. Un octet de adresă este diferențiat de octetul de date folosind cel de-al 9 bit. Pentru adresă, al nouălea bit = 1, iar pentru octetul de date, acest bit este 0. Când microcontrolerul A (master) dorește să transmită date către unul dintre mai mulți sclavi (să zicem, microcontrolerului C), acesta trimite mai întâi octetul de adresă. (Toți sclavii vor avea inițial SM2 = 1 așa cum se arată în figura). Toți sclavii vor primi acest octet de adresă, generandu-se astfel întrerupere în toți sclavii. Rutina de tratare a întreruperii va verifica dacă se potrivește cu adresa proprie. Deoarece toți sclavii au adrese unice, numai unui sclav (microcontrolerul C) i se va potrivi cu adresa. Astfel sclavul dorit este identificat, iar acest sclav isi va șterge bitul SM2 al registrului SCON și va fi gata să primească octeții de date trimiși de master așa cum se arată în figura. Ceilalti sclavi (neadresați) ignoră octeții de date primiți, deoarece bitul lor SM2 este încă 1.

Canalul UART din MCU 8051

Obtinerea frecventelor de comunicatie seriala la canalul UART din MCU 8051

$$\text{Mode 0 Baud Rate} = \frac{\text{Oscillator Frequency}}{12}$$

$$\text{Modes 1, 3 Baud Rate} = \frac{2^{\text{SMOD}}}{32} \times \frac{\text{Oscillator Frequency}}{12 \times [256 - (\text{TH1})]}$$

(standardizare) (quartz) (TIMER 1 in modul 2)

$$\text{Mode 2 Baud Rate} = \frac{2^{\text{SMOD}}}{64} \times (\text{Oscillator Frequency})$$

$$\text{Modes 1, 3 Baud Rate} = \frac{2^{\text{SMOD}}}{32} \times (\text{Timer 1 Overflow Rate})$$

(TIMER 1 in modul 1)

PCON

TH1

Baud Rate	fosc	SMOD	Timer 1		
			C/T	Mode	Reload Value
Mode 0 Max: 1 MHz	12 MHz	X	X	X	X
Mode 2 Max: 375K	12 MHz	1	X	X	X
Modes 1, 3: 62.5K	12 MHz	1	0	2	FFH
19.2K	11.059 MHz	1	0	2	FDH
9.6K	11.059 MHz	0	0	2	FDH
4.8K	11.059 MHz	0	0	2	FAH
2.4K	11.059 MHz	0	0	2	F4H
1.2K	11.059 MHz	0	0	2	E8H
137.5	11.986 MHz	0	0	2	1DH
110	6 MHz	0	0	2	72H
110	12 MHz	0	0	1	FEEDH

Logica de control a intreruperilor in MCU 8051

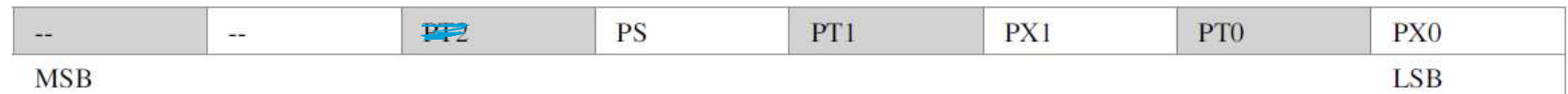
Gestioneaza 5 surse de intreruperi la nivelul MCU 8051. Trei surse sunt interne: cele 2 canale timer si unitatea UART, iar 2 surse sunt externe (se primesc pe pinii INTi).

Registrul IE
Interrupt Enable



Valideaza individual fiecare sursa de intreruperi si global operarea in intreruperi.

Registrul IP
Interrupt Priority

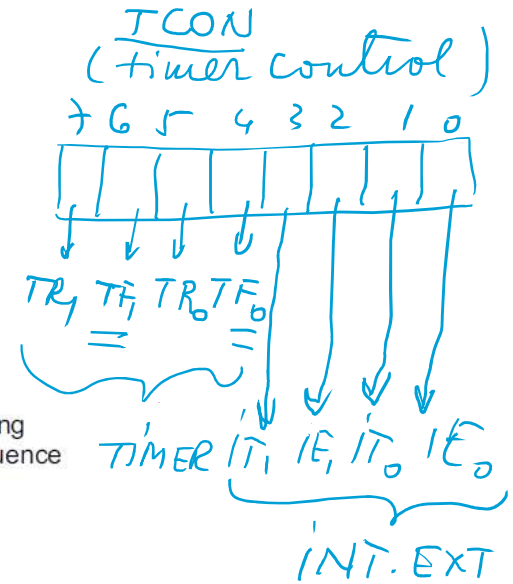
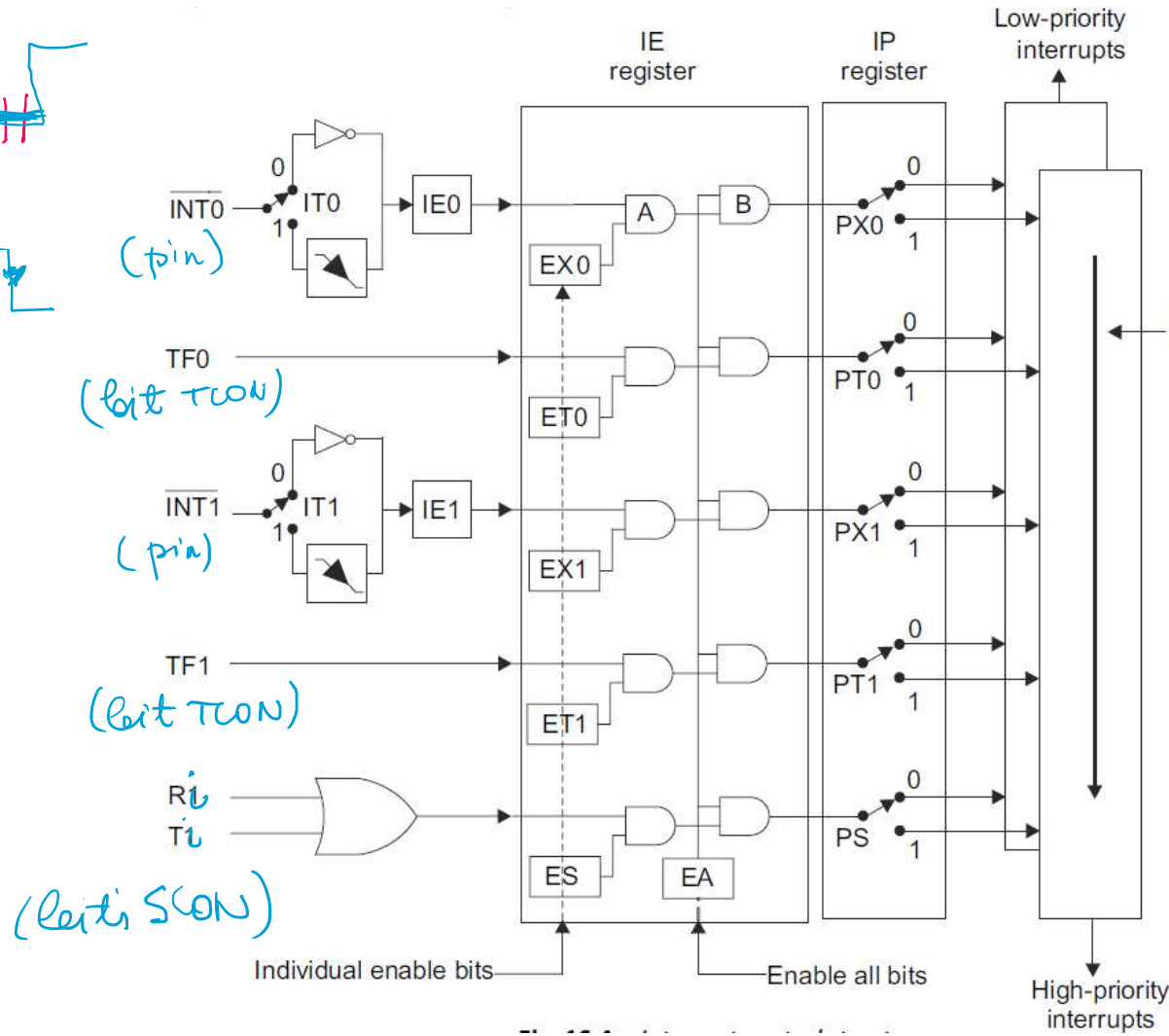


Plaseaza sursele de intreruperi pe 2 nivele de prioritati.

Logica de control a intreruperilor in MCU 8051

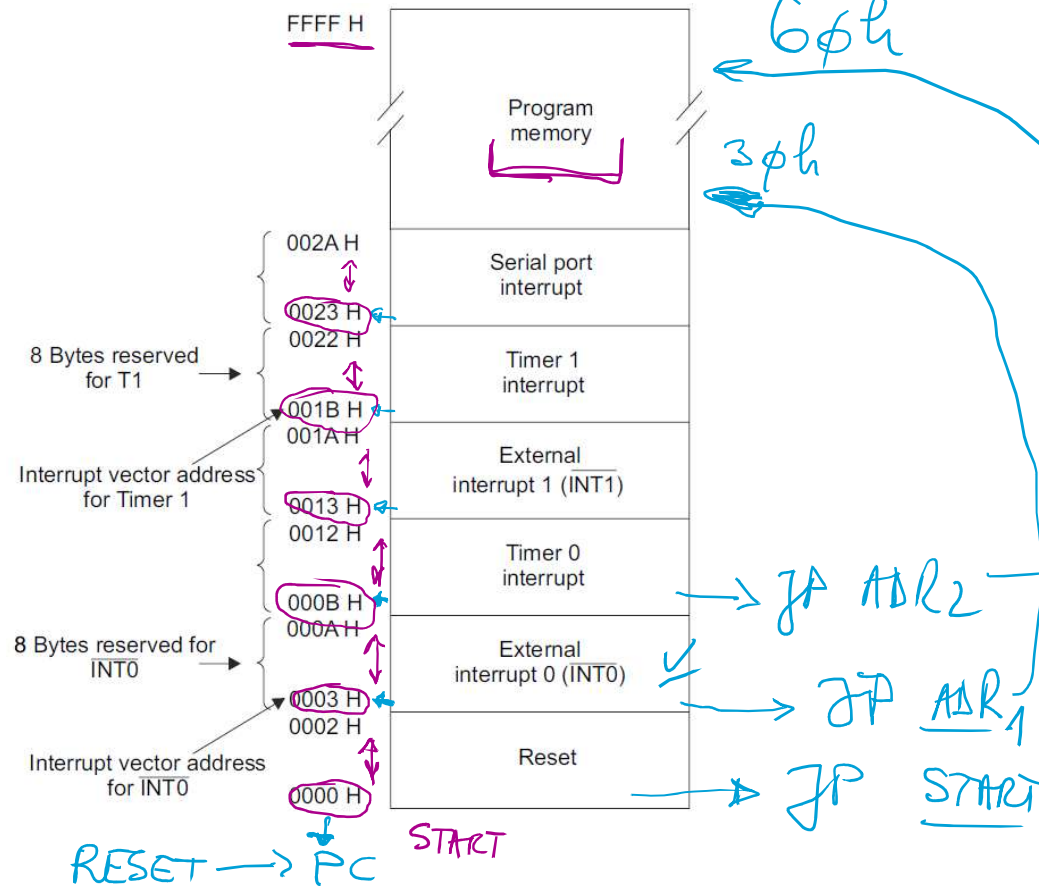
IT_i - interrupt type
 IE_i - edge
 $IT_0 = 0$
 (TCON)

$IT_0 = 1$
 (TCON)



- ?
1. SERIAL (H)
 2. TF1 (L)
 3. INT1 (?)

Logica de control a intreruperilor in MCU 8051

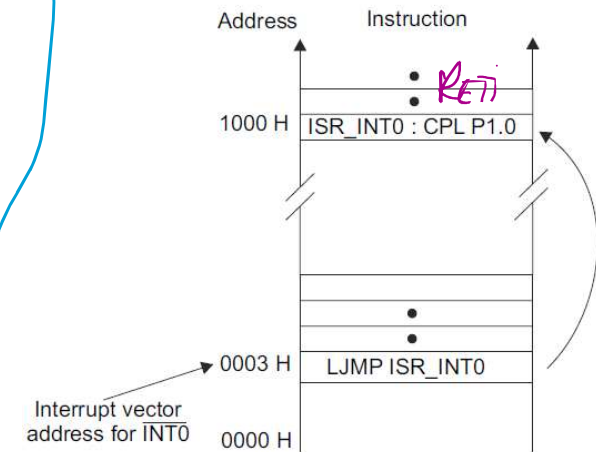


ISR_INT0
in service routine INT0

→ RETI - return from interrupt

CPL P1.0 → P1.0

LJMP - long jump



Logica de control a intreruperilor in MCU 8051

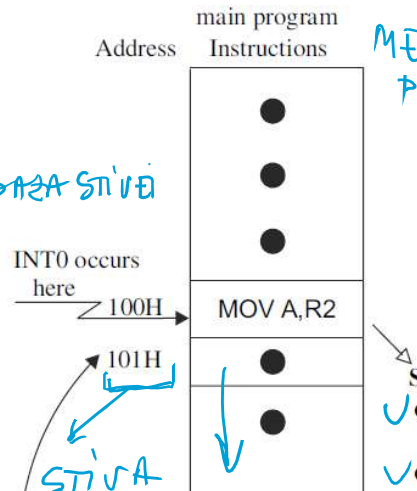
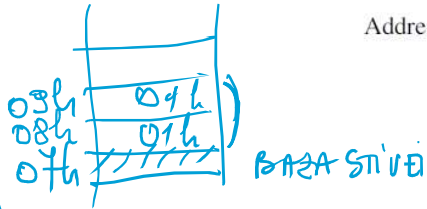
MOV A, R2
A ← R2

move

Progr. principal

MEM PROGRAME

RAM
INTERNA



101H

STIVA

INT0 occurs here

101H

Resume execution of next instruction in main program

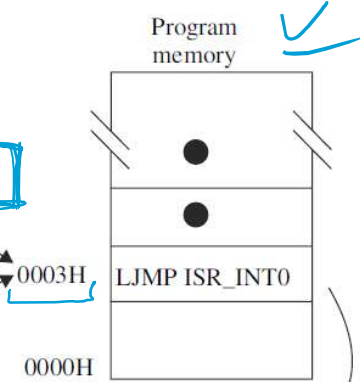
Steps taken by 8051 hardware

- Complete execution of current instruction
- Save PC (return address) on the stack
- Save status of IE flag internally
- Interrupts of same or lower priority are disabled
- PC is loaded with ISR vector address (0003H)

Steps taken by RETI instruction

- IE flag is restored, and interrupts are enabled
- PC is retrieved from stack

Interrupt vector address for INT0



ISR_INT0:

- Save all registers used in this ISR on to the stack using PUSH instructions
- Instructions of ISR
- Retrieve all saved registers using POP instructions
- RETI

MEM PROG

(LIFO)

Setul de instructiuni al MCU 8051

Instructiuni aritmetice

Instructiuni logice

Instructiuni de transfer cu memoria de date interna

Instructiuni de transfer cu memoria de date externa

Instructiuni de transfer cu memoria de programe

Instructiuni de manipulare la nivel de bit

Instructiuni de salt, lucru cu subrutine

Instructiuni de salt conditionat

Adresare directa

Adresare indirecta

Adresare cu registru

Adresare imediata

Notatii folosite

Rn	— Register R7–R0 of the currently selected Register Bank.
direct	— 8-bit internal data location's address. This could be an Internal Data RAM location (0–127) or a SFR [i.e., I/O port, control register, status register, etc. (128–255)].
$@R_i$	— 8-bit internal data RAM location (0–255) addressed indirectly through register R1 or R0.
#data	— 8-bit constant included in instruction.
#data 16	— 16-bit constant included in instruction.
addr 16	— 16-bit destination address. Used by LCALL & LJMP. A branch can be anywhere within the 64K-byte Program Memory address space.
addr 11	— 11-bit destination address. Used by ACALL & AJMP. The branch will be within the same 2K-byte page of program memory as the first byte of the following instruction.
rel	— Signed (two's complement) 8-bit offset byte. Used by SJMP and all conditional jumps. Range is –128 to +127 bytes relative to first byte of the following instruction.
bit	— Direct Addressed bit in Internal Data RAM or Special Function Register.

$R_i \in \{R_0, R_1\}$
 $@R_i \leftrightarrow (R_i)$
 $\#d_8 ; \#d_{16}$

Instructiuni care afecteaza indicatorii de conditii

Instruction	Flag			Instruction	Flag		
	C	OV	AC		C	OV	AC
ADD	X	X	X	CLR C	O		
ADDC	X	X	X	CPL C	X		
SUBB	X	X	X	ANL C, bit	X		
MUL	O	X		ANL C, /bit	X		
DIV	O	X		ORL C, bit	X		
DA	X			ORL C, /bit	X		
RRC	X			MOV C, bit	X		
RLC	X			CJNE	X		
SETB C	1						

$ADD\ A, 31h \quad A \leftarrow A + (31h)$
 $ADD\ A, \#31h \quad A \leftarrow A + \#31h$
 $\{ \begin{array}{l} MOV\ R_1, 31h \\ ADD\ A, @R_1 \end{array} \quad \begin{array}{l} R_1 \leftarrow (31h) \\ A \leftarrow A + (?) \end{array}$
 $\{ \begin{array}{l} MOV\ R_1, \#31h \\ ADD\ A, @R_1 \end{array} \quad \begin{array}{l} A \leftarrow A + (?) \\ ((R_1)) \end{array}$

ADDC - addition with carry
 SUBB - subtract with borrow

Instructioni aritmetice

24 0010 0100 (BCD)
 18(h) 0001 1000 (bin)

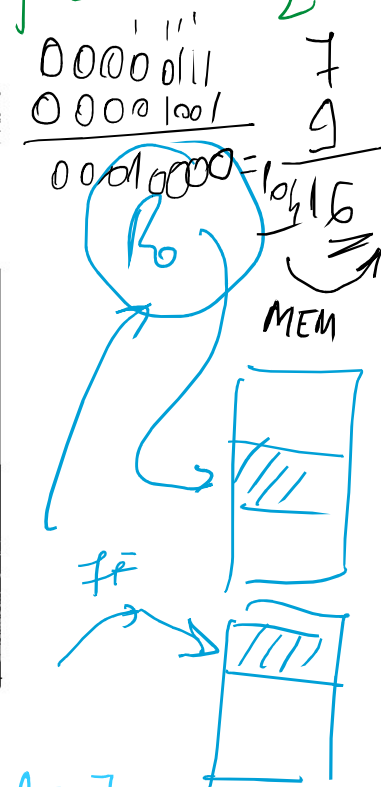
CLK = 12MHz

Cy ← 0
 SUBB = SUB

SUBB - Subtract with Borrow

C = Carry
 MUL - multiply
 DIV - divide

Mnemonic	Operation	Addressing Modes				Execution Time (μs)
		Dir	Ind	Reg	Imm	
ADD A, <byte>	A = A + <byte>	X	X	X	X	1
ADDC A, <byte>	A = A + <byte> + C	X	X	X	X	1
SUBB A, <byte>	A = A - <byte> - C	X	X	X	X	1
INC A	A = A + 1	Accumulator only				1
INC <byte>	<byte> = <byte> + 1	X	X	X		1
INC DPTR	DPTR = DPTR + 1	Data Pointer only				2
DEC A	A = A - 1	Accumulator only				1
DEC <byte>	<byte> = <byte> - 1	X	X	X		1
MUL AB	B:A = B x A	ACC and B only				4
DIV AB	A = Int [A/B] B = Mod [A/B]	ACC and B only				4
DA A	Decimal Adjust	Accumulator only				1



ADD - addition
 ADD A, 7FH = { MOV R0, #7FH
 ADD A, @R0
 ADD A, #7FH = { MOV R7, #7FH
 ADD A, R7

ADD A, 7FH (direct addressing)
 ADD A, @R0 (indirect addressing)
 ADD A, R7 (register addressing)
 ADD A, #127 (immediate constant)

A ← A + (7FH)
 A ← A + (R0)
 A ← A + R7
 A ← A + #7FH

A = 7
 B = 3
 Cat 2
 Real 1

79h / 79h (10) → 0111 1001 (12)
 79h (10) → 0111 1001 (12)
 ADD A, #7FH = 7FH

DIV: 7 / 3 → 2
 A
 B

Mnemonic	Description	Byte	Oscillator Period
ARITHMETIC OPERATIONS			
ADD A,Rn	Add register to Accumulator	1	12
ADD A,direct	Add direct byte to Accumulator	2	12
ADD A,@Ri	Add indirect RAM to Accumulator	1	12
ADD A,#data	Add immediate data to Accumulator	2	12
ADDC A,Rn	Add register to Accumulator with Carry	1	12
ADDC A,direct	Add direct byte to Accumulator with Carry	2	12
ADDC A,@Ri	Add indirect RAM to Accumulator with Carry	1	12
ADDC A,#data	Add immediate data to Acc with Carry	2	12
SUBB A,Rn	Subtract Register from Acc with borrow	1	12
SUBB A,direct	Subtract direct byte from Acc with borrow	2	12
SUBB A,@Ri	Subtract indirect RAM from ACC with borrow	1	12
SUBB A,#data	Subtract immediate data from Acc with borrow	2	12
INC A	Increment Accumulator	1	12
INC Rn	Increment register	1	12
INC direct	Increment direct byte	2	12
INC @Ri	Increment direct RAM	1	12
DEC A	Decrement Accumulator	1	12
DEC Rn	Decrement Register	1	12
DEC direct	Decrement direct byte	2	12
DEC @Ri	Decrement indirect RAM	1	12

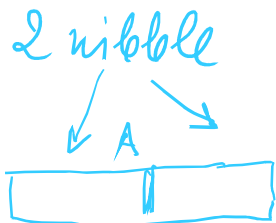
Mnemonic	Description	Byte	Oscillator Period
ARITHMETIC OPERATIONS (Continued)			
INC DPTR	Increment Data Pointer	1	24
MUL AB	Multiply A & B	1	48
DIV AB	Divide A by B	1	48
DA A	Decimal Adjust Accumulator	1	12

Instructioni logice

$f_{clk} = 12MHz$

$25h \leftarrow 25h \text{ AND } A$
 $15h \leftarrow 25h \text{ AND } \#25h$

Mnemonic	Operation	Addressing Modes				Execution Time (μs)
		Dir	Ind	Reg	Imm	
ANL A, <byte>	A = A .AND. <byte>	X	X	X	X	1
ANL <byte>, A	<byte> = <byte> .AND. A	X				1
ANL <byte>, #data	<byte> = <byte> .AND. #data	X				2
ORL A, <byte>	A = A .OR. <byte>	X	X	X	X	1
ORL <byte>, A	<byte> = <byte> .OR. A	X				1
ORL <byte>, #data	<byte> = <byte> .OR. #data	X				2
XRL A, <byte>	A = A .XOR. <byte>	X	X	X	X	1
XRL <byte>, A	<byte> = <byte> .XOR. A	X				1
XRL <byte>, #data	<byte> = <byte> .XOR. #data	X				2
ORL A CLR A	A = 00H	Accumulator only				1
CPL A	A = .NOT. A	Accumulator only				1
RL A	Rotate ACC Left 1 bit	Accumulator only				1
RLC A	Rotate Left through Carry	Accumulator only				1
RR A	Rotate ACC Right 1 bit	Accumulator only				1
RRC A	Rotate Right through Carry	Accumulator only				1
SWAP A	Swap Nibbles in A	Accumulator only				1

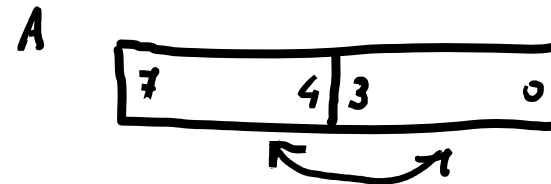


2 nibble
 4 3h
 3 4h
 SWAP

ANL A, 7FH (direct addressing)
 ANL A, @R1 (indirect addressing)
 ANL A, R6 (register addressing)
 ANL A, #53H (immediate constant)

Mnemonic	Description	Byte	Oscillator Period
LOGICAL OPERATIONS			
ANL A,Rn	AND Register to Accumulator	1	12
ANL A,direct	AND direct byte to Accumulator	2	12
ANL A,@Ri	AND indirect RAM to Accumulator	1	12
ANL A,#data	AND immediate data to Accumulator	2	12
ANL direct,A	AND Accumulator to direct byte	2	12
ANL direct,#data	AND immediate data to direct byte	3	24
ORL A,Rn	OR register to Accumulator	1	12
ORL A,direct	OR direct byte to Accumulator	2	12
ORL A,@Ri	OR indirect RAM to Accumulator	1	12
ORL A,#data	OR immediate data to Accumulator	2	12
ORL direct,A	OR Accumulator to direct byte	2	12
ORL direct,#data	OR immediate data to direct byte	3	24
XRL A,Rn	Exclusive-OR register to Accumulator	1	12
XRL A,direct	Exclusive-OR direct byte to Accumulator	2	12
XRL A,@Ri	Exclusive-OR indirect RAM to Accumulator	1	12
XRL A,#data	Exclusive-OR immediate data to Accumulator	2	12
XRL direct,A	Exclusive-OR Accumulator to direct byte	2	12
XRL direct,#data	Exclusive-OR immediate data to direct byte	3	24
CLR A	Clear Accumulator	1	12
CPL A	Complement Accumulator	1	12

Mnemonic	Description	Byte	Oscillator Period
LOGICAL OPERATIONS (Continued)			
RL A	Rotate Accumulator Left	1	12
RLC A	Rotate Accumulator Left through the Carry	1	12
RR A	Rotate Accumulator Right	1	12
RRC A	Rotate Accumulator Right through the Carry	1	12
SWAP A	Swap nibbles within the Accumulator	1	12



END 8 apr 2026

dest - adr ind R₁
 src - adr reg R₄

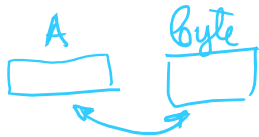
Instructiuni de transfer cu memoria de date interna

mov - move

f_{clk} = 12 MHz

Mnemonic	Operation	Addressing Modes				Execution Time (μs)
		Dir	Ind	Reg	Imm	
MOV A, <src>	A = <src>	X	X	X	X	1
MOV <dest>, A	<dest> = A	X	X	X		1
MOV <dest>, <src>	<dest> = <src>	X	X	X	X	2
MOV DPTR, #data16	DPTR = 16-bit immediate constant.				X	2
PUSH <src>	INC SP : MOV "@SP", <src>	X				2
POP <dest>	MOV <dest>, "@SP" : DEC SP	X				2
XCH A, <byte>	ACC and <byte> exchange data	X	X	X		1
XCHD A, @Ri	ACC and @Ri exchange low nibbles		X			1

MOV @R₁, R₄



XCHD



Mnemonic	Description	Byte	Oscillator Period
DATA TRANSFER			
MOV A,Rn	Move register to Accumulator	1	12
MOV A,direct	Move direct byte to Accumulator	2	12
MOV A,@Ri	Move indirect RAM to Accumulator	1	12
MOV A,#data	Move immediate data to Accumulator	2	12
MOV Rn,A	Move Accumulator to register	1	12
MOV Rn,direct	Move direct byte to register	2	24
MOV Rn,#data	Move immediate data to register	2	12
MOV direct,A	Move Accumulator to direct byte	2	12
MOV direct,Rn	Move register to direct byte	2	24
MOV direct,direct	Move direct byte to direct	3	24
MOV direct,@Ri	Move indirect RAM to direct byte	2	24
MOV direct,#data	Move immediate data to direct byte	3	24
MOV @Ri,A	Move Accumulator to indirect RAM	1	12

$f_{osc} = 12 \text{ MHz}$
 $12 \text{ T} = 1 \mu\text{s}$
 $\rightarrow \frac{1}{12} \mu\text{s}$

Mnemonic	Description	Byte	Oscillator Period
DATA TRANSFER (Continued)			
MOV @Ri,direct	Move direct byte to indirect RAM	2	24
MOV @Ri,#data	Move immediate data to indirect RAM	2	12
MOV DPTR,#data16	Load Data Pointer with a 16-bit constant	3	24
MOVC A,@A+DPTR	Move Code byte relative to DPTR to Acc	1	24
MOVC A,A+PC	Move Code byte relative to PC to Acc	1	24
MOVX A,@Ri	Move External RAM (8-bit addr) to Acc	1	24
MOVX A,DPTR	Move External RAM (16-bit addr) to Acc	1	24
MOVX @Ri,A	Move Acc to External RAM (8-bit addr)	1	24
MOVX @DPTR,A	Move Acc to External RAM (16-bit addr)	1	24
PUSH direct	Push direct byte onto stack	2	24
POP direct	Pop direct byte from stack	2	24
XCH A,Rn	Exchange register with Accumulator	1	12
XCH A,direct	Exchange direct byte with Accumulator	2	12
XCH A,@Ri	Exchange indirect RAM with Accumulator	1	12
XCHD A,@Ri	Exchange low-order Digit indirect RAM with Acc	1	12

Instructiuni de transfer cu memoria de programe

PSEN (PSEL)

Mnemonic	Operation	Execution Time (μs)
MOVC A,@A+DPTR	Read Pgm Memory at (A+DPTR)	2
MOVC A,@A+PC	Read Pgm Memory at (A+PC)	2

MOVC A, @A+DPTR
 ↓
 move ↓
 code ↓
 dest ↓
 8b 16b

MOVC A, @A+PC
 A ← (A+PC)

MOVX @DPTR, A
 (DPTR) ← A
 16b
 64Koct

{ MOVX A, @Ri
 A ← (Ri)
 { MOV @Ri, A
 (Ri) ← A

int
ext (EA)

MOVX
 move ↓
 external

$\overline{RD}$ $\overline{WR}$

MOVX A, @DPTR
16b

↓
64Koct

A ← (DPTR)

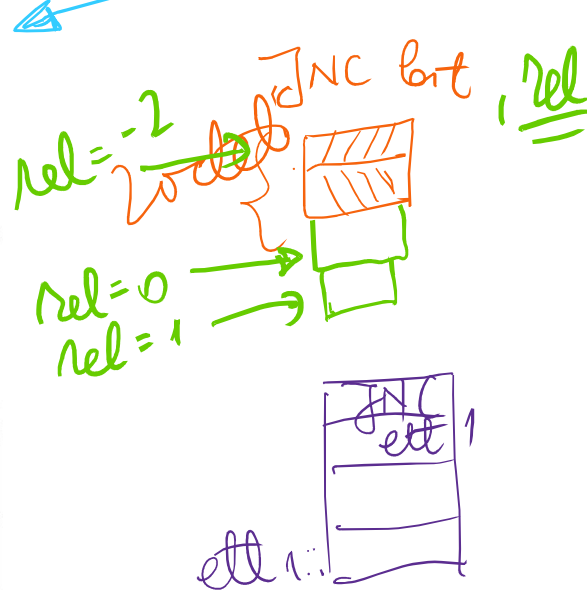
Instructiuni de transfer cu memoria de date externa

Address Width	Mnemonic	Operation	Execution Time (μs)
8 bits	MOVX A,@Ri	Read external RAM @Ri	2
8 bits	MOVX @Ri,A	Write external RAM @Ri	2
16 bits	MOVX A,@DPTR	Read external RAM @DPTR	2
16 bits	MOVX @DPTR,A	Write external RAM @DPTR	2

Instructiuni de manipulare la nivel de bit

Mnemonic		Operation	Execution Time (μ s)
ANL	C,bit	$C = C \text{ AND bit}$	2
ANL	C,/bit	$C = C \text{ AND NOT bit}$	2
ORL	C,bit	$C = C \text{ OR bit}$	2
ORL	C,/bit	$C = C \text{ OR NOT bit}$	2
MOV	C,bit	$C = \text{bit}$	1
MOV	bit,C	$\text{bit} = C$	2
CLR	C	$C = 0$	1
CLR	bit	$\text{bit} = 0$	1
SETB	C	$C = 1$	1
SETB	bit	$\text{bit} = 1$	1
CPL	C	$C = \text{NOT } C$	1
CPL	bit	$\text{bit} = \text{NOT bit}$	1
JC	rel	Jump if $C = 1$	2
JNC	rel	Jump if $C = 0$	2
JB	bit,rel	Jump if bit = 1	2
JNB	bit,rel	Jump if bit = 0	2
JBC	bit,rel	Jump if bit = 1; CLR bit	2

CLK = 12 MHz



Mnemonic		Description	Byte	Oscillator Period
BOOLEAN VARIABLE MANIPULATION				
CLR	C	Clear Carry	1	12
CLR	bit	Clear direct bit	2	12
SETB	C	Set Carry	1	12
SETB	bit	Set direct bit	2	12
CPL	C	Complement Carry	1	12
CPL	bit	Complement direct bit	2	12
ANL	C,bit	AND direct bit to CARRY	2	24
ANL	C,/bit	AND complement of direct bit to Carry	2	24
ORL	C,bit	OR direct bit to Carry	2	24
ORL	C,/bit	OR complement of direct bit to Carry	2	24
MOV	C,bit	Move direct bit to Carry	2	12
MOV	bit,C	Move Carry to direct bit	2	24
JC	rel	Jump if Carry is set	2	24
JNC	rel	Jump if Carry not set	2	24
JB	bit,rel	Jump if direct Bit is set	3	24
JNB	bit,rel	Jump if direct Bit is Not set	3	24
JBC	bit,rel	Jump if direct Bit is set & clear bit	3	24

JBC - jump if bit (bit = 1) and clear bit (bit = 0)

Instrucțiuni de salt, lucru cu subrutine

Mnemonic	Operation	Execution Time (μ s)
JMP addr	Jump to addr	2
JMP @A+DPTR	Jump to A+DPTR	2
CALL addr	Call subroutine at addr	2
RET	Return from subroutine	2
RETI	Return from interrupt	2
NOP	No operation	1

JMP @A + DPTR
PC ← A + DPTR
↓ 8b ↓ 16b

PC ← addr

din subrutina
interrupție

Mnemonic	Description	Byte	Oscillator Period
PROGRAM BRANCHING			
ACALL addr11	Absolute Subroutine Call	2	24
LCALL addr16	Long Subroutine Call	3	24
RET	Return from Subroutine	1	24
RETI	Return from interrupt	1	24
AJMP addr11	Absolute Jump	2	24
LJMP addr16	Long Jump	3	24
SJMP rel	Short Jump (relative addr)	2	24

The Table lists a single "JMP addr" instruction, but in fact there are three—SJMP, LJMP and AJMP—which differ in the format of the destination address. JMP is a generic mnemonic which can be used if the programmer does not care which way the jump is encoded.

The SJMP instruction encodes the destination address as a relative offset, as described above. The instruction is 2 bytes long, consisting of the opcode and the relative offset byte. The jump distance is limited to a range of -128 to +127 bytes relative to the instruction following the SJMP.

The LJMP instruction encodes the destination address as a 16-bit constant. The instruction is 3 bytes long, consisting of the opcode and two address bytes. The destination address can be anywhere in the 64K Program Memory space.

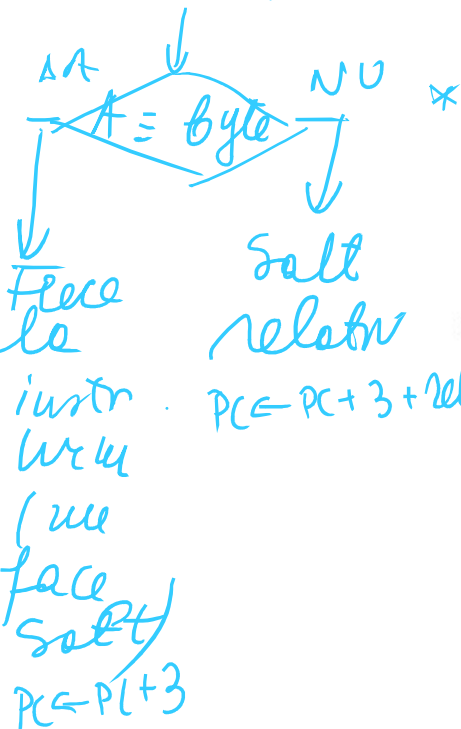
The AJMP instruction encodes the destination address as an 11-bit constant. The instruction is 2 bytes long, consisting of the opcode, which itself contains 3 of the 11 address bits, followed by another byte containing the low 8 bits of the destination address. When the instruction is executed, these 11 bits are simply substituted for the low 11 bits in the PC. The high 5 bits stay the same. Hence the destination has to be within the same 2K block as the instruction following the AJMP.

CJNE - compare and jump if not equal

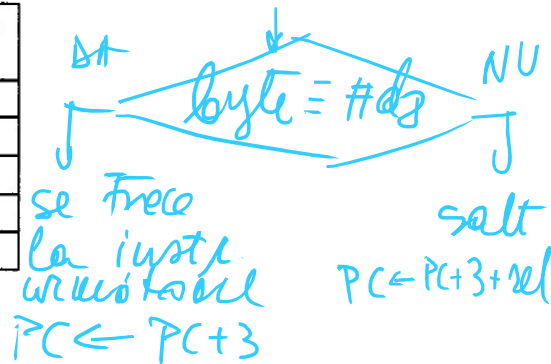
Instructiuni de salt conditionat

CJNE byte, #data, rel

CJNE A, byte, rel



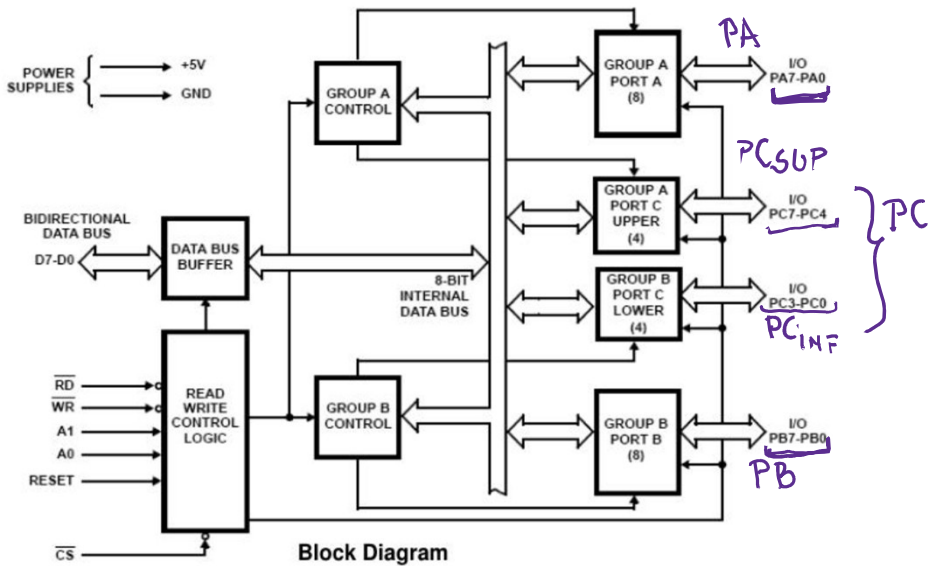
Mnemonic	Operation	Addressing Modes				Execution Time (μs)
		Dir	Ind	Reg	Imm	
JZ rel	Jump if A = 0					2
JNZ rel	Jump if A ≠ 0					2
DJNZ <byte>,rel	Decrement and jump if not zero	X		X		2
CJNE A,<byte>,rel	Jump if A ≠ <byte>	X			X	2
CJNE <byte>,#data,rel	Jump if <byte> ≠ #data		X	X		2



Mnemonic	Description	Byte	Oscillator Period
PROGRAM BRANCHING (Continued)			
JMP @A+DPTR	Jump indirect relative to the DPTR	1	24
JZ rel	Jump if Accumulator is Zero	2	24
JNZ rel	Jump if Accumulator is Not Zero	2	24
CJNE A,direct,rel	Compare direct byte to Acc and Jump if Not Equal	3	24
CJNE A,#data,rel	Compare immediate to Acc and Jump if Not Equal	3	24

Mnemonic	Description	Byte	Oscillator Period
PROGRAM BRANCHING (Continued)			
CJNE Rn,#data,rel	Compare immediate to register and Jump if Not Equal	3	24
CJNE @Ri,#data,rel	Compare immediate to indirect and Jump if Not Equal	3	24
DJNZ Rn,rel	Decrement register and Jump if Not Zero	2	24
DJNZ direct,rel	Decrement direct byte and Jump if Not Zero	3	24
NOP	No Operation	1	12

8255: Programmable Peripheral Interface



interfata MS MEMORY MAPPING

- 40 pins; NMOS (CMOS) +5V.
- 3 porturi paralele x 8 biti = 24 pins I/O (PA, PB, PC)
- Porturile sunt grupate in grupe Group A, Group B

Gr. A = { PA, PCsup }
Gr. B = { PB, PCinf }

PA₀ ÷ PA₇
PB₀ ÷ PB₇
PC₀ ÷ PC₇

interfata user (24 pins I/O)

INTEL: 8255 (PPI), 8253 (TIMER), 8251 (USART), 8259 (PIC), 8257 (DMB)
↓
PC x86 x2 x1 8250 (2 x 8257) 8259 8257

SOUTH BRIDGE (ASIC),
ZiLOG: Z80, PIO, CTC, SIO
MOTOROLA 6800, 6800p 8b 16b
are I/O (Programabil) - x memory mapping

- 3 moduri operare (gr. A)
(gr. B - 2 moduri de operare)

moduri: 0 - mod operare port paralel
1 - comunicatii paralele unidirect.
2 - -11- -11- bidirectionale

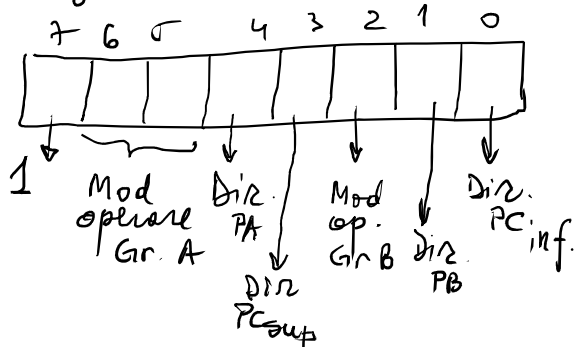
modul 1, 2 - pentru conectare cu printere
interfata cu MS
D₀ ÷ D₇ (B₀ ÷ B₇)
A₀, A₁ - 2² = 4 locatii (cine are 4 adrese la care raspunde)
RD, WR - (mag. control) RESET (activ 1)
CS - chip select; validare comunic CPU - 8257

$\overline{CS} = 0$

$A_1 A_0$	$\overline{RD}$	$\overline{WE}$	Operatie CPU cu 8255
0 0	1	0	scrie PA
0 1	1	0	scrie PB
1 0	1	0	scrie PC
1 1	1	0	scrie RCC (reg. cuv. com) (*)
0 0	0	1	citire PA
0 1	0	1	citire PB
1 0	0	1	citire PC
1 1	0	1	- invalida -
x x	1 1		- invalida (neop.)

Programare circ. 8255

1) Programare cuv. com. în RCC → a)



(primul format al cuv. de com)

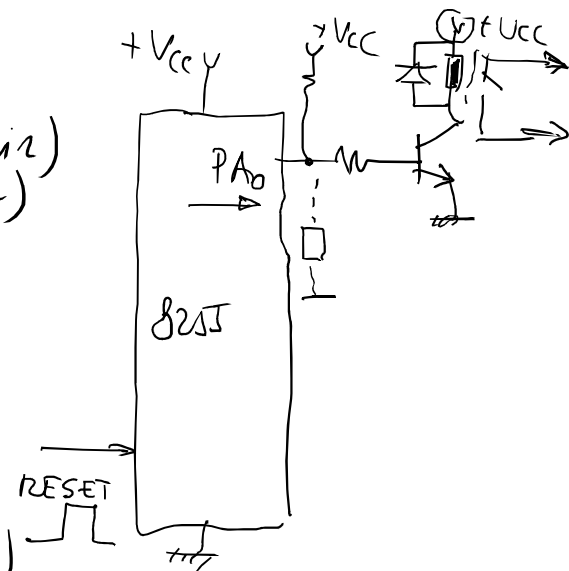
→

$b_6 b_5$	Operare Gr. A
0 0	mod 0 oper (port par.)
0 1	mod 1 (com par unidir)
1 0	mod 2 (com par. bidir)
1 1	invalida

Directionalitate

0 - output
1 - input

b_2	Operare Gr. B
0	mod 0 oper (port paralel)
1	mod 1 oper.

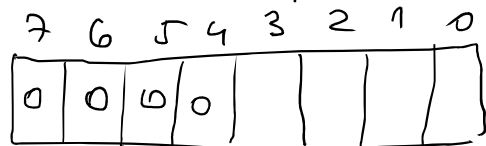


OBS

- a) - Cuv. de com se transmite pe adresa RCC (*)
 - RESET - se pierde programarea 8255
 → b) - se operează cu liniile de port folosind adresele PA, PB sau PC

Al 2-les format al cu. de comando'

- operează doar asupra PC și deci anterior acestei a fost programul cu primul format al Cw. de cum să opereze în mod 0 (PC_{sup} și PC_{inf}) iar direcționalitatea PC să fie output
- avantaj - operare individuale cu linile PC



Specifică
primul din model
PC care
va fi actionat

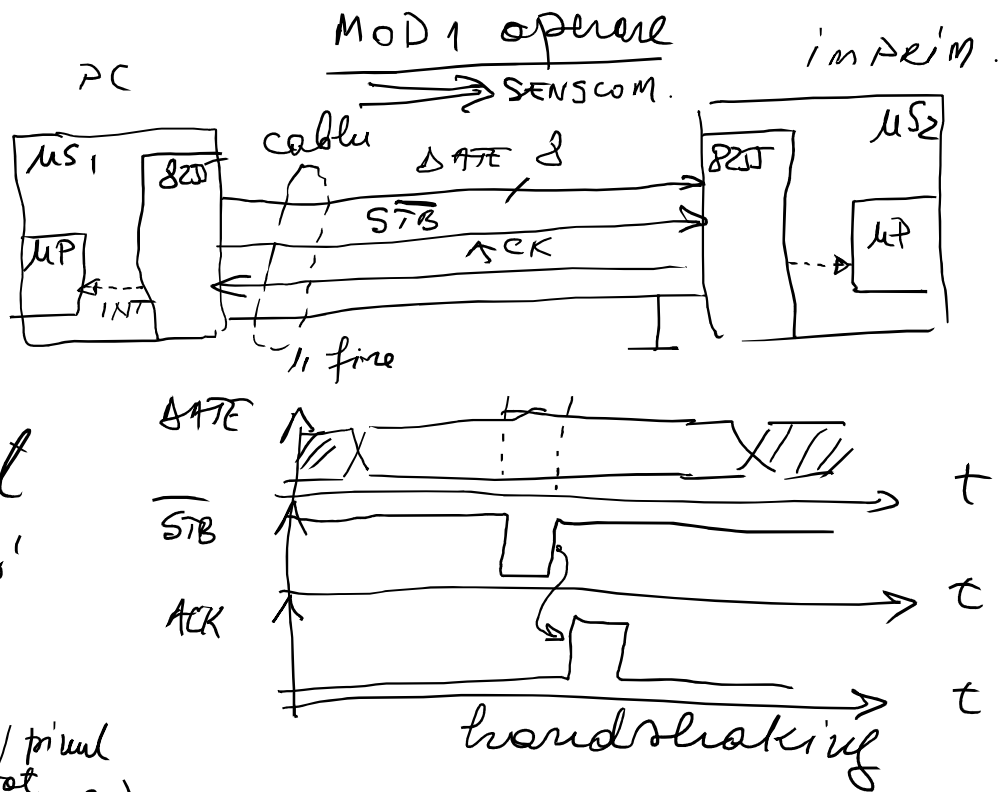
0 (left / right
 spectral
 tree in 0)

1 (left / right
 spectral
 tree in 1)

b_3	b_2	b_1	Pin specification	Hex
0	0	0	PC0	
0	0	1	PC1	
0	1	0	PC2	
0	1	1	PC3	
1	0	0	PC4	
1	0	1	PC5	
1	1	0	PC6	
1	1	1	PC7	

$$0000 \underbrace{1010}_{(2)} = 0Ah \Rightarrow PC_5 \leftarrow 0$$

$$00001011_{(2)} = 0Bh \Rightarrow PC_5 \leftarrow 1$$



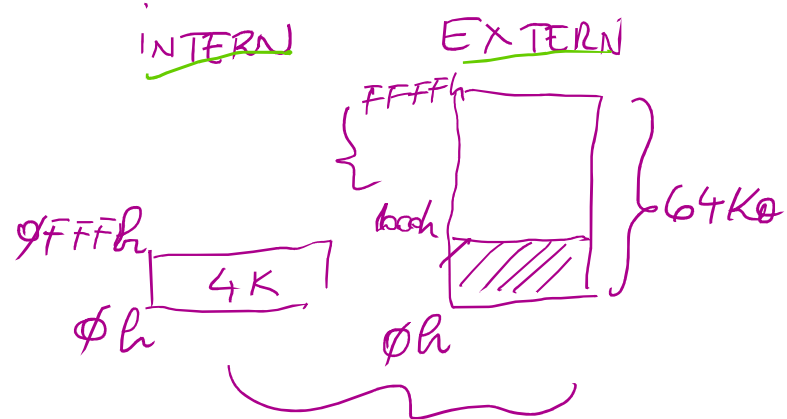
- ⊕ Conectarea circuitelor de memorie la magistralele 8051
- * - memorie de programe (externă) - 64 Ko - $\overline{PSEN}$ ($\overline{PSEL}$)
 - * - memorie de date (externă) - 64 Ko - $\overline{RD}$; $\overline{WR}$
- $\overline{EA} = 0$ (configurare lucru cu mag. externă)

MEM PROG

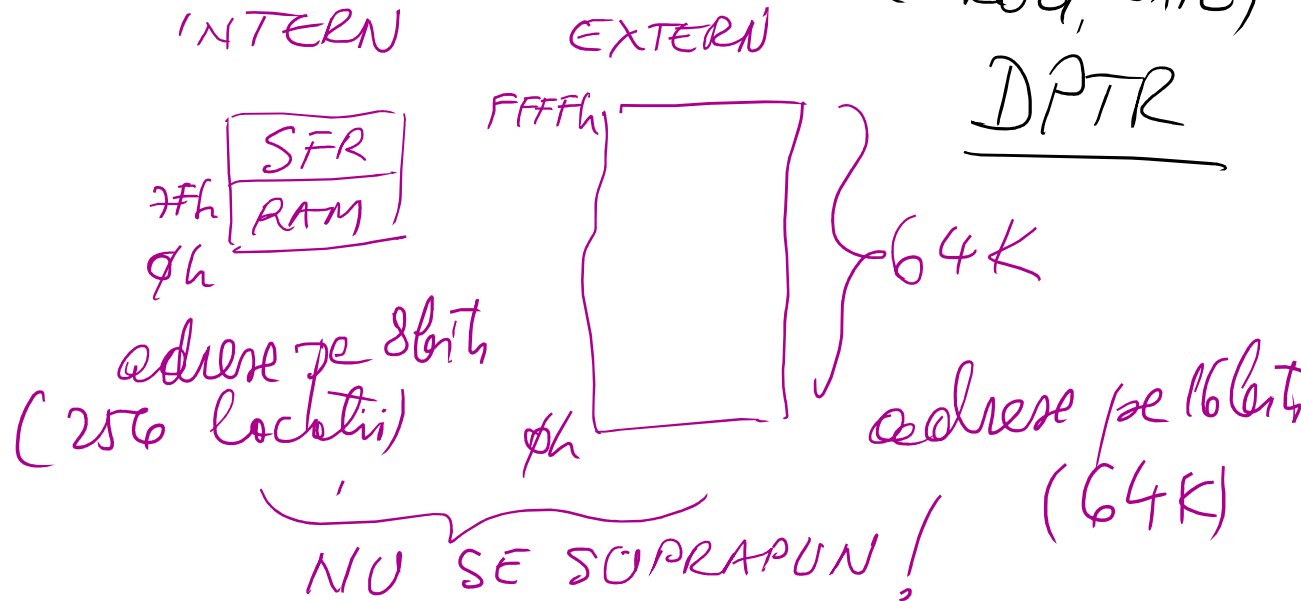
MEM DATE

LUCRU
CU MEM. EXT
(PROG, DATE)

DPTR



Primi 4K vin
din memor. programe
interne!



1) Conectarea mem. de progr. externe -

Problema

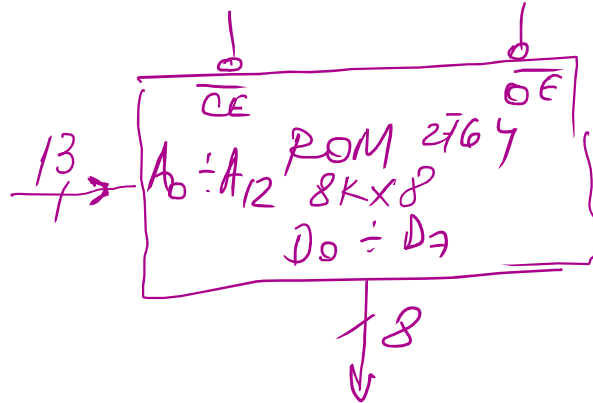
Să se conecteze la un microcontroller 8051 - 8K ROM în spațiul mem. de progr. externe începând cu adresa 1000h (adresa va fi completă), folosind soluție circuitul ROM 2764 cu organizare 8Kx8.

I.1. Prezentare circuit cu care realizăm implementarea

ROM 8Kx8
(2764)

$$8K = 2^3 \cdot 2^{10} = 2^{13}$$

$\Rightarrow$ 13 linii
adrese



I.2 Calculul nr. circuite necesar

$$\frac{8K \times 8}{8K \times 8} = 1 \times 1 = 1 \text{ c'c} \quad (\text{ROM}_0)$$

I.3 - Tabelul de adrese

0 0 0 1 1 1 1 1 1 1 1 1 1 1 1 1
 $A_{15} A_{14} A_{13} A_{12} A_{11} A_{10} A_9 A_8 A_7 A_6 A_5 A_4 A_3 A_2 A_1 A_0$

in locatia intr-un
 circ ROM 8Kx8

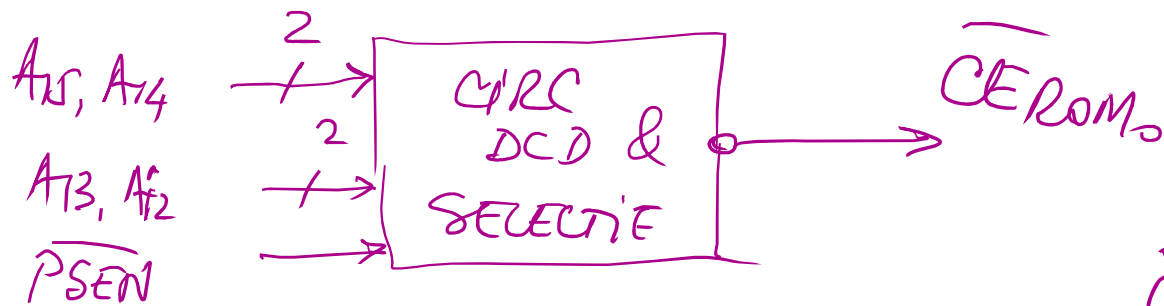
																Adresa	Circuit
0	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	1000h	ROM ₀
0	0	1	0	1	1	1	1	1	1	1	1	1	1	1	1	2FFFh	

zone I
 zone II

zone III (13 linii adrese)

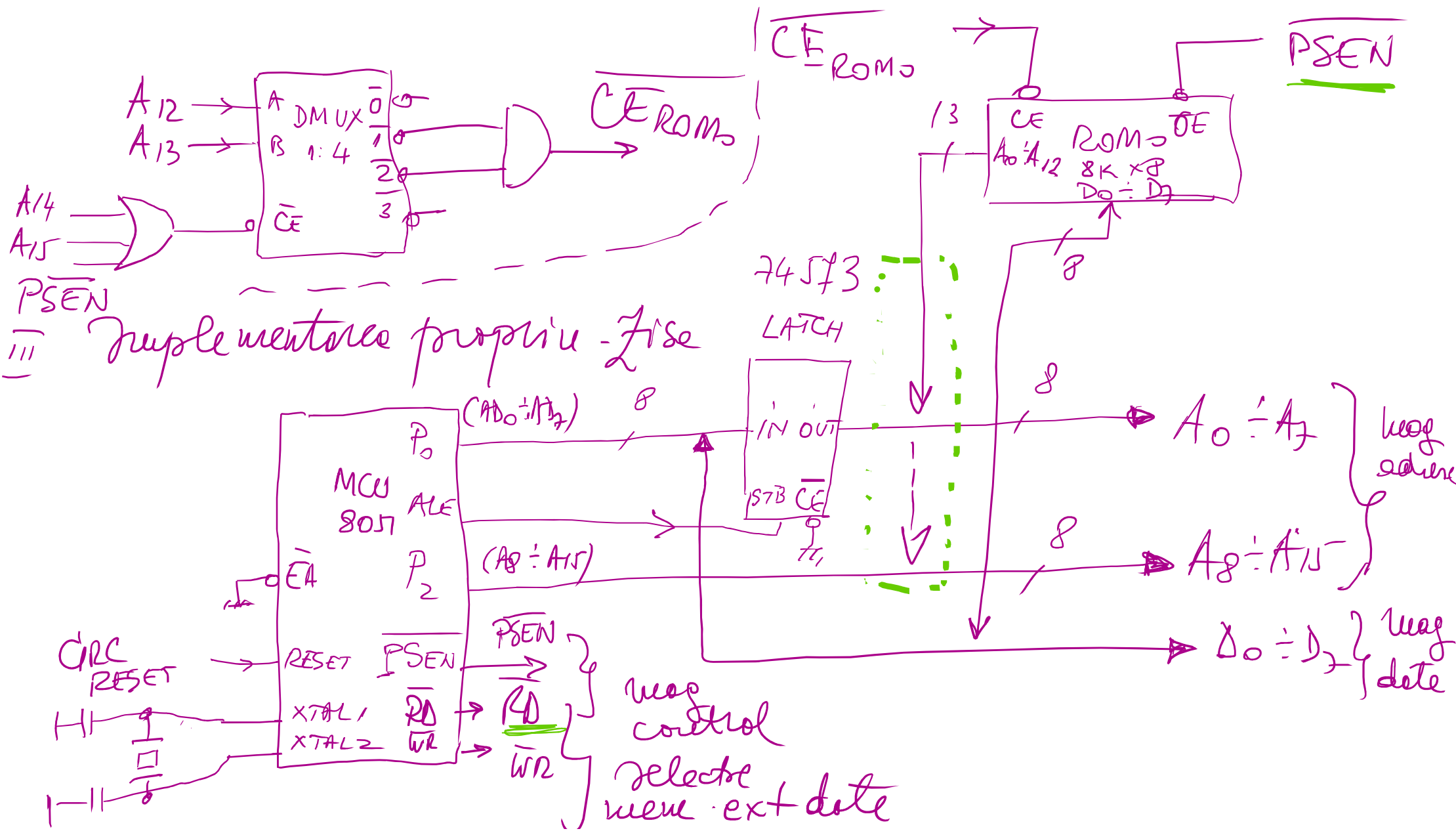
II Circuitul de decodare si selectie

Sch. bloc



=> MUX 1:4

- circ trebuie sa
 decodifice 2 comb.
 pe linia $A_{13} A_{12}$
 => MUX-ul folosit
 trebuie sa aiba 2
 intrari selectie (4 iesiri) =>



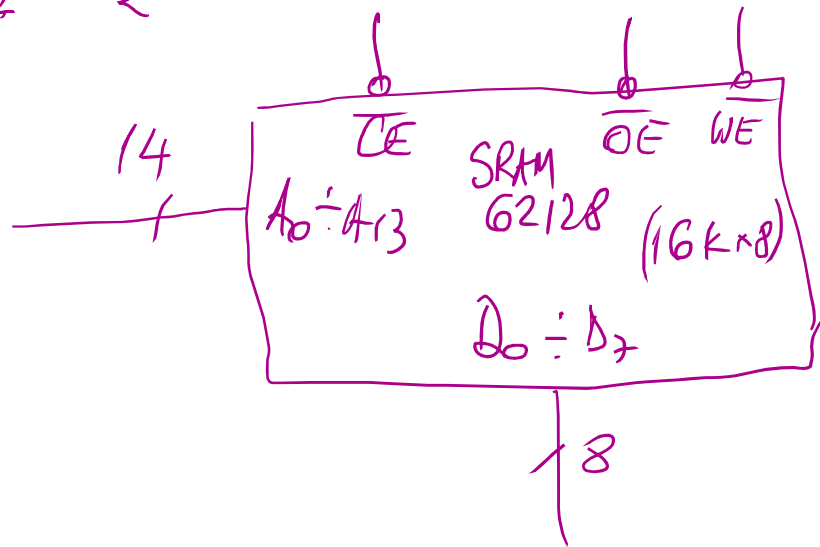
2) Conectarea memoriei de date externe —

Problema:

Se va conecta 16 Kb memorie SRAM la microprocesor folosind circuitul 62128 (16K x 8). Adresa de început a zonei de memorie 500h (adresa va fi completă)

Soluție

I.1 Prezentarea circuitului 62128 (SRAM)



16K x 8

$2^4 \cdot 2^{10} = 2^{14}$ - linii de adresare

I.2. Calculul nr. circuite necesare

$$\frac{16K \times 8}{16K \times 8} = 1 \times 1 = 1 \text{ circuit SRAM (SRAM}_0\text{)}$$

I.3 Tabelul de adrese (mem. de date ext - 64 Ko) → nr. locatii in SRAM

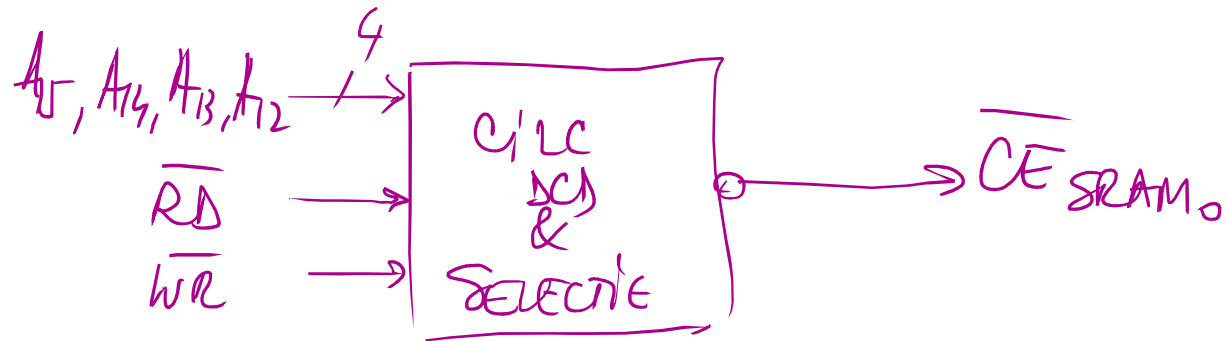
A_{15}	A_{14}	A_{13}	A_{12}	A_{11}	A_{10}	A_9	A_8	A_7	A_6	A_5	A_4	A_3	A_2	A_1	A_0	ADR HEXA	Circuit	
0	1	0	1	1	0	0	0	0	0	0	0	0	1	0	0	0	5000h	} SRAM ₀
1	0	1	0	0	1	1	1	1	1	1	1	1	1	1	1	1	8FFFh	

Z_{II}

Z_{III} (14 biti)

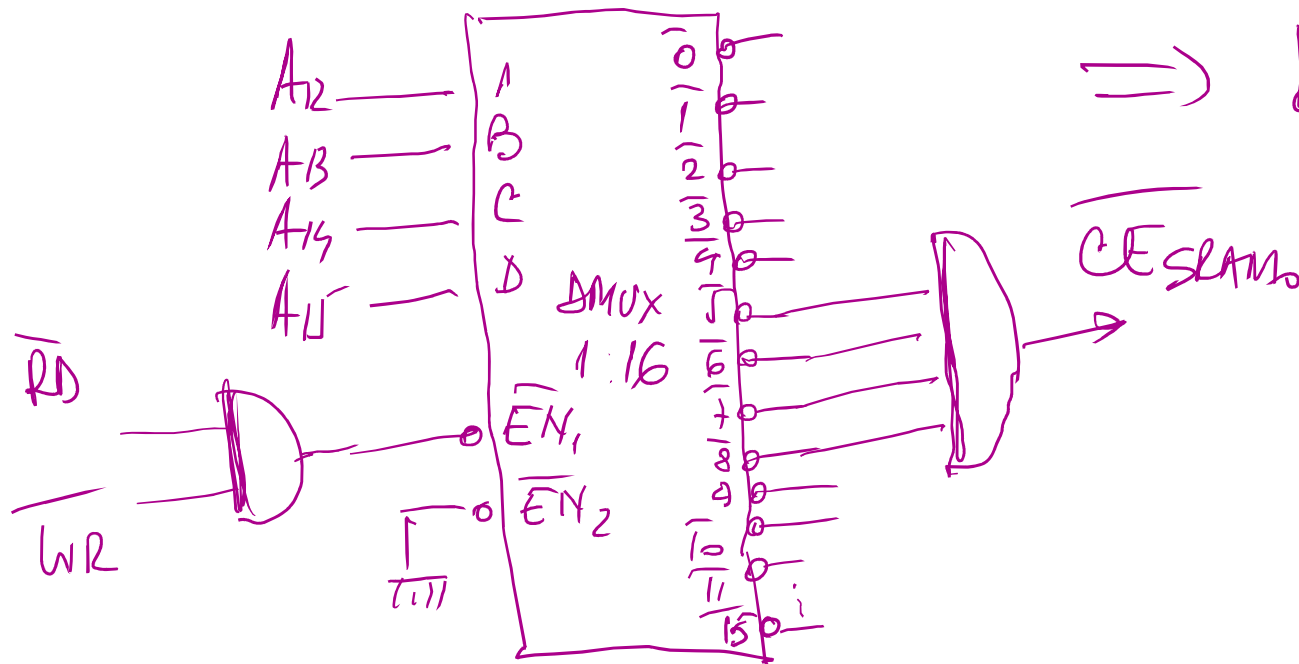
nu exista Z_I !

II Circuitul de selectie



- 4 semnale selectie
(zona II tabel)

- 4 combinatii care trebuie identificate



$\Rightarrow$ DMUX { 4 semnale selectie
 { care 4 intrari

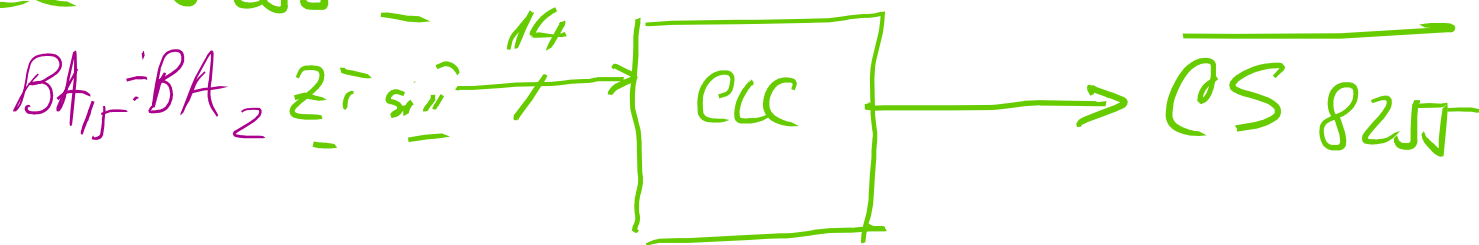
$\Rightarrow$ DMUX 1:16
(care 4 intrari selectie)

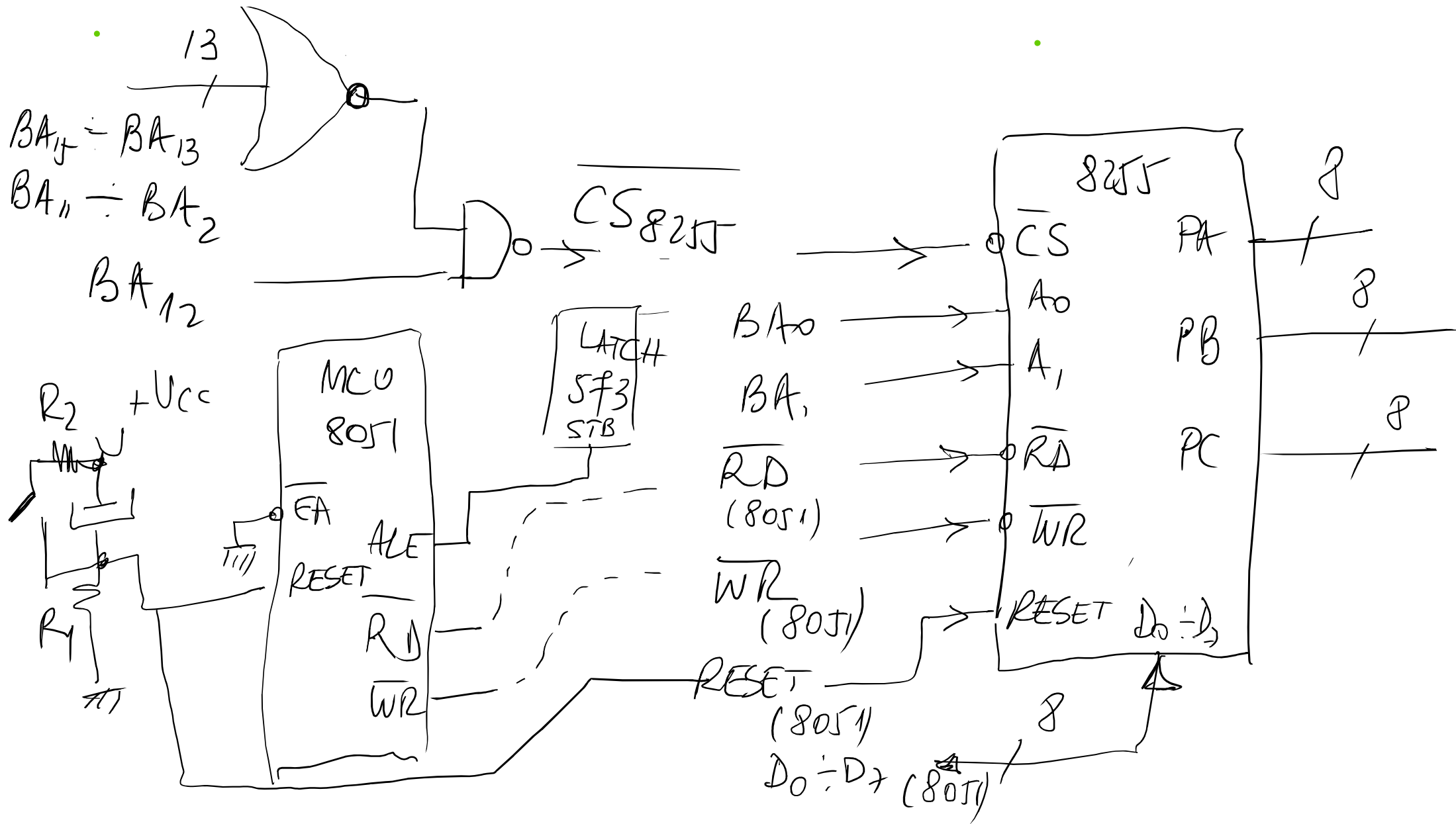
Conectarea circuitului 8255 în spațiul adreselor
memoriei de date extern de la MCU 8051 -

Pas 1

- 1/ Se întocmește tabelul de adresare - se pun
în evidență zonele de biți
- în particular în problemă
L₂ și Z₃ coincid -
2/ Proiectarea circuitului de
decodare a adreselor și selecte
a circ 8255 -

biți nemodificați
biți selecte
biți care se aplică
circuitului



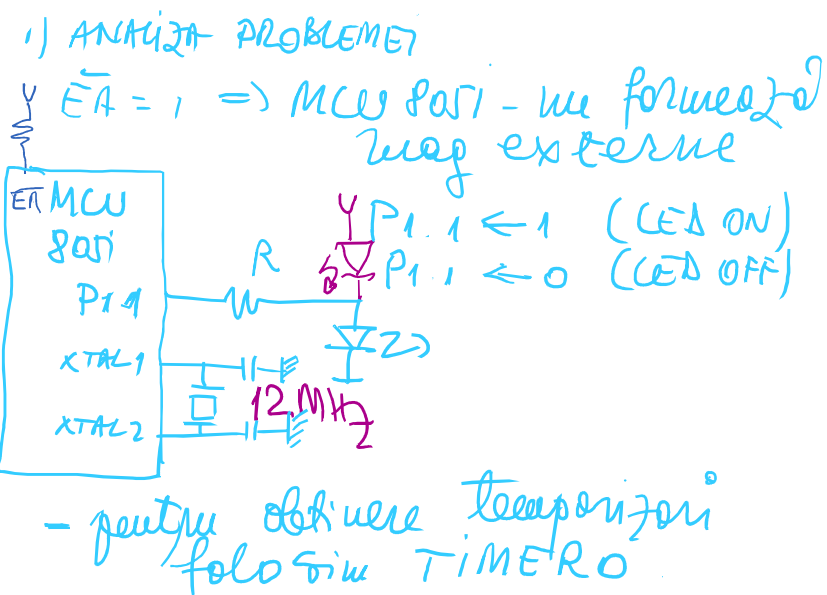


Problema 1. Un MCU 8051, configurat cu $\overline{EA} = 1$ si avand cristalul de cuarț de 12 MHz, are conectat la linia P1.1 un LED cu catodul la masa. Realizati programul in limbaj de asamblare 8051 prin intermediul caruia LED-ul se aprinde timp de 1 s si este stins 0,5 s, dupa care acest proces se reia.

Solutie

Metodologie de rezolvare:

- Analiza problemei (conceptul de rezolvare)
- Programarea circuitelor I/O
- Elaborare Schema logica (organigrama)
- Implementare in limbaj de asamblare



TIMER 0
- Regim lucru temporizator
 $TMOD \ C/\overline{T} = 0$

- $12MHz \rightarrow \begin{cases} 0,5s \ (2Hz) \\ 1s \ (1Hz) \end{cases}$

$1MHz \rightarrow 1\mu s$

$1MHz \rightarrow 1Hz \ (\frac{10^6}{1})$

$1MHz \rightarrow 2Hz \ (\frac{10^6}{5 \cdot 10^5})$

$\frac{1MHz}{50000} = 20Hz \ (50\mu s)$

- operam. in interrupturi generate de TIMER 0 cu frecvente:

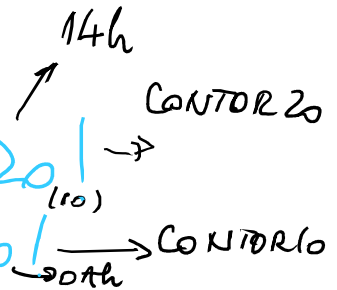
$\Rightarrow$ mod de operare pe 16 biti
MOD 1 OPERARE TIMER 0'

$\rightarrow$ se impune divizare suplimentară (prin SW)

Aleg divizare la canal TIMER 50.000 (10)

- pentru a ajunge la 106 se adaugă divizare supl. cu 20 (10)

- pentru a ajunge la 51200 se adaugă divizare supl. cu 10 (10)



2/ Programare cîc i/o

- TIMER 0
- LOGICA INTR
- P1.1 / $P_{1.1} \leftarrow 1$ (LED ON)
 $P_{1.1} \leftarrow 0$ (LED OFF)

TIMER

- TH_0, TL_0 ?

$\frac{1\text{MHz}}{20\text{kHz}} = \frac{10^6}{20} = 50.000(10)$

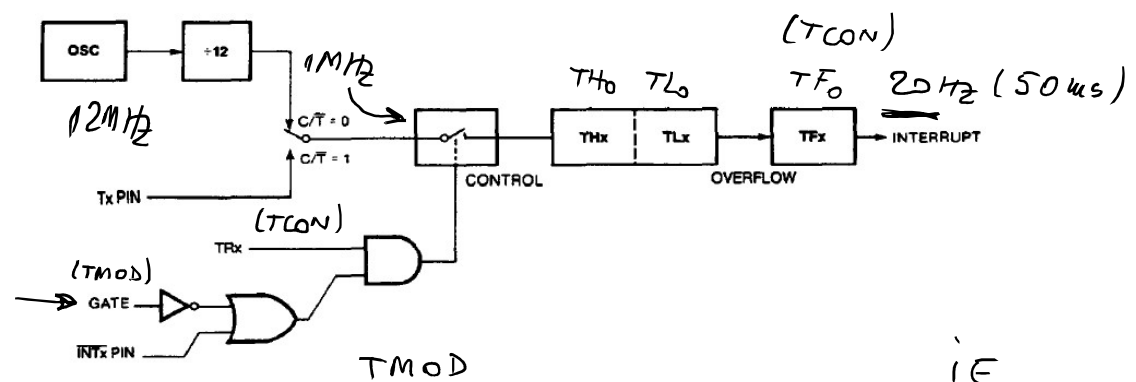
$65536_{(10)} - 50.000_{(10)} = 15536_{(10)} = 3CB0H$
 (stare finali) (nr star) (stare initiale)

- * $TH_0 \leftarrow 3Ch$; $TL_0 \leftarrow B0h$
- * $TMOD \leftarrow 01h$
- * $TCON \leftarrow 10h$

INTERRUPT

$IE \leftarrow 82h$

Subrutina tratur intr TIMER0: 000Bh



	7	6	5	4	3	2	1	0
	GATE	C/T	M ₁	M ₀	GATE	C/T	M ₁	M ₀
	Timer 1				Timer 0			
	0	0	0	0	0	0	0	0

TCON

	7	6	5	4	3	2	1	0
	TF ₁	TR ₁	TF ₀	TR ₀	Folositi la logica de intreruperi			
	0	0	0	1				

$00010000 \Rightarrow 10h$

15536 / 16 = 971 / 16 = 60 / 3 = 20

23h 1Bh 13h 000Bh 0003h

↑ ↑ ↑ ↑ ↑

EA ES ETI EXI ETO EXO

MSB ↓ 1 0 0 0 1 0 = 82h

3)

PP
progr
principal

Initializare
- CMC I/O
(TIMER0, INTR)
- variabile

⊗ Actiuni pt.
= aprindere LED
stingere
- Condusctia
suplimentare
 timp

~~TIMER0~~ 50ms

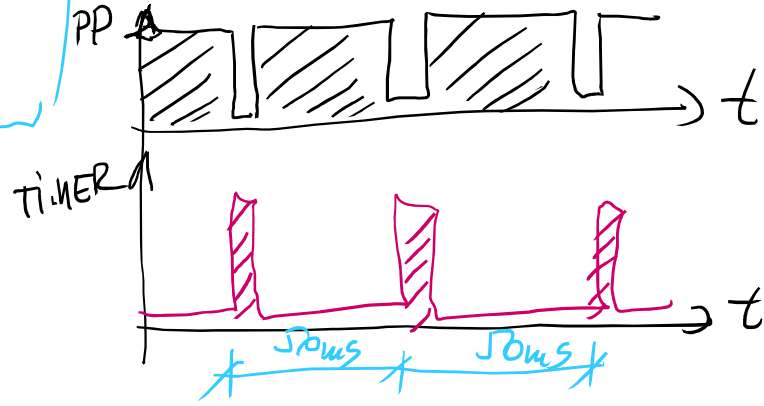
SEM-TIME_SOMS ← 1

RESTART
NUMARARE
TIMER0

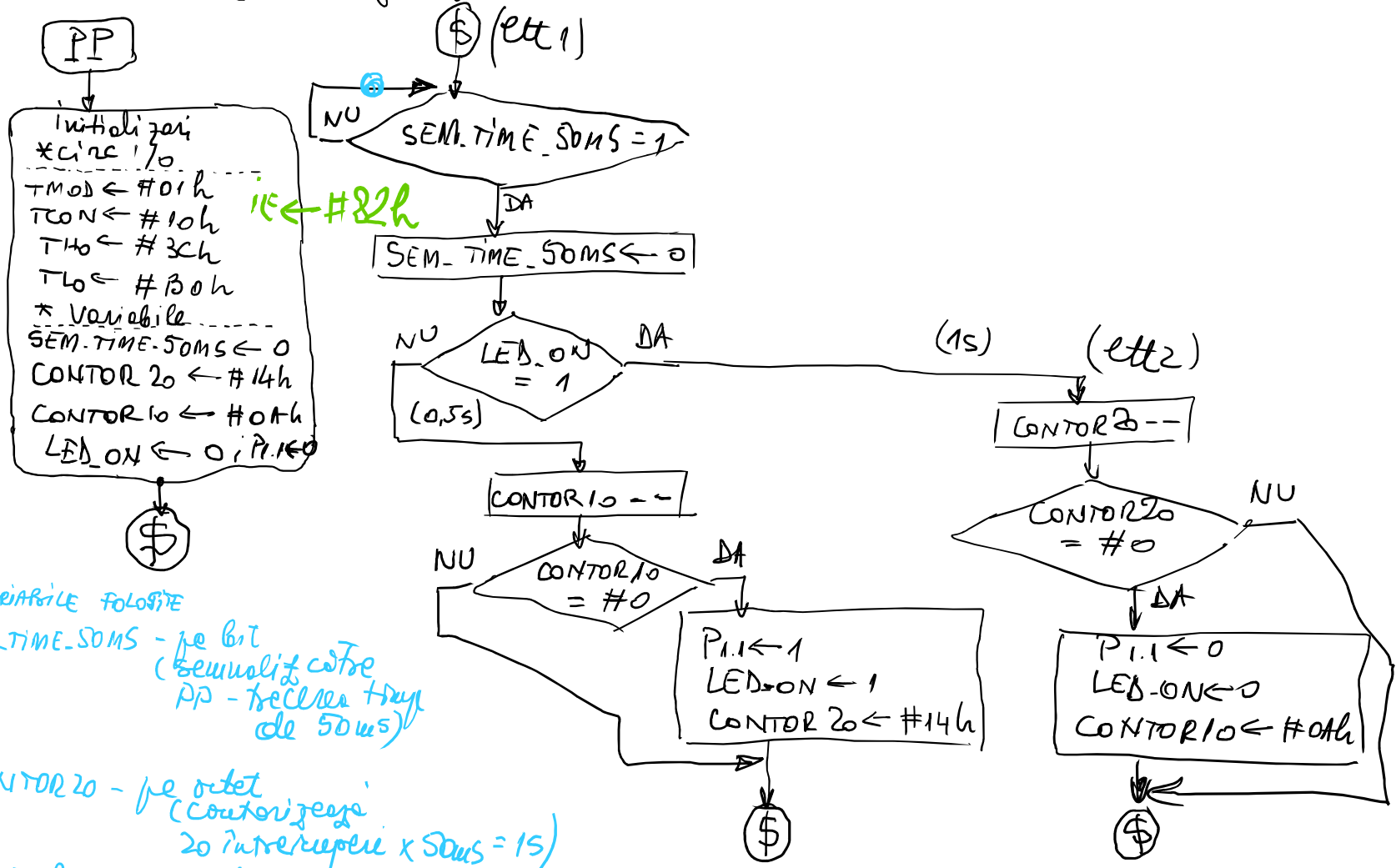
RET

INT TIM0
SEM-TIME_SOMS ← 0

ROVENIRE
tCPU



3) Schéma logique (organigramme)



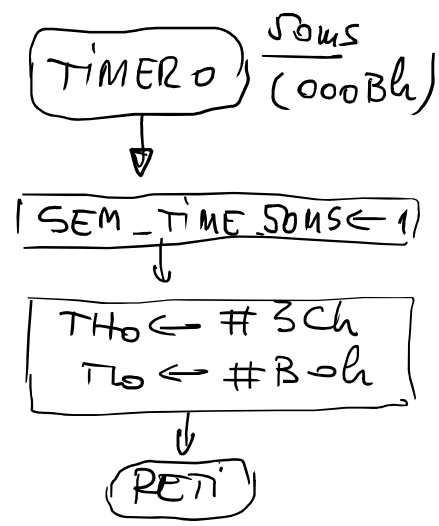
VARIABLE FOLOSITE

SEM.TIME.S0MS - pe bit
(semnaliz catre PP - fiecare timp de 50ms)

CONTOR20 - pe octet
(contorizeaza 20 interupere x 50ms = 1s)

CONTOR10 - pe octet
(contorizeaza 10 intr. x 50ms = 0.5s)

LED_ON - imagine SW pt LED (1-ON, 0-OFF)



4) LIMBAJ ASM51

• ORG 0h

START: LJMP 100h

SEM-TIME-50MS BIT 00h

LED-ON BIT 01h

CONTR10 DATA 30h

CONTR20 DATA 31h

• ORG 100h

MOV TMOD, #01h

MOV TCON, #10h

MOV TH0, #3Ch

MOV TL0, #B0h

MOV IE, #82h

CLR SEM-TIME-50MS

MOV CONTR1, #0Ah

MOV CONTR2, #14h

CLR LED-ON

CLR P1.1

ett1: JNB SEM-TIME-50MS ett1

CLR SEM-TIME-50MS

JB LED-ON ett2

DEC CONTR10

CJNE CONTR10, #00, ett1

SETB P1.1

SETB LED-ON

MOV CONTR20, #14h

SJMP ett1

ett2: DEC CONTR20
CJNE CONTR20, #00h, ett1

CLR P1.1

CLR LED-ON

MOV CONTR10, #0Ah

SJMP ett1

• ORG 000Bh

000Bh: SETB SET-TIME-50MS

MOV TH0, #3Ch

MOV TL0, #B0h

RET

• END

END 22 apr 2026